CLIO

Design, Implementation, and Evaluation of a
Programmable Virtual Quantum Processing Unit

Advaith Praveen
Archeum Studios

Definitive research artifact · Built from the verified repository implementation and retained evidence

Artifact map

Abstract And Introduction

Architecture And Isa

Correctness

Discussion Limitations Conclusion

Engine And Runtime

Experimental Results

External Baseline

Methodology

Replay And Compatibility

Resource And Trace Evaluation

Studio

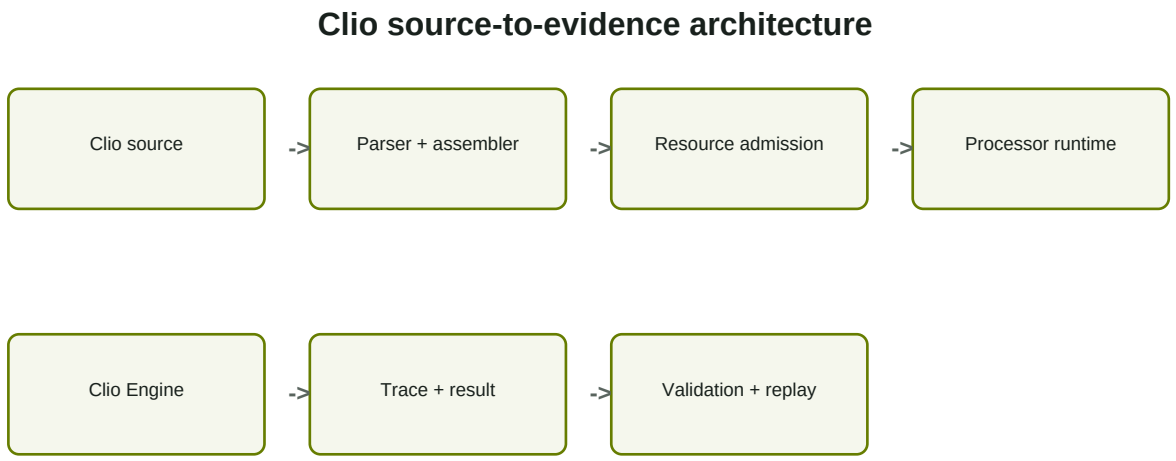


Figure 1. The owned Clio pipeline from source through evidence.

Abstract

Clio VQPU is a software-defined virtual quantum processing architecture with an owned instruction set, processor-state model, state-vector engine, hybrid classical-quantum runtime, assembler, tracing and replay model, resource admission, validation suite, developer interfaces, and visual processor laboratory. This work describes the design and implementation of Clio and evaluates its correctness, observability, resource behavior, replay compatibility, and performance scaling. The built-in engine is independent of external quantum frameworks. Exact differential evaluation against Qiskit Statevector covers six algorithm families with maximum observed amplitude error below `7e-16` under matched little-endian semantics. Sixty-seven automated Rust tests and generated numerical properties exercise gates, control flow, measurement, resource traps, algorithms, observation, trace bounds, and replay. A ten-repetition release-build protocol retains 330 measurements with environment and checksum provenance. Clio is a classical simulation system and makes no claim of physical quantum computation or quantum advantage.

Introduction

Gate-model simulators usually present circuits as library objects submitted to a simulator. That abstraction is useful but leaves processor-like concerns-typed machine state, explicit program counters, classical control, traps, admission, instruction traces, and replay compatibility-distributed across framework layers. Clio investigates whether those concerns can form one coherent virtual processor architecture without outsourcing its identity to an established quantum framework.

The contribution is an integrated artifact rather than a wrapper: Clio Assembly is parsed and assembled into a typed instruction stream; Clio Runtime executes the stream against architectural state; Clio Engine owns double-precision state-vector evolution and measurement collapse; Clio Trace records instruction transitions; Clio Resource rejects inadmissible programs before allocation; and Clio Studio exposes the real runtime through a local visual laboratory.

The central research question asks whether a software-defined VQPU can remain programmable, inspectable, reproducible, and resource-aware while maintaining correctness against analytic and established-simulator baselines. The answer is bounded by the supported ISA and tested scales. Evidence supports numerical and algorithmic correctness for the implemented subset, exact same-identity replay, useful machine-level inspection, and conservative state-memory admission. It does not establish hardware behavior, quantum advantage, or general equivalence beyond evaluated programs.

The artifact contributes: (1) a compact hybrid classical-quantum ISA; (2) explicit processor and qubit-lifecycle semantics; (3) an independent state-vector backend; (4) deterministic seeded execution and structured replay; (5) resource and trace admission; (6) analytic, property, and Qiskit differential validation; (7) a CLI, Rust SDK, and Studio interface; and (8) a retained reproducibility package linking raw evidence to figures and claims.

Architecture and Instruction Set

Clio uses a source-to-machine pipeline: text is parsed into directive and instruction records, labels are resolved during assembly, semantic allocation state is checked, a conservative execution plan is admitted, and the runtime performs fetch/decode/execute transitions. The typed executable retains source spans, metadata, the required qubit count, branch properties, and a source SHA-256 identity.

Architectural state includes a program counter, lifecycle status, sixteen signed 64-bit classical registers, sixteen three-state measurement registers, allocated logical qubits, comparison state, instruction counter, shot index, and total shots. The quantum state belongs to the selected backend but is addressed by typed logical qubit identifiers. Qubit zero is the least-significant state-vector bit.

The ISA includes lifecycle operations, standard single- and multi-qubit gates, arbitrary-axis rotations, controlled phase, measurement, checked classical arithmetic and logic, explicit comparison, conditional and unconditional branches, and `HALT`. Observation reports are debugger services rather than instructions, preventing state inspection from changing program semantics.

Allocation is contiguous and release is deliberately restrictive. `QFREE` may release only the highest logical qubit and only when its probability of one is within numerical zero tolerance. This permits safe vector truncation without an implicit partial trace. Reset samples and collapses when necessary, applies X after outcome one, and records the reset outcome.

Classical arithmetic traps on overflow, zero divisors, the signed minimum divided by minus one, and invalid shifts. Branches require a valid comparison where applicable. All branch targets are resolved before execution. Backward branches are permitted but admitted against the host's entire instruction ceiling and remain subject to wall-clock limits.

Correctness and Differential Validation

Clio combines analytic known-answer workloads with generated numerical properties. These layers catch different failures: known answers expose algorithm or endianness mistakes, inverse tests expose non-unitary updates, invariant checks expose normalization and reduced-state errors, and dense-reference comparison exposes indexing defects that could survive self-consistency tests.

The current independent-reference test applies the same generated `RY` matrices through a full output-index matrix-vector rule, rather than Clio Engine's paired in-place traversal. Across 256 operations on four qubits, every complex amplitude agrees within `1e-12`. Generated inverse tests cover 768 rotations across widths one through six and recover each initial state within the same tolerance. Five-qubit generated states retain unit total probability; every single-qubit reduced state has unit trace and Hermitian coherence.

Algorithm validation is performed through the complete parser, assembler, resource planner, runtime, engine, and result validator. The two-qubit Grover program returns its marked state on every shot. The seeded three-qubit workload exceeds its frozen threshold. The QFT program applies general controlled phases and its inverse before recovering basis state `101` exactly. These observations support correctness only for the implemented gates, tested scales, declared numeric tolerance, and frozen runtime semantics.

Discussion

Clio demonstrates that processor abstractions can make hybrid quantum simulation more explicit. Program counters, typed registers, traps, admission, and traces provide stable locations for debugging and research evidence. The cost is additional parsing, planning, state-machine transitions, and trace storage compared with direct engine calls. Retained benchmarks quantify those costs for the tested machine rather than assuming they are negligible.

The restrictive qubit lifecycle trades flexibility for transparent correctness. A density-matrix backend or explicit partial-trace operation could safely support arbitrary release, but silently discarding entanglement would violate the current state contract. Likewise, debugger observations remain host operations to avoid conflating scientific measurement with architectural execution.

Limitations

Clio is a classical, exponentially scaling state-vector simulator. It is not a physical QPU and does not provide quantum speedup. The ISA is finite, the external baseline covers pure-state subsets rather than matched dynamic teleportation, and evaluation occurs primarily on one macOS arm64 host. Floating-point tolerance, compiler behavior, process scheduling, and trace serialization affect results.

The ten-repetition dataset was initially collected before the repository's baseline commit and is therefore candidate evidence until recollected against the committed hash. Peak RSS is platform-native and not equivalent to heap allocation. The fixed trace-event estimate is conservative for evaluated workloads but requires adversarial monitoring as schemas evolve. Replay packages provide integrity, not producer authenticity or signatures.

Studio is local-only and unauthenticated. It must not be exposed directly on a network. The product does not currently include Python bindings, physical quantum hardware adapters, call/return instructions, arbitrary qubit release, or distributed execution. These absences define the released architecture rather than promises for another product version.

Responsible Claims

Clio is described as a programmable virtual quantum processing architecture implemented in software. "Quantum" refers to simulated gate-model state and algorithms. Results establish agreement with analytic expectations and Qiskit for evaluated circuits; they do not establish physical quantum behavior, advantage, supremacy, or hardware equivalence. Performance claims identify compiler profile, host, repetition count, and workload.

Conclusion

Clio integrates a custom assembly language and ISA, explicit processor state, an independent quantum engine, a hybrid runtime, bounded tracing, deterministic replay, resource admission, validation, developer tools, a visual processor laboratory, and reproducible research evidence. The implementation executes representative algorithms and dynamic classical control, matches an established state-vector baseline at floating-point precision for six pure-state families, and exposes machine state without replacing the owned runtime. The project supports the bounded conclusion that a coherent, inspectable, and resource-aware VQPU can be designed and implemented as a complete classical software artifact.

References

1. M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press. 2. IBM Quantum, "Qiskit Statevector API," https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector. 3. P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, 2020. 4. D. P. DiVincenzo, "The Physical Implementation of Quantum Computation," *Fortschritte der Physik*, 2000. 5. Clio VQPU repository evidence, specifications, ADRs, source, and retained benchmark manifests included with this artifact.

Engine and Runtime Implementation

Clio Engine stores `2^n` double-precision complex amplitudes. Single-qubit gates traverse pairs whose indices differ in the target bit. Controlled gates inspect control and target masks; swap exchanges only indices with unequal selected bits. Every operation validates allocation and operand distinctness and checks normalization after mutation.

Measurement computes the sum of squared magnitudes for indices whose target bit is one. The runtime derives a shot-specific ChaCha8 stream from the declared seed and shot index, samples the outcome, zeros the rejected subspace, and renormalizes the selected amplitudes. Deterministic zero- and one-probability outcomes do not consume an ambiguous branch of randomness.

Runtime execution creates fresh processor and engine state for every shot. It increments logical and architectural instruction counters, enforces global instruction and wall-time limits, records mutations, and requires explicit `HALT`. Measurement strings are emitted in highest-written-register-first order. The final architectural state belongs to the last shot, while aggregate counts cover all shots.

The observation subsystem computes basis probabilities, per-qubit marginals, tensor-product Pauli expectations, squared norm, and single-qubit reduced density matrices. Studio inspection forces one bounded state-enabled shot. This inspection result is intentionally separate from the declared multi-shot result.

Trace records are schema-identified instruction transitions. For up to eight qubits, state-enabled traces retain every basis amplitude after each instruction. Larger states omit complete snapshots. This bound prevents the debugger from silently multiplying trace size by an unbounded state dimension.

Preliminary Experimental Results

This chapter records smoke evidence, not final performance conclusions. The retained dataset contains 50 release-build measurements spanning ten qubit configurations, five depth configurations, five shot configurations, three trace configurations, and direct-engine versus full-runtime paths, each with two recorded repetitions after warm-up.

The generated figures report medians from the raw CSV. Qubit scaling combines fixed depth with exponentially growing state memory. Depth and shot studies isolate repeated gate execution and repeated complete program execution respectively. The trace study compares off, instruction, and bounded state-snapshot modes for the same Bell workload. Exact measured values remain in the processed CSV and are never manually transcribed into claims.

These results are sufficient to validate the evidence pipeline and reveal gross regressions. They are insufficient for stable overhead ratios, cross-machine claims, external-simulator comparisons, or inferential statistics. Final evaluation requires the ten-repetition protocol, a clean committed revision, additional peak-memory and trace-size instrumentation, and a mature external baseline under matched semantics.

External Simulator Baseline

The external baseline uses IBM Qiskit 2.5.0 `quantum_info.Statevector`. Circuits are constructed independently with Qiskit operations and compared with Clio final state snapshots. Both systems treat qubit zero as the least-significant computational-basis bit. A unit global phase is aligned before amplitude error is calculated.

Six families are currently compared: Bell, three-qubit GHZ, two-qubit Grover, QFT/inverse-QFT recovery, Bernstein-Vazirani secret `1011`, and balanced-parity Deutsch-Jozsa. Reports retain maximum complex-amplitude error, total-variation distance, Jensen-Shannon divergence, and normalization error. Every retained case passes the frozen `1e-12` amplitude and probability thresholds; observed errors are at floating-point roundoff scale.

Teleportation is validated through Clio's dynamic measurement-and-feed-forward known-answer test rather than the pure-state external adapter. Extending the external baseline to matched dynamic-circuit semantics remains separate work and is not implied by the current exact-state comparison.

The adapter follows the official [Qiskit Statevector API](https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector), including its computational-basis probability and subsystem-order conventions.

Experimental Methodology

Scope and research questions

The evaluation is organized around correctness, performance scaling, trace cost, resource admission, reproducibility, and inspectability. The current evidence package answers a bounded subset: numerical invariants, independent-reference agreement for single-qubit sequences, known-answer algorithm behavior, and smoke-scale runtime trends. It does not yet establish comparison with a mature external simulator or final performance claims.

Correctness protocol

Property tests use a fixed 64-bit linear-congruential sequence so every generated operation is reproducible without an opaque test-framework seed. For widths one through six, 128 rotation operations are applied and then inverted in reverse order. Norm is checked after every operation and the recovered amplitude vector is compared component-wise with tolerance `1e-12`. A separately indexed dense matrix-vector implementation serves as an independence check against the engine's amplitude-pair traversal for 256 generated operations. Reduced density matrices are checked for unit trace and Hermitian off-diagonal structure.

Known-answer tests cover Bell and GHZ correlations, teleportation, Bernstein-Vazirani, Deutsch-Jozsa, Grover, and QFT/inverse-QFT recovery. Deterministic algorithms require exact shot agreement. The three-qubit one-iteration Grover workload uses a frozen seed and a conservative success threshold of 0.70, below its analytic probability of 25/32.

Benchmark protocol

`clio-bench` constructs workloads from checked-in source-generation rules, performs one unrecorded warm-up per configuration, and records every repetition using a monotonic clock. The smoke protocol uses two repetitions; the eventual final protocol uses ten. Families vary allocated qubits, rotation depth, Bell-state shot count, and trace level. Each raw record includes schema, family, parameter, repetition, elapsed nanoseconds, shots, qubits, executed instructions, trace events, and admitted memory estimate.

Release profile execution is mandatory for retained performance evidence. Environment metadata records operating system, architecture, compiler version, build profile, repetition count, smoke/final status, and Git commit when available. Raw CSV and generated artifacts receive SHA-256 checksums. Medians are reported with observed minima and maxima; smoke data is explicitly unsuitable for strong comparative claims or confidence intervals.

Reproduction

Run `cargo run --release -p clio-bench -- --smoke --output-dir research/benchmarks`, followed by `python3 research/benchmarks/scripts/process_results.py`. The first command regenerates raw records, environment metadata, and the raw checksum. The second derives the summary and all quantitative SVG figures. Run the full Rust quality gate before accepting evidence.

Threats to validity

The current timings come from one machine, include full SDK parsing and execution, and have only two smoke repetitions. Cache state, process scheduling, CPU frequency, compiler version, and background activity can influence measurements. Trace serialization size and peak resident memory are not yet measured. The independent dense reference is intentionally structured differently but remains repository-owned; a mature external baseline is still required before final cross-simulator claims.

Replay and Compatibility

Clio replay packages are self-contained JSON envelopes with a stable schema identifier. Each package retains the original source, typed assembled executable, complete deterministic result, engine semantic identity, RNG semantic identity, producer operating system and architecture, and a SHA-256 checksum over the semantic payload.

Verification proceeds in four stages. First, schema, engine, and RNG identities must match the running implementation. Second, the payload checksum must match. Third, rebuilding the retained source must produce the identical typed executable and source hash. Fourth, execution under the retained source directives must reproduce the complete result exactly, including measurement counts, final architectural state, and structured trace.

This contract intentionally rejects silent forward compatibility. A future engine or RNG may provide an explicit migration tool, but it cannot claim exact replay under the current identity. Producer OS and architecture are informational because deterministic same-build semantics are tested across the semantic package; performance replay is not promised.

Resource and Trace Evaluation

The admission model computes exact raw state-vector bytes as `16 x 2^n`, adds runtime headroom, and estimates retained trace storage before allocation. The evaluation serializes actual traces for the same admitted workload and records child-process peak resident memory in platform-native units.

The first probe revealed that the original 192-byte per-event trace estimate was not conservative for JSON instruction and state-snapshot events. Instruction traces used roughly 300 KB against a 172 KB estimate, while bounded state traces used roughly 528 KB. This negative result caused the per-event bound to be raised to 640 bytes. Re-evaluation estimates 573,440 bytes: 1.91 times the measured instruction trace and 1.09 times the measured state trace. The corrected estimator therefore preserves headroom for both tested modes.

Trace-off JSON still contains a small empty trace envelope, so its serialized size is nonzero even though admitted trace-event storage is zero. Peak RSS values are useful within the recorded macOS environment but are not equated with heap allocation or portable across operating systems. Broader workload shapes remain a threat to the fixed per-event bound and should be covered by future adversarial trace-size tests.

Clio Studio

Clio Studio is a local visual processor laboratory compiled into the Rust workspace. Its HTTP server binds to loopback by default and embeds all HTML, CSS, and JavaScript. There are no external assets, remote services, accounts, or telemetry. A restrictive content-security policy allows only same-origin resources.

The workspace centers the assembly editor and execution timeline. Canonical examples can be loaded without leaving the application. Build, run, and validate controls call Clio SDK endpoints. The editor reports line and column and retains local working source. Keyboard execution uses the conventional command/control-enter gesture.

The timeline exposes instruction step, mnemonic, measurement, branch outcome, and lifecycle transition. A scrubber selects one real trace event and updates the complete bounded state-vector view. Basis rows show complex amplitude and probability. The processor inspector displays status, program counter, backend, seed, shot, virtual qubits, classical registers, measurements, and resource plan. Results display empirical counts and typed validation reports.

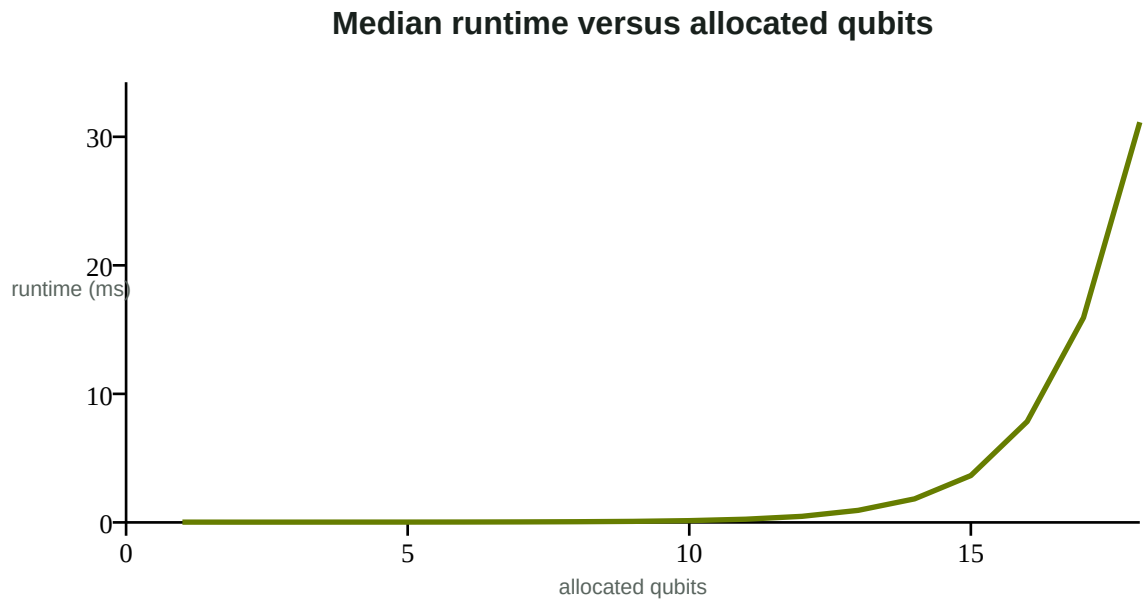
Replay export creates a real checksummed package. Import performs server-side checksum, identity, executable, and exact-result verification before loading source. Studio contains no duplicate quantum simulator. Presentation logic never fabricates amplitudes or reimplements gate semantics.

The current product is a desktop-style local web application rather than a signed native shell. This preserves a small trusted surface and cross-platform operation but delegates window integration to the browser. Packaging includes the Studio binary and launch instructions.

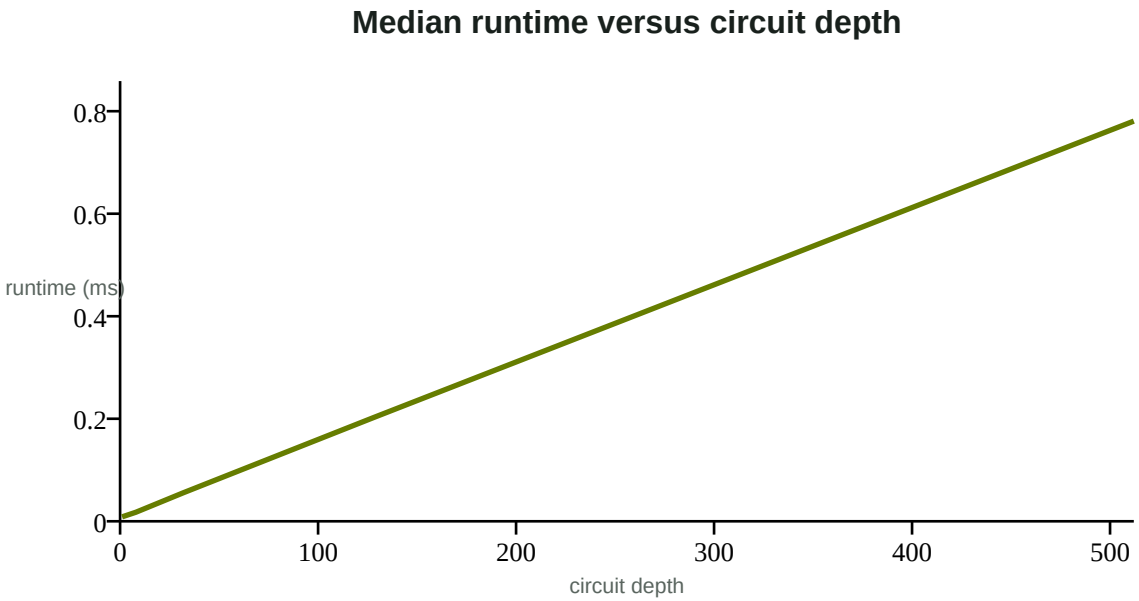
Product and Evaluation Figures

Clio Studio displaying a real 1,024-shot Bell execution and a separate seven-step bounded inspection trace.

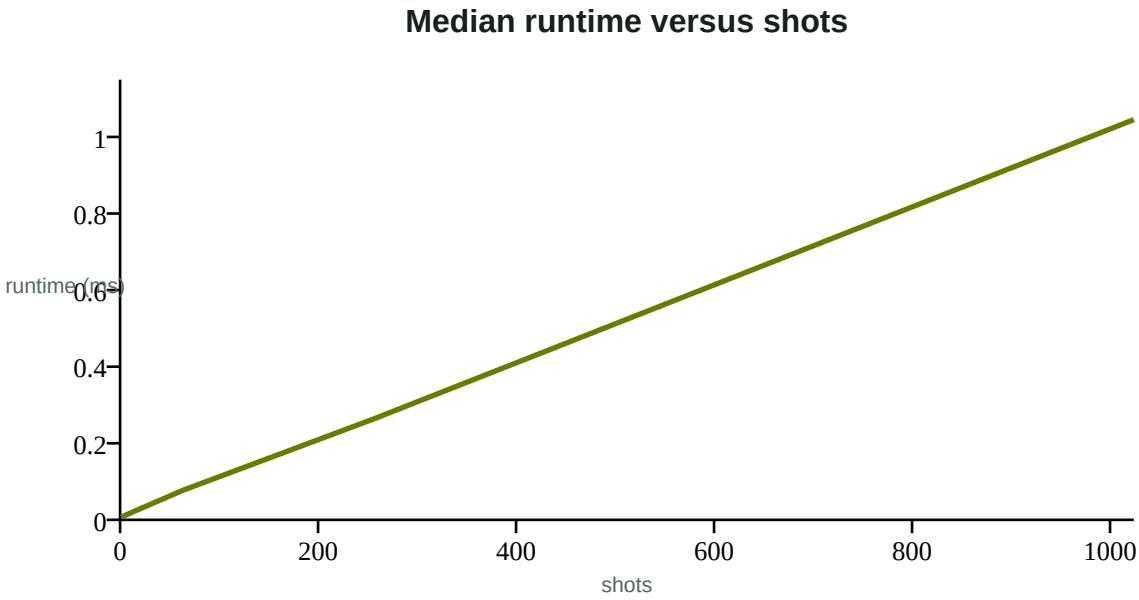
All plots below are generated from the retained ten-repetition release-build dataset. Exact records, environment, commands, and checksums accompany the repository.



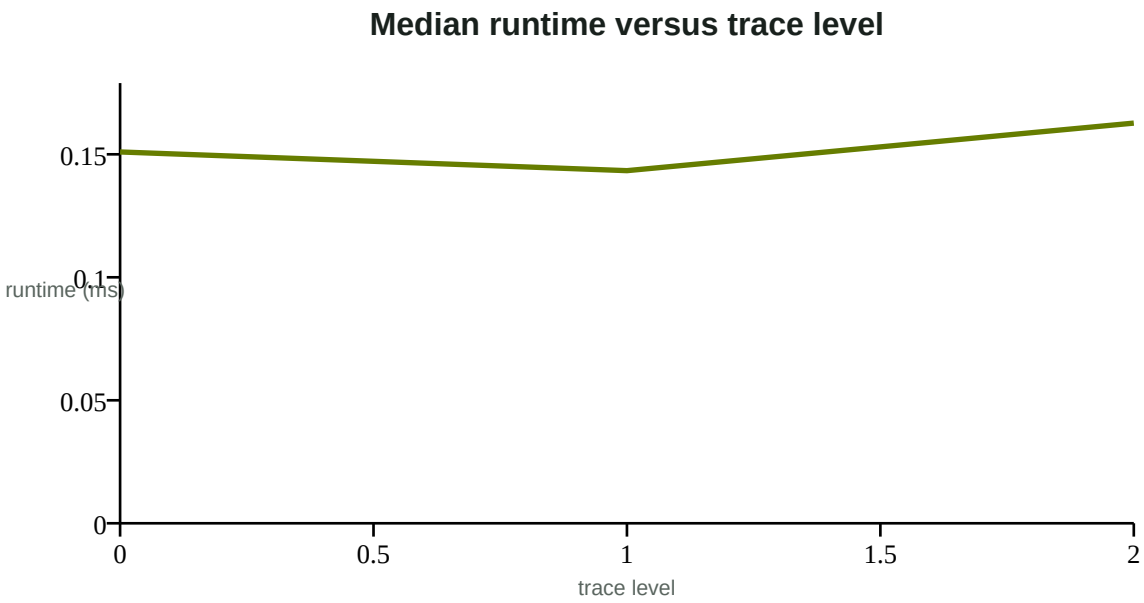
Generated final-protocol median for allocated qubits; source: benchmark-summary.csv.



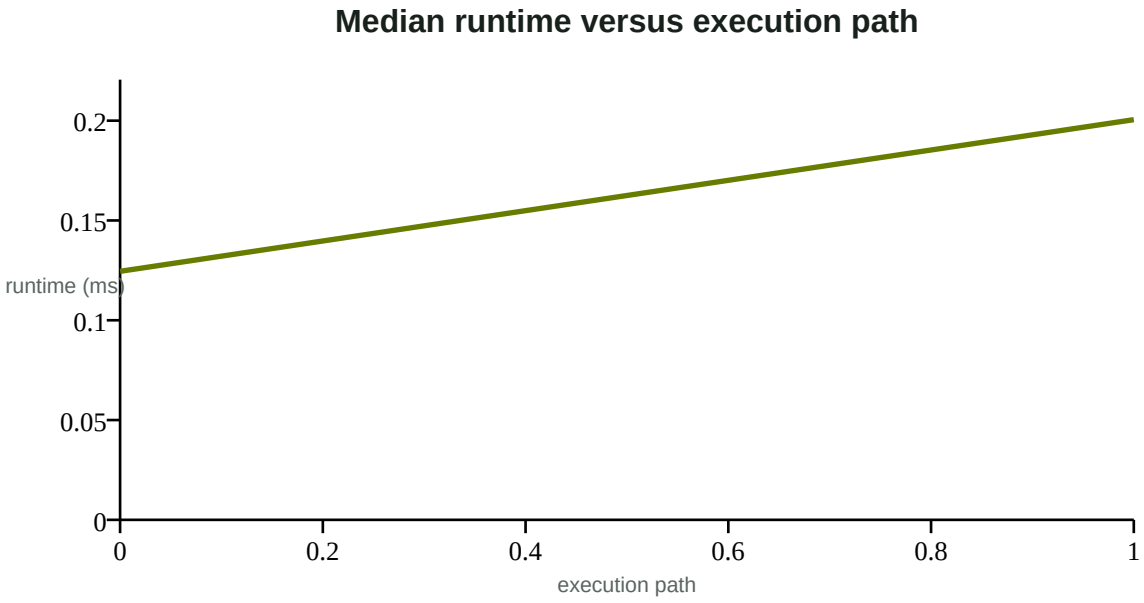
Generated final-protocol median for circuit depth; source: benchmark-summary.csv.



Generated final-protocol median for shots; source: benchmark-summary.csv.



Generated final-protocol median for trace level; source: benchmark-summary.csv.



Generated final-protocol median for execution path; source: benchmark-summary.csv.

Reproducibility Tables

research/benchmarks/external/qiskit-comparison.csv

schema	algorithm	qubits	max_amplitude_error	total_variation_distance	jensen_shannon_divergenc	clio_norm_error	qiskit_norm_error	passed
clio-qiskit-comparison-1	bell	2	1.1102230246251565e-16	2.220446049250313e-16	0.0	2.220446049250313e-16	2.220446049250313e-16	True
clio-qiskit-comparison-1	ghz3	3	1.1102230246251565e-16	2.220446049250313e-16	0.0	2.220446049250313e-16	2.220446049250313e-16	True
clio-qiskit-comparison-1	grover2	2	6.661338147750939e-16	6.661338147750939e-16	2.9582283945787943e-31	4.440892098500626e-16	8.881784197001252e-16	True
clio-qiskit-comparison-1	qft_roundtrip	3	6.661338147750939e-16	6.661338147750939e-16	2.9582283945787943e-31	4.440892098500626e-16	8.881784197001252e-16	True
clio-qiskit-comparison-1	bv_1011	5	6.661338147750939e-16	9.43689570931383e-16	0.0	4.440892098500626e-16	1.4432899320127035e-15	True
clio-qiskit-comparison-1	deutsch_jozsa_balanced	4	5.551115123125783e-16	7.771561172376096e-16	8.008566259537326e-17	4.440892098500626e-16	1.1102230246251565e-15	True

research/benchmarks/processed/resource-trace-results.csv

schema	case	estimated_trace_bytes	actual_trace_bytes	trace_estimate_ratio	estimated_total_bytes	peak_rss_native_units	peak_rss_units
clio-resource-trace-1	trace-off	0	61	0.0	65608	1851392	bytes
clio-resource-trace-1	trace-instructions	573440	299607	1.9139739725707343	639048	2605056	bytes
clio-resource-trace-1	trace-state	573440	527574	1.086937567052205	639048	3227648	bytes

Appendix: spec/architecture/first-end-to-end-checklist.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# First Executable Path: Bell State

This is a private implementation checklist, not a reduced product definition. The path is complete only when source reaches real
  state-vector measurement through the processor and produces retained evidence.

## 0. Freeze cross-cutting decisions

- [x] ADR: basis/index order, matrix orientation, control/target mapping, and display bit strings
- [x] ADR: checked classical arithmetic, flags, branch targets, and fall-through
- [x] ADR: RNG algorithm, seed derivation per shot, and replay compatibility
- [x] ADR: initial allocation/free/reset semantics
- [ ] Define diagnostic, trap, resource, and trace error codes used by this path

## 1. Architecture types and ISA

- [x] Bounded register indices, logical qubit IDs, source spans, diagnostics, and hashes
- [x] Typed `QALLOC`, `QH`, `QCX`, `QMEASURE`, and `HALT` instruction representations
- [x] Program metadata for name, seed, shots, budget, and trace level
- [x] Instruction metadata and source mapping
- [ ] Unit tests for all constructors, bounds, and serialization-ready shapes

## 2. Source front end

- [ ] Bounded lexer with comments, directives, typed operands, labels, and source spans
- [x] Parser for the Bell source with stable diagnostics
- [x] Semantic validation for directives, operand types, allocation order, and use-before-allocation
- [x] Assembly to typed instruction indices and an internally identified executable
- [ ] Positive snapshot plus wrong-register, missing-operand, unknown-mnemonic, overflow, and oversized-input tests

## 3. Reference engine

- [x] Checked state allocation and `|0...0>` initialization
- [x] `H` and controlled-X using the frozen basis convention
- [x] Norm/probability calculation with declared tolerance
- [x] State-derived measurement, collapse, renormalization, and deterministic RNG injection
- [x] Exact pre-measurement Bell amplitudes and correlation tests
- [x] Seed-repeatability and many-shot statistical tests without requiring an exact 50/50 split

## 4. Resource admission

- [x] Checked `16 x 2^n` raw memory calculation and conservative overhead model
- [x] Qubit, state/total memory, instruction, shot, and trace limits
- [x] Admission before engine allocation with explicit rejection evidence
- [ ] Boundary, overflow, and malicious giant-request tests

## 5. Runtime state machine

- [x] Initialize all registers, unset measurements, status, program counter, counters, and metadata
- [x] Fetch/decode/execute each Bell instruction through a backend interface
- [x] Map logical qubits to engine state without leaking raw indices into ISA callers
- [x] Write measurements; keep unset distinct from zero
- [x] Define per-shot reset and aggregate `00`/`11` counts
- [x] Correct `Ready -> Running -> Halted` transitions and typed backend trap paths
- [ ] Instruction/time cancellation budgets and no continuation after trap

## 6. Trace and results

- [x] Version-identified admitted events for allocation, gates, measurement, status, and halt
- [ ] No full vector by default; safe small-state snapshots only when requested
- [ ] Result contains hashes, build/engine/RNG identifiers, seed, shots, counts, final architectural state policy, budgets/usage,
  warnings, and metrics
- [ ] Integrity-checked trace/result encoding and malformed-input tests
- [ ] Supported same-contract seeded replay test

## 7. Interfaces and validation

- [x] Rust SDK parse/check/assemble/estimate/run workflow
- [ ] CLI `check`, `build`, `run`, `estimate`, `inspect`, and `trace` portions needed by Bell, including JSON and exit-code tests
- [x] Analytic validation: only `00` and `11`, exact pre-measurement probabilities within tolerance
- [ ] Differential validation against a selected primary reference with convention translation documented
- [ ] `cargo fmt`, clippy with warnings denied, unit, integration, conformance, property, and regression checks green

## Exit evidence

- [x] `examples/bell-state/main.clio` executes through parser, validator, assembler, runtime, and Clio Engine
- [x] Seeded run is repeatable under the recorded environment and RNG contract
- [ ] Trace is parseable and corresponds to actual state transitions
- [x] Resource plan precedes allocation
- [ ] Specifications and user tutorial match tested behavior
- [ ] No parser-only or hard-coded behavior is described as implemented
```

Appendix: spec/architecture/overview.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# Clio Architecture Overview

Status: initial design contract; normative details will be refined through reviewed ADRs and conformance tests.

## System boundary

Clio accepts bounded Clio Assembly source, produces a validated executable instruction stream, and executes that stream as a virtual
  processor. The runtime owns state transitions and delegates quantum-state operations through a backend trait. Clio Engine is the
  required built-in reference implementation. Optional adapters cannot change ISA-visible behavior without an explicit capability or
  compatibility error.

```text
source -> parser -> AST -> validator -> assembler -> executable
                                                       |
                                            resource admission
                                                       |
                                                 Clio Runtime
                                               /       |        \
                                          Engine    Trace     Results
```

## Initial module boundaries

- `clio-core`: shared bounded identifiers, source spans, diagnostics, and architecture metadata; no execution logic.
- `clio-isa`: typed operands, instructions, opcodes, and instruction metadata.
- `clio-parser`: source text to spanned AST; no semantic acceptance.
- `clio-assembler`: symbol resolution, semantic validation, and executable construction.
- `clio-engine`: independent state-vector mathematics and measurement.
- `clio-backend`: capability-oriented backend interface used by the runtime.
- `clio-resource`: conservative plans, budgets, usage, and admission decisions.
- `clio-trace`: bounded structured events and package encoding.
- `clio-runtime`: the only ISA execution state machine.
- `clio-validation`: conformance and comparison metrics.
- `clio-sdk`: stable orchestration API over parser, assembler, runtime, and artifacts.
- `clio-cli`: command adapter with human and machine output.

Dependencies must point toward contracts: CLI/SDK depend on subsystems; runtime depends on ISA, backend, resource, and trace contracts;
  the engine does not depend on the runtime, parser, CLI, or UI.

## Processor state

`ProcessorState` is serializable where safe and contains:

- program counter as an instruction index;
- `Ready | Running | Paused | Completed | Halted | Trapped | ResourceRejected | ValidationFailed` status;
- sixteen signed 64-bit classical registers `r0..r15`, initialized to zero;
- sixteen measurement registers `m0..m15`, initialized to `Unset`, then `Zero` or `One`;
- typed virtual-qubit records and logical-to-engine mapping;
- opaque quantum-state/backend handle rather than a serialized arbitrary backend object;
- comparison and measurement flags with defined writers;
- seed, RNG identity, shot index/count, instruction counter, budgets and usage;
- trace configuration, warnings, diagnostics, and runtime/build metadata;
- a bounded call stack only if `CALL` and `RET` are accepted by a later ADR.

An unset measurement register is an error when read. Moving a set measurement value to a classical register yields integer `0` or `1`.

## Classical arithmetic

Clio uses checked signed 64-bit arithmetic. Overflow, division or modulo by zero, invalid shifts, and `i64::MIN / -1` trap; they never
  wrap silently. Bitwise operations use the signed register bit representation and right shift is arithmetic. `CMP lhs, rhs` sets explicit
   ordering information consumed by conditional branches. Branch targets are resolved and range-checked during assembly. Backward branches
   are allowed only under configured global instruction and wall-clock budgets; their resource plan conservatively assumes the full
  instruction budget.

## Virtual-qubit lifecycle

```text
Undeclared -> Allocated -> Measured (still allocated) -> Reset/operated -> Allocated
                        \-> Freed
Allocated ----------------> Freed
```

`QALLOC qN` creates a typed logical handle and maps it to an engine position. Duplicate allocation traps or is rejected during static
  validation when provable. Quantum operations require allocation. `QFREE` requires the qubit to be separable and in `|0>` unless the
  final engine design defines and tests a trace-preserving compaction rule; the first implementation should reject unsafe release. Freed
  handles cannot be reused implicitly within the same execution. `QRESET` measures/collapses as necessary and produces `|0>` according to
  the engine convention.

## Execution lifecycle

1. Parse bounded input and retain source spans.
2. Resolve symbols and validate types, directives, control-flow targets, capabilities, and declared limits.
3. Assemble a versioned in-memory executable and compute its program hash.
4. Estimate worst-case required resources known statically and reject budgets before state-vector allocation.
5. For every shot, initialize architectural state, derive a deterministic shot RNG stream from the configured seed, and run
  fetch/decode/execute.
6. Emit bounded trace events around state transitions. A failed trace write cannot silently erase provenance; policy determines trap or
  explicit downgrade before execution.
7. Stop only on normal fall-through completion, `HALT`, pause/cancellation, deterministic trap, or configured budget rejection.
8. Aggregate counts and produce typed result metadata. Preserve per-shot final state only when explicitly requested within bounds.

`HALT` produces `Halted`; reaching exactly one instruction beyond the final instruction produces `Completed`. Other out-of-range program
  counters trap. A validation or resource failure occurs before `Running`.

## Endianness and state-vector convention

The proposed reference convention maps logical qubit `q0` to bit 0 of the basis index (little-endian basis indexing). The amplitude array
  index is the integer represented by basis bits; `|q(n-1)...q0>` is used for display. Gate matrices multiply column state vectors. This
  decision must be locked in an ADR before engine implementation and tested across all gates, measurement, display, import/export, and
````

```
  differential adapters.

## Traps and errors

Source and semantic failures return spanned diagnostics before execution. Malformed executables fail safe. Runtime faults become typed
  traps containing code, program counter, instruction, source location if available, and non-secret context. Unsupported capabilities fail
   explicitly. Undefined behavior is not part of Clio ISA.

## Determinism and replay

Classical execution is deterministic. Seeded reference-engine execution targets identical measurement sequences only for the same
  executable, seed policy, RNG algorithm revision, compatible engine/build contract, and documented numerical conditions. External
  hardware receives configuration replay only. Bit-for-bit cross-platform replay is not promised without evidence.

## Resource safety

For `n` active qubits, raw state memory is checked as `16 * 2^n` bytes. Total estimates add engine, mapping, runtime, program, result, and
   bounded trace overhead conservatively. All exponentiation, multiplication, counters, and sizes use checked arithmetic. Admission
  includes qubit, raw/total memory, source/executable size, instruction, shot, time, stack, and trace limits.

## Initial decisions requiring ADRs

1. Basis order and qubit mapping.
2. Allocation/free semantics and state-vector compaction.
3. RNG algorithm, per-shot stream derivation, and replay compatibility.
4. Overflow, comparison, shift, and fall-through semantics.
5. Which observation operations are ISA instructions versus debugger/SDK queries.
6. Executable and package serialization/integrity formats.
7. Trace backpressure, truncation, and downgrade behavior.
8. Backend capability negotiation and numerical tolerances.
```

Appendix: spec/architecture/processor-state.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Processor state

The architectural state contains the program counter, lifecycle status, sixteen signed classical registers, sixteen three-state
  measurement registers, allocated virtual-qubit identifiers, the latest signed comparison state, per-shot instruction counter, shot
  index, and total shots. Quantum amplitudes are backend state addressed through allocated logical qubits.

Status begins `Ready`, enters `Running`, and terminates as `Halted`, `Completed`, or `Trapped`. Validation and resource failures occur
  before active execution and are represented by `ValidationFailed` and `ResourceRejected` at host boundaries. `Paused` is reserved for an
   external debugger controller; the current batch runtime does not synthesize pauses.

Each instruction transition records status before and after, source span, allocation/release, measurement or classical mutation, branch
  decision, comparison state, and-when enabled and bounded-a complete small-state snapshot. The runtime increments the program counter
  before ordinary execution and replaces it with a resolved branch target when a branch is taken.
```

Appendix: spec/architecture/studio.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Studio architecture

Clio Studio is a loopback HTTP application embedded in the Rust workspace. Static HTML, CSS, and JavaScript are compiled into the
  `clio-studio` binary. JSON endpoints call Clio SDK, Clio Resource, and Clio Replay directly. The frontend contains presentation and
  interaction logic only; it does not simulate gates, measurements, registers, resources, validation, or replay.

Primary execution and inspection are separate requests. `run` preserves declared shots and trace settings for results. `inspect` forces
  one bounded state-enabled shot for timeline debugging. This prevents the debugger from rewriting the evidence-bearing execution
  configuration. Replay verification occurs server-side before imported source is accepted.
```

Appendix: spec/assembly/grammar.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# Clio Assembly Grammar

Status: initial lexical and syntactic contract for `.clio` source. Semantic operand alternatives remain governed by the ISA specification.

## Conventions

- UTF-8 source; identifiers and mnemonics are ASCII in the initial grammar.
- Lines end with LF or CRLF.
- `;` begins a comment through end of line.
- Mnemonics and directives are case-insensitive; labels and program names are case-sensitive.
- Horizontal whitespace separates tokens. Newlines terminate directives and instructions.
- Numeric angles are finite decimal/scientific literals interpreted as radians.
- The implementation enforces configured source, line, token, identifier, instruction, and literal-size limits.

## EBNF

```ebnf
program          = { blank_line | statement_line }, EOF ;
statement_line   = ws, [ label, ws ], [ directive | instruction ], ws,
                   [ comment ], newline ;
blank_line       = ws, [ comment ], newline ;

label            = identifier, ":" ;
directive        = ".", directive_name, [ ws1, directive_value ] ;
directive_name   = "program" | "seed" | "shots" | "budget" | "trace" ;
directive_value  = string | identifier | signed_integer | key_value_list ;
key_value_list   = key_value, { ws, ",", ws, key_value } ;
key_value        = identifier, ws, "=", ws, scalar ;

instruction      = mnemonic, [ ws1, operand_list ] ;
operand_list     = operand, { ws, ",", ws, operand } ;
operand          = classical_register | measurement_register | qubit_ref
                 | signed_integer | float | identifier | string ;

classical_register   = ("r" | "R"), register_index ;
measurement_register = ("m" | "M"), register_index ;
register_index       = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7"
                     | "8" | "9" | "10" | "11" | "12" | "13" | "14" | "15" ;
qubit_ref            = ("q" | "Q"), unsigned_integer ;

signed_integer   = [ "+" | "-" ], unsigned_integer ;
unsigned_integer = digit, { digit | "_" } ;
float            = [ "+" | "-" ],
                   (digits, ".", [digits] | ".", digits | digits),
                   [ ("e" | "E"), [ "+" | "-" ], digits ] ;
digits           = digit, { digit | "_" } ;
digit            = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

identifier       = ident_start, { ident_continue } ;
ident_start      = "A"..."Z" | "a"..."z" | "_" ;
ident_continue   = ident_start | digit | "-" ;
string           = '"', { string_character | escape }, '"' ;
escape           = "\\", ("\\" | '"' | "n" | "r" | "t") ;
comment          = ";", { any_character_except_newline } ;
ws               = { " " | "\t" } ;
ws1              = (" " | "\t"), ws ;
newline          = "\n" | "\r\n" ;
```

## Structural rules

- A label may share a line with an instruction or directive and points to that instruction; labels on otherwise empty lines point to the
  next instruction.
- Duplicate labels and unresolved targets are semantic errors.
- `.program` occurs at most once. `.seed`, `.shots`, `.budget`, and `.trace` occur at most once unless the final directive contract
  explicitly permits merging.
- Directives do not emit instructions and therefore do not change label addresses.
- Registers outside `r0..r15` and `m0..m15` are lexical/semantic diagnostics, never truncated.
- Qubit reference bounds and lifecycle correctness are semantic/runtime concerns.
- Unknown mnemonics and directives receive suggestions only when edit distance is unambiguous.

## Diagnostics

All failures carry a stable code, message, file identifier, byte span, one-based line/column rendering, explanation, and a safe correction
   hint when possible. Invalid UTF-8, oversized input, integer overflow, non-finite float, and token-limit failures have distinct codes.
  Recovery must be bounded and must not cascade indefinitely.

## Example

```clio
.program bell_state
.seed 42
.shots 1024
.budget memory=1GB
.trace instructions

QALLOC q0
QALLOC q1
QH q0
QCX q0, q1
QMEASURE q0, m0
QMEASURE q1, m1
HALT
```
````

Appendix: spec/decisions/0000-template.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# ADR-NNNN: Decision title

- Status: Proposed
- Date: YYYY-MM-DD
- Owners: Advaith Praveen / Archeum Studios
- Related specifications:

## Context

Describe the concrete architectural problem, constraints, evidence, and affected compatibility or research claims.

## Decision

State the chosen behavior precisely enough to test. Include invariants, failure behavior, bounds, serialization implications, and
  migration rules where relevant.

## Alternatives considered

Describe credible alternatives and why they were not chosen. Do not use straw alternatives.

## Consequences

List positive, negative, operational, security, performance, documentation, and research consequences.

## Verification

Identify specification sections, conformance tests, integration tests, benchmarks, and documentation that prove the decision is
  implemented.

## Revisit conditions

State evidence or constraints that would justify superseding this ADR. Never edit an accepted decision to hide history; supersede it with
  a new ADR.
```

Appendix: spec/decisions/0001-basis-order.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# ADR-0001: State-vector basis and qubit order

- Status: Accepted
- Date: 2026-07-16
- Owners: Advaith Praveen / Archeum Studios

## Context

Every gate, measurement, displayed bit string, trace, and external adapter needs one basis convention. Leaving this implicit creates
  locally plausible but mutually incompatible results.

## Decision

Clio uses little-endian basis indexing: logical `q0` selects bit 0 of the amplitude-array index. State vectors are column vectors and gate
   matrices left-multiply them. Human-readable basis strings are displayed as `|q(n-1)...q0>`, with the highest allocated logical qubit on
   the left. For `QCX control, target`, the first operand controls the second. The first implementation allocates logical qubits
  contiguously and never changes an allocated handle's meaning.

## Alternatives considered

Big-endian indexing matches some diagram orderings but complicates low-bit masking. Backend-native conventions were rejected because they
  would make ISA-visible behavior depend on the backend.

## Consequences

Bit masks and pair iteration are simple; display order differs from logical numeric order and must be documented. External adapters must
  translate explicitly.

## Verification

Gate, Bell-state, measurement, display, and differential tests must cover asymmetric states so reversal cannot pass unnoticed.

## Revisit conditions

Only a proven interoperability or performance issue justifies supersession; stored artifacts then require an explicit compatibility
  boundary.
```

Appendix: spec/decisions/0002-classical-semantics.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# ADR-0002: Classical arithmetic, flags, branches, and completion

- Status: Accepted
- Date: 2026-07-16
- Owners: Advaith Praveen / Archeum Studios

## Context

Processor execution must not depend on Rust build mode or host-language overflow behavior.

## Decision

Classical registers are signed 64-bit integers initialized to zero. Arithmetic is checked. Overflow, division or modulo by zero, `i64::MIN
   / -1`, and shift amounts outside `0..64` produce typed runtime traps. Right shift is arithmetic. `CMP a, b` records signed ordering:
  `ZERO` iff equal and `NEGATIVE` iff `a < b`. `JZ`, `JNZ`, `JLT`, and `JGT` consume this comparison state; using them before `CMP` traps.
   Successful measurement sets exactly one measurement flag. Reaching the instruction count by normal increment produces `Completed`;
  `HALT` produces `Halted`; any other out-of-range target traps.

## Alternatives considered

Wrapping arithmetic is predictable but hides likely program errors. Saturating arithmetic loses information. Updating comparison flags
  after every arithmetic operation is familiar in hardware but makes dependencies less explicit.

## Consequences

Programs are portable and failures deterministic. Extra checked operations have acceptable reference-runtime cost. The first Bell subset
  uses only `HALT`, but later classical implementation must follow this contract.

## Verification

Boundary tests cover every overflow, invalid shift, comparison ordering, branch-before-compare, explicit halt, fall-through, and malformed
   target.

## Revisit conditions

Only a documented workload requirement may add explicit wrapping or saturating opcodes; existing instruction semantics remain unchanged.
```

Appendix: spec/decisions/0003-rng-and-replay.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# ADR-0003: RNG, shot seeding, and replay boundary

- Status: Accepted
- Date: 2026-07-16
- Owners: Advaith Praveen / Archeum Studios

## Context

Seeded measurement must be reproducible without accidentally promising identical behavior across arbitrary engines or future incompatible
  algorithms.

## Decision

The built-in engine uses ChaCha8 with a 256-bit seed. A user `u64` seed is expanded deterministically: four little-endian `u64` words
  formed by SplitMix64 from the user seed. Each shot creates an independent stream by applying SplitMix64 to `seed XOR shot_index` before
  expansion. RNG identity is recorded as `chacha8-splitmix64-shot-v1`. Probability sampling consumes one `f64` draw in `[0,1)` per
  non-deterministic measurement; deterministic probability-zero/one measurements consume no draw.

Seeded replay promises the same measurement sequence only for the same executable hash, shot count, RNG identity, reference-engine
  semantic revision, and compatible floating-point behavior. External backends receive configuration replay only. Cross-platform
  bit-for-bit packages are not promised without validation.

## Alternatives considered

One shared stream across shots makes parallel execution order observable. Host-default RNGs lack a stable identity. Recording random
  outcomes alone would imitate replay without reproducing execution.

## Consequences

Shots can be scheduled independently while retaining deterministic results. RNG changes require a new identity and explicit compatibility
  handling.

## Verification

Golden seed-expansion tests, repeated execution tests, independent-shot tests, and package metadata checks cover the contract.

## Revisit conditions

Security requirements, parallelism evidence, or a validated portability issue may introduce a new RNG revision without redefining revision
   1.
```

Appendix: spec/decisions/0004-qubit-lifecycle.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# ADR-0004: Initial virtual-qubit lifecycle

- Status: Accepted
- Date: 2026-07-16
- Owners: Advaith Praveen / Archeum Studios

## Context

Dynamic removal of a qubit from an entangled state is not equivalent to deleting an array dimension. Logical handles must remain stable
  and invalid use must be deterministic.

## Decision

Within one shot, each logical handle transitions `Unallocated -> Allocated -> Freed` and is not implicitly reusable. Initial allocation
  must be contiguous from `q0`; this keeps basis mapping stable for the reference implementation. Operations require `Allocated`.
  Measurement does not deallocate. `QRESET` measures/collapses and applies X when needed, leaving the qubit allocated in `|0>`. `QFREE` is
   accepted only when the qubit is provably in computational `|0>` and separable within numerical tolerance; the initial engine may
  conservatively reject free when it cannot prove this. Successful free removes the highest mapped qubit only, avoiding remapping live
  handles. Other free orders trap as unsupported lifecycle operations.

## Alternatives considered

Arbitrary compaction complicates mappings and traces. Implicit measurement on free changes programs probabilistically. Reusable logical
  names obscure use-after-free diagnostics.

## Consequences

The initial model is safe and inspectable but restrictive. A later accepted ADR may add a free-list or density-matrix trace-out while
  preserving explicit handle semantics.

## Verification

Tests cover contiguous allocation, duplicate allocation, use before allocation, use after free, measured-but-live qubits, reset,
  non-highest free rejection, and entangled free rejection.

## Revisit conditions

Representative programs or backend capabilities may justify safe arbitrary release with a fully specified quantum operation and mapping
  model.
```

Appendix: spec/isa/overview.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Clio ISA Overview

Status: initial instruction inventory and semantic rules. The current executable subset is explicitly identified below; an instruction is
  supported only after its execution and tests land.

## Design rules

Clio ISA is typed, compact, and orthogonal. Instructions operate only on declared operand kinds: classical register (`r0..r15`),
  measurement register (`m0..m15`), logical qubit (`qN` within configured bounds), signed integer immediate, finite floating-point angle,
  or resolved instruction target. Opcodes have stable internal numeric assignments only after an ADR freezes the executable format.

Every final instruction entry must define mnemonic, opcode, operands, state transition, deterministic/probabilistic class, flags, errors,
  trap behavior, trace encoding, source example, and linked conformance cases.

## Families

| Family | Mnemonics | Initial semantics |
|---|---|---|
| Control | `NOP`, `HALT`, `TRAP` | No-op, explicit halt, explicit typed trap |
| Movement | `MOV`, `LOADI` | Register/measurement movement and signed immediate load |
| Arithmetic | `ADD`, `SUB`, `MUL`, `DIV`, `MOD` | Checked signed 64-bit operations |
| Logic | `AND`, `OR`, `XOR`, `NOT`, `SHL`, `SHR` | Signed-register bit operations; shifts are validated |
| Compare/branch | `CMP`, `JMP`, `JZ`, `JNZ`, `JLT`, `JGT` | Compare flags and resolved instruction targets |
| Qubit lifecycle | `QALLOC`, `QFREE`, `QRESET` | Typed logical lifecycle with engine synchronization |
| Single-qubit | `QX`, `QY`, `QZ`, `QH`, `QS`, `QSDG`, `QT`, `QTDG`, `QRX`, `QRY`, `QRZ` | Standard unitary gates using specified matrix
  convention |
| Multi-qubit | `QCX`, `QCZ`, `QSWAP`, `QCCX` | Distinct allocated operands and specified control order |
| Measurement | `QMEASURE` | State-derived binary measurement and collapse into `mN` |
| Synchronization | `QBARRIER` | Trace/scheduling boundary; no false physical timing claim |

## Currently executable internal subset

`QALLOC`, `QRESET`, `QFREE`, the complete listed single-qubit gate set, `QCX`, `QCZ`, `QSWAP`, `QCCX`, `QMEASURE`, classical
  `MOV`/`LOADI`, `ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `AND`, `OR`, `XOR`, `NOT`, `SHL`, `SHR`, register/immediate `CMP`, `JMP`, `JZ`, `JNZ`,
   `JLT`, `JGT`, and `HALT` execute through the Rust runtime. Arithmetic is checked and illegal operations trap. Backward branches execute
   under instruction and wall-clock limits. Safe release currently requires highest-first computational-zero qubits and closes further
  allocation for that shot. Other inventory entries remain specified targets and are not advertised as executable.

`CALL` and `RET` are deferred from the accepted inventory until a bounded stack contract is approved. `QPROB`, `QEXPECT`, and `QSAMPLE`
  initially belong to debugger/SDK observation APIs because embedding variable-sized results in processor registers is not yet coherently
  defined. An ADR may introduce typed ISA forms later without advertising them prematurely.

## Operand sketches

```text
MOV   rD, rS | mS
LOADI rD, immediate
ADD   rD, rA, rB | immediate
CMP   rA, rB | immediate
JZ    label
QALLOC qN
QRX   qN, finite-angle-radians
QCX   qControl, qTarget
QMEASURE qN, mD
TRAP  nonnegative-code
```

The exact accepted arithmetic operand grammar will be frozen before parser implementation. The assembler rejects ambiguous or wrong-typed
  forms.

Flags

- `ZERO`, `NEGATIVE`: written by `CMP` and, if approved, arithmetic operations; branch dependencies must be explicit in the full table.
- `MEASURED_ZERO`, `MEASURED_ONE`: mutually exclusive and written by successful `QMEASURE`.
- `RESOURCE_WARNING`, `TRACE_ENABLED`: runtime indicators, not substitutes for admission decisions.

Flags start cleared. Instructions that do not define a flag write preserve it.

Quantum semantics

All gate names refer to matrices written in the architecture's basis and orientation convention. Angles are radians and must be finite.
  Multi-qubit operands must be distinct. Operations on unallocated or freed handles trap. Measurement probability is derived from
  normalized amplitudes; the selected outcome collapses and renormalizes the state. Numerical drift beyond specified tolerance is a
  numerical trap, not silently repaired unless an explicitly bounded normalization policy is specified.

Directives are not instructions

`.program`, `.seed`, `.shots`, `.budget`, and `.trace` configure metadata and execution. Labels are assembly symbols. Breakpoints and
  checkpoints are debugger facilities. These do not consume program counters unless a later normative ISA entry explicitly says otherwise.

Conformance rule

Each accepted mnemonic receives positive, boundary, wrong-type, invalid-state, resource, trace, and serialization tests as applicable.
  Parser recognition alone never qualifies as ISA support.
Controlled phase

`QCPHASE control, target, angle` multiplies only computational-basis amplitudes whose control and target bits are both one by `exp(i x
  angle)`. Operands must be distinct and the angle must be a finite number of radians. `QCZ` is the exact special case at pi. This
  primitive supports QFT decompositions while preserving the little-endian convention in ADR-0001.

Observation is deliberately not an ISA instruction: basis probabilities, marginals, Pauli expectations, norm, and reduced single-qubit
  states are debugger/SDK queries and cannot affect architectural execution.
```

Appendix: docs/concepts/what-clio-is-not.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# What Clio Is Not

Clio is not a physical quantum processor or physical quantum computer. Its built-in engine uses classical computation to simulate
  quantum-state evolution. General state-vector memory grows exponentially: `n` qubits require `2^n` complex amplitudes, approximately `16
   x 2^n` raw bytes at double precision before overhead.

Clio does not turn a classical laptop into quantum hardware, demonstrate quantum advantage or supremacy, provide unlimited qubits, replace
   physical QPUs, or claim to be the first VQPU. It is not promised to be faster than mature simulation systems. Floating-point behavior,
  available memory, trace volume, and replay compatibility impose real limits.

Clio is also not a renamed wrapper around Qiskit, Cirq, PennyLane, or another simulator. Optional adapters can support interoperability
  and differential validation, but the processor architecture and built-in engine remain independently functional. It is not a cloud
  platform, physical control stack, distributed quantum network, fault-tolerant compiler, or arbitrary host-code environment.

Clio's value is an explicit, inspectable, resource-aware processor model and integrated research platform-not a claim that classical
  simulation is physical quantum computation.
```

Appendix: docs/concepts/what-clio-is.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# What Clio Is

Clio VQPU is a programmable virtual processor architecture for gate-model and hybrid classical-quantum programs. It is designed as a
  coherent machine: Clio Assembly is parsed and validated into Clio ISA instructions, Clio Runtime executes those instructions against
  explicit processor state, and Clio Engine supplies an independent built-in quantum-state implementation.

The processor model includes a program counter, statuses, classical and measurement registers, typed logical qubits, flags, budgets,
  instruction and shot counters, diagnostics, and trace configuration. Measurement can enter architectural state and influence later
  classical branches. This makes instruction lifecycle and hybrid control inspectable rather than hiding them behind a circuit-only call.

Clio Trace records bounded structured execution events. Resource admission estimates state-vector and runtime needs before dangerous
  allocations. Conformance, known-answer, property, statistical, fuzz, and differential validation provide evidence for correctness. The
  CLI, Rust and Python SDKs, and Clio Studio are interfaces to the same Rust implementation.

The project combines a usable open-source product with a reproducible research artifact. Its final claims will be constrained by
  implementation, primary-source literature, retained experimental data, and documented limitations.
```

Appendix: docs/reference/isa-summary.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Clio ISA summary

Clio Assembly is line-oriented and case-insensitive for mnemonics and typed identifiers. Qubit zero is the least-significant basis bit.

| Family | Instructions | Semantics |
|---|---|---|
| Lifecycle | `QALLOC QRESET QFREE` | Contiguous allocation, measured reset, highest-zero release |
| Single qubit | `QX QY QZ QH QS QSDG QT QTDG QRX QRY QRZ` | Double-precision unitary state transition |
| Multi qubit | `QCX QCZ QCPHASE QSWAP QCCX` | Distinct allocated operands; controlled phase uses radians |
| Observation | `QMEASURE` | Probability sampling followed by collapse and renormalization |
| Movement | `MOV LOADI` | Measurement/register movement and signed immediates |
| Arithmetic | `ADD SUB MUL DIV MOD` | Checked signed 64-bit operations |
| Logic | `AND OR XOR NOT SHL SHR` | Signed bitwise operations with checked shifts |
| Control | `CMP JMP JZ JNZ JLT JGT HALT` | Explicit comparison state and resolved branch targets |

Program directives declare name, seed, shots, memory budget, and trace level. Debugger observations (`observe`, state snapshots, reduced
  states) remain outside the executable ISA so inspection cannot change program behavior.
```

Appendix: docs/reference/release.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Definitive release procedure

1. Confirm a clean committed revision and run `make check` and `make evidence`.
2. Run the Qiskit differential adapter in its pinned isolated environment.
3. Recollect the ten-repetition benchmark protocol and regenerate plots.
4. Render and visually inspect the research paper.
5. Run `packaging/scripts/build-release.sh` from the repository root.
6. Verify the source archive, binary archives, SBOM, and SHA-256 manifest on macOS and Linux.
7. Create the single definitive Git tag only after the release audit passes.
8. Publish the GitHub release, archive the exact source and evidence on Zenodo, record the DOI in `CITATION.cff`, and publish the Archeum
  Studios product page.

GitHub, Zenodo, DOI registration, signing credentials, and public product-page publication require project-owner accounts and are
  intentionally not automated by local build scripts.
```

Appendix: docs/reference/sdk.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Clio SDK reference

The Rust SDK exposes a source-first API:

- `build(source)` parses, validates, and returns the typed executable.
- `disassemble(source)` returns a numbered typed instruction listing.
- `estimate(source, limits)` performs resource admission without state allocation.
- `execute(source, limits)` runs all declared shots.
- `inspect(source, limits)` forces one shot with bounded state snapshots.
- `observe(source, limits)` returns norm, basis probabilities, marginals, Pauli expectations, and reduced single-qubit states.
- `validate(source, limits)` executes a recognized canonical workload and returns a typed report.

Every fallible operation returns `SdkError`, preserving validation diagnostics, resource rejection, and runtime traps. Hosts must set
  `ResourceLimits` deliberately for untrusted programs. The default is a local-development ceiling, not a universal service policy.

Observation and inspection are debugger operations, not ISA instructions. They cannot be inserted into a program to affect architectural
  state.
```

Appendix: docs/reference/security-model.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Security model

Clio accepts untrusted assembly text but does not promise safe execution without host limits. Admission bounds active qubits, shots,
  estimated memory, instructions, trace bytes, and wall-clock execution. Checked classical arithmetic traps on overflow, division by zero,
   invalid remainder, and invalid shifts. State allocation uses checked arithmetic and Rust safe code; every workspace crate forbids
  `unsafe`.

Backward branches are admitted against the complete host instruction ceiling. Trace estimates use a conservative per-event bound validated
   against serialized bounded-state traces. Runtime instruction and time budgets remain active after admission.

Clio Studio binds to loopback, has no remote asset dependencies or telemetry, and applies a restrictive content-security policy. It has no
   authentication and must not be exposed directly to a network. Replay packages are integrity checked but not cryptographically signed;
  SHA-256 detects mutation, not a malicious producer.

The Qiskit adapter is an optional research dependency and is never imported by the built-in engine, runtime, CLI, SDK, or Studio. External
   baseline environments should remain isolated.
```

Appendix: docs/reference/studio.md

The following checked-in artifact is reproduced for semantic and implementation audit.

```
# Clio Studio

Run `cargo run --release -p clio-studio` and open `http://127.0.0.1:4317`. Studio binds to loopback by default, serves no third-party
  resources, sends no telemetry, and executes programs through Clio SDK.

The editor loads canonical examples and preserves the working source in browser-local storage. Build checks syntax and semantics. Run
  performs the declared shot execution, then performs a separate single-shot bounded inspection so timeline scrubbing cannot alter the
  primary result. Validate selects the canonical known-answer validator from program metadata.

The processor inspector reports architectural status, program counter, backend, seed, shot, virtual qubits, classical registers,
  measurement registers, and the admitted resource plan. The quantum-state panel displays complete amplitudes only for the runtime's
  bounded small-state trace policy. It does not use isolated Bloch spheres for entangled states.

Replay export creates a checksummed package through `clio-replay`. Import verifies schema, checksum, engine identity, RNG identity,
  executable equality, and exact deterministic reproduction before loading its source.

Studio is a local processor laboratory, not a hosted multi-user service. Binding it to a non-loopback address requires an external
  authenticated reverse proxy and an explicit deployment security review.
```

Appendix: docs/tutorials/bell-state.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# Bell State Through Clio VQPU

The current internal executable path supports the Bell workload in `examples/bell-state/main.clio`. This is a verified vertical slice of
  the eventual processor, not the complete Clio ISA or definitive release.

Check and estimate it before execution:

```bash
cargo run -p clio-cli -- check examples/bell-state/main.clio
cargo run -p clio-cli -- estimate examples/bell-state/main.clio
```

Execute through parser, semantic validation, assembly, resource admission, Clio Runtime, and the built-in Clio Engine:

```bash
cargo run -p clio-cli -- run examples/bell-state/main.clio
```

The engine initializes `|00>`, applies H to logical `q0`, then applies controlled-X from `q0` to `q1`. Under Clio's little-endian
  basis-index convention, the only nonzero amplitudes before measurement are indices 0 and 3, each `1/sqrt(2)`. Measuring `q0` collapses
  the shared state, so measuring `q1` must agree. Across many shots, only `00` and `11` are valid; their exact counts are seeded samples
  and need not be equal.

Use JSON for structured results or request the complete admitted trace/result object:

```bash
cargo run -p clio-cli -- run examples/bell-state/main.clio --json
cargo run -p clio-cli -- trace examples/bell-state/main.clio
```

The result records source hash, engine and RNG semantic identities, seed, shots, resource plan, final architectural state, counts, and
  real instruction transitions. Clio still performs classical state-vector simulation; this workflow does not use physical quantum
  hardware.
````

Appendix: docs/tutorials/classical-isa.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# Classical Instructions and Guarded Loops

Clio classical registers are signed 64-bit integers initialized to zero. Arithmetic uses checked semantics: overflow, division or modulo
  by zero, and invalid shift counts trap deterministically. Bitwise operations act on the signed register bit representation, and right
  shift is arithmetic.

Binary instructions use `OP destination, left-register, right-register-or-immediate`. `CMP` stores explicit signed comparison state
  consumed by `JZ`, `JNZ`, `JLT`, and `JGT`. A conditional branch before `CMP` traps.

Backward branches are supported under global instruction and wall-clock budgets:

```clio
LOADI r0, 0
loop:
ADD r0, r0, 1
CMP r0, 5
JLT loop
HALT
```

Configure CLI safeguards with `--instruction-limit N` and `--time-limit-ms N`. Backward-branch programs receive conservative
  trace/resource admission because their iteration count is not statically assumed.
````

Appendix: docs/tutorials/ghz-state.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# GHZ States

Clio includes verified 3-, 4-, and 5-qubit GHZ workloads under `examples/ghz-state/`. Each prepares

`(|00...0> + |11...1>) / sqrt(2)`

by applying H to `q0` and controlled-X from `q0` to every other qubit. Every qubit is then measured. Correct executions contain only the
  all-zero and all-one bit strings.

```bash
cargo run -p clio-cli -- run examples/ghz-state/5-qubits/main.clio
cargo run -p clio-cli -- validate examples/ghz-state/5-qubits/main.clio
```

State inspection is intentionally bounded to eight qubits:

```bash
cargo run -p clio-cli -- inspect examples/ghz-state/4-qubits/main.clio
```

Inspection runs one seeded shot with state snapshots and reports complex amplitudes and probabilities after real instructions. It is not
  enabled by default because snapshots scale exponentially.
````

Appendix: docs/tutorials/measurement-branching.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# Measurement-Driven Classical Control

Run the verified hybrid example:

```bash
cargo run -p clio-cli -- run examples/measurement-branching/main.clio
```

The program applies H to `q0`, measures into three-state register `m0`, moves the set result into signed classical register `r0`, compares
   it with zero, and follows a forward branch. Both paths write the measurement value to `r1` before halting.

An unset measurement register is not zero. `MOV r0, m0` traps if `m0` has not been written during the shot. `JZ` similarly traps without
  an executed `CMP`. These rules prevent hidden initialization assumptions.

The trace records the measurement change, classical register mutations, and whether each branch was taken. The program is reinitialized
  for every shot, and its seeded outcome stream follows ADR-0003. This remains classical simulation of quantum-state evolution, not
  physical quantum execution.
````

Appendix: docs/tutorials/oracle-algorithms.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# Oracle Algorithms

Clio's Bernstein-Vazirani workloads recover hidden strings exactly through phase kickback and Hadamard interference. The Deutsch-Jozsa
  workloads classify constant-zero, constant-one, and balanced-parity oracles.

```bash
cargo run -p clio-cli -- validate examples/bernstein-vazirani/secret-1011/main.clio
cargo run -p clio-cli -- validate examples/deutsch-jozsa/balanced-parity/main.clio
```

The checked-in validators require every shot to match the analytic result. These small noiseless state-vector workloads are correctness
  tests, not evidence of quantum advantage.
````

Appendix: docs/tutorials/teleportation.md

The following checked-in artifact is reproduced for semantic and implementation audit.

````
# Quantum Teleportation

Clio's teleportation workloads verify hybrid quantum-classical execution across three analytically known input states: `|0>`, `|1>`, and
  `|+>`.

The program prepares the input on `q0`, creates a Bell pair between `q1` and `q2`, performs sender-side gates and measurements, moves
  those results into classical registers, and conditionally applies Z and X corrections to `q2`. The receiver is then measured in the
  appropriate verification basis.

```bash
cargo run -p clio-cli -- validate examples/teleportation/zero/main.clio
cargo run -p clio-cli -- validate examples/teleportation/one/main.clio
cargo run -p clio-cli -- validate examples/teleportation/plus/main.clio
```

Each canonical workload requires all 1,024 receiver checks to match the expected bit and all four sender measurement combinations to
  occur. The plus workload applies H at the receiver before measurement, converting expected `|+>` into deterministic `|0>` for
  known-answer verification.

This demonstrates state transfer inside Clio's simulated processor state. It is not physical communication or physical quantum
  teleportation.
````

Appendix: crates/clio-engine/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Independent built-in double-precision state-vector quantum engine.

#![forbid(unsafe_code)]

use clio_backend::{BackendError, QuantumBackend, SingleQubitGate};
use clio_core::QubitId;
use num_complex::Complex64;
use serde::{Deserialize, Serialize};

const NUMERICAL_TOLERANCE: f64 = 1.0e-12;
const INVERSE_SQRT_2: f64 = std::f64::consts::FRAC_1_SQRT_2;

/// Clio's built-in state-vector backend.
#[derive(Clone, Debug)]
pub struct StateVectorEngine {
    amplitudes: Vec<Complex64>,
    qubits: usize,
}

/// One Pauli factor in a tensor-product observable.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub enum Pauli {
    /// Identity.
    I,
    /// Pauli X.
    X,
    /// Pauli Y.
    Y,
    /// Pauli Z.
    Z,
}

/// Reduced density matrix of one qubit, in computational-basis order.
#[derive(Clone, Copy, Debug, PartialEq, Serialize, Deserialize)]
pub struct ReducedQubitState {
    /// Real components of the 2x2 matrix.
    pub real: [[f64; 2]; 2],
    /// Imaginary components of the 2x2 matrix.
    pub imaginary: [[f64; 2]; 2],
}

impl Default for StateVectorEngine {
    fn default() -> Self {
        Self::new()
    }
}

impl StateVectorEngine {
    /// Creates the zero-qubit scalar state.
    #[must_use]
    pub fn new() -> Self {
        Self {
            amplitudes: vec![Complex64::new(1.0, 0.0)],
            qubits: 0,
        }
    }

    /// Returns a read-only amplitude snapshot for bounded inspection callers.
    #[must_use]
    pub fn amplitudes(&self) -> &[Complex64] {
        &self.amplitudes
    }

    /// Returns the squared norm.
    #[must_use]
    pub fn norm_sqr(&self) -> f64 {
        self.amplitudes.iter().map(Complex64::norm_sqr).sum()
    }

    /// Returns exact computational-basis probabilities in basis-index order.
    #[must_use]
    pub fn basis_probabilities(&self) -> Vec<f64> {
        self.amplitudes.iter().map(Complex64::norm_sqr).collect()
    }

    /// Returns `[P(0), P(1)]` for one qubit.
    pub fn marginal_probabilities(&self, qubit: QubitId) -> Result<[f64; 2], BackendError> {
        let one = self.probability_one(qubit)?;
        Ok([1.0 - one, one])
    }

    /// Returns the expectation of a tensor product; omitted qubits are identity.
    pub fn pauli_expectation(&self, factors: &[(QubitId, Pauli)]) -> Result<f64, BackendError> {
        let mut masks = [0_usize; 3];
        for &(qubit, pauli) in factors {
            let bit = 1_usize << self.require(qubit)?;
            if pauli != Pauli::I && masks.iter().any(|mask| mask & bit != 0) {
                return Err(BackendError::DuplicateOperand);
            }
            match pauli {
                Pauli::I => {}
                Pauli::X => masks[0] |= bit,
                Pauli::Y => masks[1] |= bit,
                Pauli::Z => masks[2] |= bit,
            }
        }
        let flip = masks[0] | masks[1];
        let mut expectation = Complex64::new(0.0, 0.0);
        for (index, amplitude) in self.amplitudes.iter().enumerate() {
```

```
            let mut phase = if (index & masks[2]).count_ones() % 2 == 0 {
                Complex64::new(1.0, 0.0)
            } else {
                Complex64::new(-1.0, 0.0)
            };
            for bit_index in 0..self.qubits {
                let bit = 1_usize << bit_index;
                if masks[1] & bit != 0 {
                    phase *= if index & bit == 0 {
                        Complex64::new(0.0, 1.0)
                    } else {
                        Complex64::new(0.0, -1.0)
                    };
                }
            }
            expectation += amplitude.conj() * phase * self.amplitudes[index ^ flip];
        }
        if expectation.im.abs() > NUMERICAL_TOLERANCE {
            return Err(BackendError::Numerical(
                "Pauli expectation acquired an imaginary component".to_owned(),
            ));
        }
        Ok(expectation.re.clamp(-1.0, 1.0))
    }

    /// Returns the reduced 2x2 density matrix for one qubit.
    pub fn reduced_qubit_state(&self, qubit: QubitId) -> Result<ReducedQubitState, BackendError> {
        let bit = 1_usize << self.require(qubit)?;
        let mut matrix = [[Complex64::new(0.0, 0.0); 2]; 2];
        for index in 0..self.amplitudes.len() {
            if index & bit == 0 {
                let zero = self.amplitudes[index];
                let one = self.amplitudes[index | bit];
                matrix[0][0] += zero * zero.conj();
                matrix[0][1] += zero * one.conj();
                matrix[1][0] += one * zero.conj();
                matrix[1][1] += one * one.conj();
            }
        }
        Ok(ReducedQubitState {
            real: matrix.map(|row| row.map(|value| value.re)),
            imaginary: matrix.map(|row| row.map(|value| value.im)),
        })
    }

    fn require(&self, qubit: QubitId) -> Result<usize, BackendError> {
        let index = qubit.index();
        if index < self.qubits {
            Ok(index)
        } else {
            Err(BackendError::Unallocated(qubit))
        }
    }

    fn ensure_normalized(&self) -> Result<(), BackendError> {
        let norm = self.norm_sqr();
        if norm.is_finite() && (norm - 1.0).abs() <= NUMERICAL_TOLERANCE {
            Ok(())
        } else {
            Err(BackendError::Numerical(format!(
                "state norm {norm:.16} is outside tolerance"
            )))
        }
    }

    fn apply_matrix(
        &mut self,
        qubit: QubitId,
        matrix: [[Complex64; 2]; 2],
    ) -> Result<(), BackendError> {
        let bit = 1_usize << self.require(qubit)?;
        for base in (0..self.amplitudes.len()).step_by(bit * 2) {
            for offset in 0..bit {
                let zero = base + offset;
                let one = zero + bit;
                let a = self.amplitudes[zero];
                let b = self.amplitudes[one];
                self.amplitudes[zero] = matrix[0][0] * a + matrix[0][1] * b;
                self.amplitudes[one] = matrix[1][0] * a + matrix[1][1] * b;
            }
        }
        self.ensure_normalized()
    }
}

impl QuantumBackend for StateVectorEngine {
    fn allocate(&mut self, qubit: QubitId) -> Result<(), BackendError> {
        if qubit.index() != self.qubits {
            return Err(BackendError::NonContiguousAllocation {
                expected: self.qubits,
                found: qubit.index(),
            });
        }
        let new_len = self
            .amplitudes
            .len()
            .checked_mul(2)
            .ok_or(BackendError::AllocationOverflow)?;
        self.amplitudes.resize(new_len, Complex64::new(0.0, 0.0));
        self.qubits += 1;
        Ok(())
    }

    fn apply_single(&mut self, qubit: QubitId, gate: SingleQubitGate) -> Result<(), BackendError> {
        let zero = Complex64::new(0.0, 0.0);
        let one = Complex64::new(1.0, 0.0);
        let i = Complex64::new(0.0, 1.0);
```

```
    let matrix = match gate {
        SingleQubitGate::X => [[zero, one], [one, zero]],
        SingleQubitGate::Y => [[zero, -i], [i, zero]],
        SingleQubitGate::Z => [[one, zero], [zero, -one]],
        SingleQubitGate::H => {
            let s = Complex64::new(INVERSE_SQRT_2, 0.0);
            [[s, s], [s, -s]]
        }
        SingleQubitGate::S => [[one, zero], [zero, i]],
        SingleQubitGate::Sdg => [[one, zero], [zero, -i]],
        SingleQubitGate::T => {
            let phase = Complex64::from_polar(1.0, std::f64::consts::FRAC_PI_4);
            [[one, zero], [zero, phase]]
        }
        SingleQubitGate::Tdg => {
            let phase = Complex64::from_polar(1.0, -std::f64::consts::FRAC_PI_4);
            [[one, zero], [zero, phase]]
        }
        SingleQubitGate::Rx(angle) => {
            validate_angle(angle)?;
            let cosine = Complex64::new((angle / 2.0).cos(), 0.0);
            let sine = Complex64::new(0.0, -(angle / 2.0).sin());
            [[cosine, sine], [sine, cosine]]
        }
        SingleQubitGate::Ry(angle) => {
            validate_angle(angle)?;
            let cosine = Complex64::new((angle / 2.0).cos(), 0.0);
            let sine = Complex64::new((angle / 2.0).sin(), 0.0);
            [[cosine, -sine], [sine, cosine]]
        }
        SingleQubitGate::Rz(angle) => {
            validate_angle(angle)?;
            let negative = Complex64::from_polar(1.0, -angle / 2.0);
            let positive = Complex64::from_polar(1.0, angle / 2.0);
            [[negative, zero], [zero, positive]]
        }
    };
    self.apply_matrix(qubit, matrix)
}

fn controlled_x(&mut self, control: QubitId, target: QubitId) -> Result<(), BackendError> {
    let control_bit = 1_usize << self.require(control)?;
    let target_bit = 1_usize << self.require(target)?;
    if control_bit == target_bit {
        return Err(BackendError::DuplicateOperand);
    }
    for index in 0..self.amplitudes.len() {
        if index & control_bit != 0 && index & target_bit == 0 {
            self.amplitudes.swap(index, index | target_bit);
        }
    }
    self.ensure_normalized()
}

fn controlled_z(&mut self, control: QubitId, target: QubitId) -> Result<(), BackendError> {
    let control_bit = 1_usize << self.require(control)?;
    let target_bit = 1_usize << self.require(target)?;
    if control_bit == target_bit {
        return Err(BackendError::DuplicateOperand);
    }
    for (index, amplitude) in self.amplitudes.iter_mut().enumerate() {
        if index & control_bit != 0 && index & target_bit != 0 {
            *amplitude = -*amplitude;
        }
    }
    self.ensure_normalized()
}

fn controlled_phase(
    &mut self,
    control: QubitId,
    target: QubitId,
    angle: f64,
) -> Result<(), BackendError> {
    validate_angle(angle)?;
    let control_bit = 1_usize << self.require(control)?;
    let target_bit = 1_usize << self.require(target)?;
    if control_bit == target_bit {
        return Err(BackendError::DuplicateOperand);
    }
    let phase = Complex64::from_polar(1.0, angle);
    for (index, amplitude) in self.amplitudes.iter_mut().enumerate() {
        if index & control_bit != 0 && index & target_bit != 0 {
            *amplitude *= phase;
        }
    }
    self.ensure_normalized()
}

fn swap(&mut self, first: QubitId, second: QubitId) -> Result<(), BackendError> {
    let first_bit = 1_usize << self.require(first)?;
    let second_bit = 1_usize << self.require(second)?;
    if first_bit == second_bit {
        return Err(BackendError::DuplicateOperand);
    }
    for index in 0..self.amplitudes.len() {
        if index & first_bit == 0 && index & second_bit != 0 {
            self.amplitudes.swap(index, index ^ first_bit ^ second_bit);
        }
    }
    self.ensure_normalized()
}

fn controlled_controlled_x(
    &mut self,
    first_control: QubitId,
    second_control: QubitId,
```

```
        target: QubitId,
    ) -> Result<(), BackendError> {
        let first_bit = 1_usize << self.require(first_control)?;
        let second_bit = 1_usize << self.require(second_control)?;
        let target_bit = 1_usize << self.require(target)?;
        if first_bit == second_bit || first_bit == target_bit || second_bit == target_bit {
            return Err(BackendError::DuplicateOperand);
        }
        for index in 0..self.amplitudes.len() {
            if index & first_bit != 0 && index & second_bit != 0 && index & target_bit == 0 {
                self.amplitudes.swap(index, index | target_bit);
            }
        }
        self.ensure_normalized()
    }

    fn probability_one(&self, qubit: QubitId) -> Result<f64, BackendError> {
        let bit = 1_usize << self.require(qubit)?;
        let probability = self
            .amplitudes
            .iter()
            .enumerate()
            .filter(|(index, _)| index & bit != 0)
            .map(|(_, amplitude)| amplitude.norm_sqr())
            .sum::<f64>();
        if probability.is_finite()
            && (-NUMERICAL_TOLERANCE..=1.0 + NUMERICAL_TOLERANCE).contains(&probability)
        {
            Ok(probability.clamp(0.0, 1.0))
        } else {
            Err(BackendError::Numerical(format!(
                "measurement probability {probability:.16} is invalid"
            )))
        }
    }

    fn collapse(&mut self, qubit: QubitId, outcome: bool) -> Result<(), BackendError> {
        let probability_one = self.probability_one(qubit)?;
        let selected_probability = if outcome {
            probability_one
        } else {
            1.0 - probability_one
        };
        if selected_probability <= NUMERICAL_TOLERANCE {
            return Err(BackendError::Numerical(
                "cannot collapse to a zero-probability outcome".to_owned(),
            ));
        }
        let bit = 1_usize << self.require(qubit)?;
        let scale = selected_probability.sqrt().recip();
        for (index, amplitude) in self.amplitudes.iter_mut().enumerate() {
            if (index & bit != 0) == outcome {
                *amplitude *= scale;
            } else {
                *amplitude = Complex64::new(0.0, 0.0);
            }
        }
        self.ensure_normalized()
    }

    fn free(&mut self, qubit: QubitId) -> Result<(), BackendError> {
        let index = self.require(qubit)?;
        let expected = self.qubits.saturating_sub(1);
        if index != expected {
            return Err(BackendError::NonHighestFree {
                expected,
                found: index,
            });
        }
        if self.probability_one(qubit)? > NUMERICAL_TOLERANCE {
            return Err(BackendError::QubitNotZero(qubit));
        }
        self.amplitudes.truncate(self.amplitudes.len() / 2);
        self.qubits -= 1;
        self.ensure_normalized()
    }

    fn qubit_count(&self) -> usize {
        self.qubits
    }
}

fn validate_angle(angle: f64) -> Result<(), BackendError> {
    if angle.is_finite() {
        Ok(())
    } else {
        Err(BackendError::Numerical(
            "rotation angle must be finite".to_owned(),
        ))
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn bell_engine() -> StateVectorEngine {
        let mut engine = StateVectorEngine::new();
        engine.allocate(QubitId::new(0)).expect("allocate q0");
        engine.allocate(QubitId::new(1)).expect("allocate q1");
        engine
            .apply_single(QubitId::new(0), SingleQubitGate::H)
            .expect("H q0");
        engine
            .controlled_x(QubitId::new(0), QubitId::new(1))
            .expect("CX q0 q1");
        engine
```

```
}

#[test]
fn creates_exact_bell_support_in_little_endian_order() {
    let engine = bell_engine();
    assert!((engine.amplitudes()[0].re - INVERSE_SQRT_2).abs() < NUMERICAL_TOLERANCE);
    assert_eq!(engine.amplitudes()[1], Complex64::new(0.0, 0.0));
    assert_eq!(engine.amplitudes()[2], Complex64::new(0.0, 0.0));
    assert!((engine.amplitudes()[3].re - INVERSE_SQRT_2).abs() < NUMERICAL_TOLERANCE);
    assert!((engine.norm_sqr() - 1.0).abs() < NUMERICAL_TOLERANCE);
}

#[test]
fn measurement_collapse_preserves_correlation() {
    let mut engine = bell_engine();
    engine
        .collapse(QubitId::new(0), true)
        .expect("measure q0=1");
    assert!((engine.probability_one(QubitId::new(1)).expect("p(q1)") - 1.0).abs() < 1e-12);
    engine
        .collapse(QubitId::new(1), true)
        .expect("measure q1=1");
    assert_eq!(engine.amplitudes()[3], Complex64::new(1.0, 0.0));
}

#[test]
fn observations_match_bell_correlations_and_reduced_state() {
    let engine = bell_engine();
    assert_eq!(engine.basis_probabilities().len(), 4);
    assert!(
        (engine
            .marginal_probabilities(QubitId::new(0))
            .expect("marginal")[1]
            - 0.5)
            .abs()
            < NUMERICAL_TOLERANCE
    );
    assert!(
        (engine
            .pauli_expectation(&[(QubitId::new(0), Pauli::X), (QubitId::new(1), Pauli::X)])
            .expect("XX")
            - 1.0)
            .abs()
            < NUMERICAL_TOLERANCE
    );
    assert!(
        (engine
            .pauli_expectation(&[(QubitId::new(0), Pauli::Y), (QubitId::new(1), Pauli::Y)])
            .expect("YY")
            + 1.0)
            .abs()
            < NUMERICAL_TOLERANCE
    );
    assert!(
        (engine
            .pauli_expectation(&[(QubitId::new(0), Pauli::Z), (QubitId::new(1), Pauli::Z)])
            .expect("ZZ")
            - 1.0)
            .abs()
            < NUMERICAL_TOLERANCE
    );
    let reduced = engine
        .reduced_qubit_state(QubitId::new(0))
        .expect("reduced state");
    assert!((reduced.real[0][0] - 0.5).abs() < NUMERICAL_TOLERANCE);
    assert!((reduced.real[1][1] - 0.5).abs() < NUMERICAL_TOLERANCE);
    assert!(reduced.real[0][1].abs() < NUMERICAL_TOLERANCE);
}

#[test]
fn controlled_phase_and_inverse_restore_state() {
    let mut engine = bell_engine();
    let original = engine.amplitudes().to_vec();
    engine
        .controlled_phase(QubitId::new(0), QubitId::new(1), 0.37)
        .expect("phase");
    engine
        .controlled_phase(QubitId::new(0), QubitId::new(1), -0.37)
        .expect("inverse");
    assert_state_close(engine.amplitudes(), &original);
}

#[allow(clippy::cast_precision_loss)] // Test widths are tiny and powers of two are exact in f64.
fn qft(engine: &mut StateVectorEngine, qubits: &[QubitId]) {
    for (target_index, &target) in qubits.iter().enumerate() {
        engine
            .apply_single(target, SingleQubitGate::H)
            .expect("QFT H");
        for (control_index, &control) in qubits.iter().enumerate().skip(target_index + 1) {
            let denominator = (1_u64 << (control_index - target_index)) as f64;
            engine
                .controlled_phase(control, target, std::f64::consts::PI / denominator)
                .expect("QFT phase");
        }
    }
    for index in 0..qubits.len() / 2 {
        engine
            .swap(qubits[index], qubits[qubits.len() - 1 - index])
            .expect("QFT swap");
    }
}

#[allow(clippy::cast_precision_loss)] // Test widths are tiny and powers of two are exact in f64.
fn inverse_qft(engine: &mut StateVectorEngine, qubits: &[QubitId]) {
    for index in 0..qubits.len() / 2 {
        engine
            .swap(qubits[index], qubits[qubits.len() - 1 - index])
```

```
            .expect("IQFT swap");
    }
    for target_index in (0..qubits.len()).rev() {
        let target = qubits[target_index];
        for control_index in (target_index + 1..qubits.len()).rev() {
            let denominator = (1_u64 << (control_index - target_index)) as f64;
            engine
                .controlled_phase(
                    qubits[control_index],
                    target,
                    -std::f64::consts::PI / denominator,
                )
                .expect("IQFT phase");
        }
        engine
            .apply_single(target, SingleQubitGate::H)
            .expect("IQFT H");
    }
}

#[test]
fn qft_inverse_roundtrip_restores_three_qubit_state() {
    let mut engine = StateVectorEngine::new();
    let qubits = [QubitId::new(0), QubitId::new(1), QubitId::new(2)];
    for qubit in qubits {
        engine.allocate(qubit).expect("allocate");
    }
    engine
        .apply_single(qubits[0], SingleQubitGate::H)
        .expect("H");
    engine
        .apply_single(qubits[1], SingleQubitGate::Ry(0.73))
        .expect("Ry");
    engine.controlled_x(qubits[1], qubits[2]).expect("CX");
    let original = engine.amplitudes().to_vec();
    qft(&mut engine, &qubits);
    inverse_qft(&mut engine, &qubits);
    assert_state_close(engine.amplitudes(), &original);
}

#[test]
fn two_qubit_grover_amplifies_marked_state_exactly() {
    let mut engine = StateVectorEngine::new();
    let q0 = QubitId::new(0);
    let q1 = QubitId::new(1);
    engine.allocate(q0).expect("q0");
    engine.allocate(q1).expect("q1");
    engine.apply_single(q0, SingleQubitGate::H).expect("H");
    engine.apply_single(q1, SingleQubitGate::H).expect("H");
    engine.controlled_z(q0, q1).expect("oracle 11");
    for q in [q0, q1] {
        engine.apply_single(q, SingleQubitGate::H).expect("H");
        engine.apply_single(q, SingleQubitGate::X).expect("X");
    }
    engine.controlled_z(q0, q1).expect("diffusion phase");
    for q in [q0, q1] {
        engine.apply_single(q, SingleQubitGate::X).expect("X");
        engine.apply_single(q, SingleQubitGate::H).expect("H");
    }
    assert!((engine.basis_probabilities()[3] - 1.0).abs() < NUMERICAL_TOLERANCE);
}

fn next_random(state: &mut u64) -> u64 {
    *state = state
        .wrapping_mul(6_364_136_223_846_793_005)
        .wrapping_add(1_442_695_040_888_963_407);
    *state
}

fn reference_single(amplitudes: &mut [Complex64], qubit: usize, matrix: [[Complex64; 2]; 2]) {
    let bit = 1_usize << qubit;
    let before = amplitudes.to_vec();
    for (index, amplitude) in amplitudes.iter_mut().enumerate() {
        let source_zero = index & !bit;
        let row = usize::from(index & bit != 0);
        *amplitude =
            matrix[row][0] * before[source_zero] + matrix[row][1] * before[source_zero | bit];
    }
}

#[test]
#[allow(clippy::cast_precision_loss)] // Deterministic test PRNG maps u64 samples into f64 angles.
fn deterministic_random_gate_sequences_preserve_norm_and_inverse_recover() {
    let mut seed = 0x434c_494f_5052_4f50;
    for width in 1..=6 {
        let mut engine = StateVectorEngine::new();
        for qubit in 0..width {
            engine.allocate(QubitId::new(qubit)).expect("allocate");
        }
        let initial = engine.amplitudes().to_vec();
        let mut operations = Vec::new();
        for _ in 0..128 {
            let index = u16::try_from(next_random(&mut seed) % u64::from(width))
                .expect("bounded qubit index");
            let qubit = QubitId::new(index);
            let angle = (next_random(&mut seed) as f64 / u64::MAX as f64 - 0.5) * 2.0;
            engine
                .apply_single(qubit, SingleQubitGate::Ry(angle))
                .expect("random Ry");
            operations.push((qubit, angle));
            assert!((engine.norm_sqr() - 1.0).abs() < NUMERICAL_TOLERANCE);
        }
        for &(qubit, angle) in operations.iter().rev() {
            engine
                .apply_single(qubit, SingleQubitGate::Ry(-angle))
                .expect("inverse Ry");
        }
```

```
        assert_state_close(engine.amplitudes(), &initial);
    }
}

#[test]
#[allow(clippy::cast_precision_loss)] // Deterministic test PRNG maps u64 samples into f64 angles.
fn independent_dense_reference_matches_random_single_qubit_sequences() {
    let mut seed = 0x5245_4645_5245_4e43;
    let mut engine = StateVectorEngine::new();
    for qubit in 0..4 {
        engine.allocate(QubitId::new(qubit)).expect("allocate");
    }
    let mut reference = engine.amplitudes().to_vec();
    for _ in 0..256 {
        let qubit = usize::try_from(next_random(&mut seed) % 4).expect("bounded index");
        let angle = (next_random(&mut seed) as f64 / u64::MAX as f64) * 2.0 - 1.0;
        let cosine = Complex64::new((angle / 2.0).cos(), 0.0);
        let sine = Complex64::new((angle / 2.0).sin(), 0.0);
        reference_single(&mut reference, qubit, [[cosine, -sine], [sine, cosine]]);
        engine
            .apply_single(
                QubitId::new(u16::try_from(qubit).expect("bounded index")),
                SingleQubitGate::Ry(angle),
            )
            .expect("engine Ry");
    }
    assert_state_close(engine.amplitudes(), &reference);
}

#[test]
#[allow(clippy::cast_precision_loss)] // Deterministic test PRNG maps u64 samples into f64 angles.
fn probability_and_reduced_state_invariants_hold_for_random_states() {
    let mut seed = 0x494e_5641_5249_414e;
    let mut engine = StateVectorEngine::new();
    for qubit in 0..5 {
        engine.allocate(QubitId::new(qubit)).expect("allocate");
    }
    for _ in 0..200 {
        let qubit = QubitId::new(
            u16::try_from(next_random(&mut seed) % 5).expect("bounded qubit index"),
        );
        let angle = next_random(&mut seed) as f64 / u64::MAX as f64;
        engine
            .apply_single(qubit, SingleQubitGate::Rx(angle))
            .expect("Rx");
    }
    assert!((engine.basis_probabilities().iter().sum::<f64>() - 1.0).abs() < 1.0e-11);
    for qubit in 0..5 {
        let marginal = engine
            .marginal_probabilities(QubitId::new(qubit))
            .expect("marginal");
        let reduced = engine
            .reduced_qubit_state(QubitId::new(qubit))
            .expect("reduced");
        assert!((marginal[0] + marginal[1] - 1.0).abs() < 1.0e-11);
        assert!((reduced.real[0][0] + reduced.real[1][1] - 1.0).abs() < 1.0e-11);
        assert!((reduced.real[0][1] - reduced.real[1][0]).abs() < 1.0e-11);
        assert!((reduced.imaginary[0][1] + reduced.imaginary[1][0]).abs() < 1.0e-11);
    }
}

#[test]
fn rejects_non_contiguous_allocation() {
    let mut engine = StateVectorEngine::new();
    assert!(matches!(
        engine.allocate(QubitId::new(1)),
        Err(BackendError::NonContiguousAllocation { .. })
    ));
}

fn one_qubit() -> StateVectorEngine {
    let mut engine = StateVectorEngine::new();
    engine.allocate(QubitId::new(0)).expect("allocate q0");
    engine
}

fn assert_state_close(actual: &[Complex64], expected: &[Complex64]) {
    assert_eq!(actual.len(), expected.len());
    for (left, right) in actual.iter().zip(expected) {
        assert!((*left - *right).norm() < NUMERICAL_TOLERANCE);
    }
}

fn states_equal_up_to_global_phase(left: &[Complex64], right: &[Complex64]) -> bool {
    let pivot = left
        .iter()
        .zip(right)
        .find(|(a, b)| a.norm() > NUMERICAL_TOLERANCE && b.norm() > NUMERICAL_TOLERANCE);
    let Some((left_pivot, right_pivot)) = pivot else {
        return left.iter().all(|value| value.norm() <= NUMERICAL_TOLERANCE)
            && right
                .iter()
                .all(|value| value.norm() <= NUMERICAL_TOLERANCE);
    };
    let phase = *left_pivot / *right_pivot;
    left.iter()
        .zip(right)
        .all(|(a, b)| (*a - phase * *b).norm() < NUMERICAL_TOLERANCE)
}

#[test]
fn pauli_gates_follow_the_declared_matrices() {
    let mut x = one_qubit();
    x.apply_single(QubitId::new(0), SingleQubitGate::X)
        .expect("X");
    assert_state_close(
        x.amplitudes(),
```

```
        &[Complex64::new(0.0, 0.0), Complex64::new(1.0, 0.0)],
    );

    let mut y = one_qubit();
    y.apply_single(QubitId::new(0), SingleQubitGate::Y)
        .expect("Y");
    assert_state_close(
        y.amplitudes(),
        &[Complex64::new(0.0, 0.0), Complex64::new(0.0, 1.0)],
    );

    x.apply_single(QubitId::new(0), SingleQubitGate::Z)
        .expect("Z");
    assert_state_close(
        x.amplitudes(),
        &[Complex64::new(0.0, 0.0), Complex64::new(-1.0, 0.0)],
    );
}

#[test]
fn inverse_phase_and_rotation_pairs_restore_state() {
    let pairs = [
        (SingleQubitGate::S, SingleQubitGate::Sdg),
        (SingleQubitGate::T, SingleQubitGate::Tdg),
        (SingleQubitGate::Rx(0.731), SingleQubitGate::Rx(-0.731)),
        (SingleQubitGate::Ry(-1.219), SingleQubitGate::Ry(1.219)),
        (SingleQubitGate::Rz(2.041), SingleQubitGate::Rz(-2.041)),
    ];
    for (gate, inverse) in pairs {
        let mut engine = one_qubit();
        engine
            .apply_single(QubitId::new(0), SingleQubitGate::H)
            .expect("prepare plus");
        let expected = engine.amplitudes().to_vec();
        engine.apply_single(QubitId::new(0), gate).expect("gate");
        engine
            .apply_single(QubitId::new(0), inverse)
            .expect("inverse");
        assert_state_close(engine.amplitudes(), &expected);
    }
}

#[test]
fn non_finite_rotation_is_rejected_without_mutation() {
    let mut engine = one_qubit();
    let before = engine.amplitudes().to_vec();
    assert!(matches!(
        engine.apply_single(QubitId::new(0), SingleQubitGate::Rx(f64::NAN)),
        Err(BackendError::Numerical(_))
    ));
    assert_eq!(engine.amplitudes(), before);
}

#[test]
fn constructs_four_qubit_ghz_state() {
    let mut engine = StateVectorEngine::new();
    for index in 0..4 {
        engine
            .allocate(QubitId::new(index))
            .expect("contiguous allocation");
    }
    engine
        .apply_single(QubitId::new(0), SingleQubitGate::H)
        .expect("H");
    for target in 1..4 {
        engine
            .controlled_x(QubitId::new(0), QubitId::new(target))
            .expect("CX");
    }
    for (index, amplitude) in engine.amplitudes().iter().enumerate() {
        if matches!(index, 0 | 15) {
            assert!((amplitude.re - INVERSE_SQRT_2).abs() < NUMERICAL_TOLERANCE);
        } else {
            assert_eq!(*amplitude, Complex64::new(0.0, 0.0));
        }
    }
}

#[test]
fn global_phase_equivalent_states_are_recognized() {
    let mut y_state = one_qubit();
    y_state
        .apply_single(QubitId::new(0), SingleQubitGate::Y)
        .expect("Y");
    let mut x_state = one_qubit();
    x_state
        .apply_single(QubitId::new(0), SingleQubitGate::X)
        .expect("X");
    assert!(states_equal_up_to_global_phase(
        y_state.amplitudes(),
        x_state.amplitudes()
    ));
}

#[test]
fn rotation_boundaries_match_known_states() {
    let identity = one_qubit();
    for gate in [
        SingleQubitGate::Rx(0.0),
        SingleQubitGate::Ry(0.0),
        SingleQubitGate::Rz(0.0),
    ] {
        let mut engine = one_qubit();
        engine
            .apply_single(QubitId::new(0), gate)
            .expect("zero rotation");
        assert_state_close(engine.amplitudes(), identity.amplitudes());
```

```
        }

        let mut rx_pi = one_qubit();
        rx_pi
            .apply_single(QubitId::new(0), SingleQubitGate::Rx(std::f64::consts::PI))
            .expect("RX pi");
        assert_state_close(
            rx_pi.amplitudes(),
            &[Complex64::new(0.0, 0.0), Complex64::new(0.0, -1.0)],
        );
    }

    #[test]
    fn controlled_z_swap_and_toffoli_match_basis_behavior() {
        let mut cz = StateVectorEngine::new();
        for index in 0..2 {
            cz.allocate(QubitId::new(index)).expect("allocate");
            cz.apply_single(QubitId::new(index), SingleQubitGate::X)
                .expect("X");
        }
        cz.controlled_z(QubitId::new(0), QubitId::new(1))
            .expect("CZ");
        assert_eq!(cz.amplitudes()[3], Complex64::new(-1.0, 0.0));
        cz.swap(QubitId::new(0), QubitId::new(1)).expect("swap");
        assert_eq!(cz.amplitudes()[3], Complex64::new(-1.0, 0.0));

        let mut ccx = StateVectorEngine::new();
        for index in 0..3 {
            ccx.allocate(QubitId::new(index)).expect("allocate");
        }
        ccx.apply_single(QubitId::new(0), SingleQubitGate::X)
            .expect("X q0");
        ccx.apply_single(QubitId::new(1), SingleQubitGate::X)
            .expect("X q1");
        ccx.controlled_controlled_x(QubitId::new(0), QubitId::new(1), QubitId::new(2))
            .expect("CCX");
        assert_eq!(ccx.amplitudes()[7], Complex64::new(1.0, 0.0));
    }

    #[test]
    fn free_requires_highest_qubit_in_zero() {
        let mut engine = StateVectorEngine::new();
        engine.allocate(QubitId::new(0)).expect("q0");
        engine.allocate(QubitId::new(1)).expect("q1");
        assert!(matches!(
            engine.free(QubitId::new(0)),
            Err(BackendError::NonHighestFree { .. })
        ));
        engine
            .apply_single(QubitId::new(1), SingleQubitGate::X)
            .expect("X q1");
        assert!(matches!(
            engine.free(QubitId::new(1)),
            Err(BackendError::QubitNotZero(_))
        ));
        engine
            .apply_single(QubitId::new(1), SingleQubitGate::X)
            .expect("restore q1");
        engine.free(QubitId::new(1)).expect("safe free q1");
        assert_eq!(engine.qubit_count(), 1);
        assert_eq!(engine.amplitudes().len(), 2);
    }
}
```

Appendix: crates/clio-runtime/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Clio processor state machine for the executable Bell-state path.

#![forbid(unsafe_code)]

use clio_backend::{BackendError, QuantumBackend, SingleQubitGate};
use clio_core::{QubitId, REGISTER_COUNT};
use clio_engine::StateVectorEngine;
use clio_isa::{ClassicalBinaryOperation, ClassicalValue, Executable, Instruction};
use clio_resource::{ExecutionPlan, ResourceError, ResourceLimits, plan};
use clio_trace::{
    ClassicalChange, ExecutionTrace, MeasurementChange, StateAmplitude, TraceComparison,
    TraceEvent, TraceStatus,
};
use rand::{Rng, SeedableRng};
use rand_chacha::ChaCha8Rng;
use serde::{Deserialize, Serialize};
use std::{
    collections::BTreeMap,
    time::{Duration, Instant},
};
use thiserror::Error;

/// Stable RNG semantic identity from ADR-0003.
pub const RNG_IDENTITY: &str = "chacha8-splitmix64-shot-v1";
/// Reference engine semantic identity for result/replay compatibility.
pub const ENGINE_IDENTITY: &str = "clio-engine-hybrid-path-1";

/// Architectural execution status.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub enum ProcessorStatus {
    /// Program loaded and ready.
    Ready,
    /// Instruction execution is active.
    Running,
    /// Paused by an external controller.
    Paused,
    /// Completed by fall-through.
    Completed,
    /// Stopped by `HALT`.
    Halted,
    /// Stopped by a runtime trap.
    Trapped,
    /// Resource admission failed.
    ResourceRejected,
    /// Semantic validation failed before runtime construction.
    ValidationFailed,
}

/// Three-state architectural measurement value.
#[derive(Clone, Copy, Debug, Default, Eq, PartialEq, Serialize, Deserialize)]
pub enum MeasurementValue {
    /// Never written in this shot.
    #[default]
    Unset,
    /// Measured zero.
    Zero,
    /// Measured one.
    One,
}

/// Result of the latest explicit signed comparison.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub enum ComparisonValue {
    /// Left operand was smaller.
    Less,
    /// Operands were equal.
    Equal,
    /// Left operand was greater.
    Greater,
}

/// Serializable architectural state retained for the final shot.
#[derive(Clone, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct ProcessorState {
    /// Next instruction index.
    pub program_counter: usize,
    /// Processor lifecycle status.
    pub status: ProcessorStatus,
    /// Signed classical registers.
    pub classical_registers: [i64; REGISTER_COUNT],
    /// Three-state measurement registers.
    pub measurement_registers: [MeasurementValue; REGISTER_COUNT],
    /// Allocated logical qubits.
    pub virtual_qubits: Vec<QubitId>,
    /// Latest comparison, unset before `CMP`.
    pub comparison: Option<ComparisonValue>,
    /// Executed instruction count in this shot.
    pub instruction_counter: u64,
    /// Current shot index.
    pub shot_index: u64,
    /// Total shots.
    pub total_shots: u64,
}

impl ProcessorState {
    fn new(shot_index: u64, total_shots: u64) -> Self {
        Self {
            program_counter: 0,
            status: ProcessorStatus::Ready,
```

```
            classical_registers: [0; REGISTER_COUNT],
            measurement_registers: [MeasurementValue::Unset; REGISTER_COUNT],
            virtual_qubits: Vec::new(),
            comparison: None,
            instruction_counter: 0,
            shot_index,
            total_shots,
        }
    }
}

/// Complete result from the built-in reference runtime.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct ExecutionResult {
    /// Result schema identity.
    pub format: String,
    /// Source hash.
    pub source_sha256: String,
    /// Reference engine semantic identity.
    pub engine: String,
    /// RNG semantic identity.
    pub rng: String,
    /// User seed.
    pub seed: u64,
    /// Executed shots.
    pub shots: u64,
    /// Counts keyed by the written measurement registers, highest index first.
    pub measurement_counts: BTreeMap<String, u64>,
    /// Admitted execution plan.
    pub plan: ExecutionPlan,
    /// Final architectural state of the last shot.
    pub final_state: ProcessorState,
    /// Structured instruction trace.
    pub trace: ExecutionTrace,
}

/// Runtime failure after semantic validation.
#[derive(Debug, Error)]
pub enum RuntimeError {
    /// Resource admission rejected the executable before engine allocation.
    #[error(transparent)]
    Resource(#[from] ResourceError),
    /// Engine/backend operation failed.
    #[error("runtime trap at shot {shot}, pc {program_counter}: {source}")]
    Backend {
        /// Shot index.
        shot: u64,
        /// Program counter.
        program_counter: usize,
        /// Backend cause.
        #[source]
        source: BackendError,
    },
    /// Program did not explicitly halt within its assembled stream.
    #[error("program completed without HALT at shot {shot}")]
    MissingHalt {
        /// Shot index.
        shot: u64,
    },
    /// An architectural precondition failed deterministically.
    #[error("runtime trap at shot {shot}, pc {program_counter}: {message}")]
    ArchitecturalTrap {
        /// Shot index.
        shot: u64,
        /// Program counter.
        program_counter: usize,
        /// Failure reason.
        message: String,
    },
    /// The global instruction budget was exhausted.
    #[error("instruction budget of {limit} was exhausted at shot {shot}, pc {program_counter}")]
    InstructionBudget {
        /// Configured limit.
        limit: u64,
        /// Shot index.
        shot: u64,
        /// Program counter.
        program_counter: usize,
    },
    /// The wall-clock execution budget was exhausted.
    #[error(
        "execution time budget of {limit_millis} ms was exhausted at shot {shot}, pc {program_counter}"
    )]
    TimeBudget {
        /// Configured milliseconds.
        limit_millis: u64,
        /// Shot index.
        shot: u64,
        /// Program counter.
        program_counter: usize,
    },
}

impl RuntimeError {
    /// Returns a stable machine-readable runtime error code.
    #[must_use]
    pub const fn code(&self) -> &'static str {
        match self {
            Self::Resource(_) => "R200",
            Self::Backend { .. } => "T201",
            Self::MissingHalt { .. } => "T202",
            Self::ArchitecturalTrap { .. } => "T203",
            Self::InstructionBudget { .. } => "T204",
            Self::TimeBudget { .. } => "T205",
        }
    }
}
```

```
/// Executes a validated program after resource admission.
pub fn run(
    executable: &Executable,
    limits: ResourceLimits,
) -> Result<ExecutionResult, RuntimeError> {
    let execution_plan = plan(executable, limits)?;
    let trace_capacity = if executable.metadata.trace_level == clio_core::TraceLevel::Off {
        0
    } else {
        usize::try_from(execution_plan.estimated_instructions).unwrap_or(0)
    };
    let mut trace = ExecutionTrace::new(executable.metadata.trace_level);
    trace.events.reserve(trace_capacity);
    let mut counts = BTreeMap::new();
    let mut logical_step = 0_u64;
    let measurement_width = measurement_width(executable);
    let mut final_state = ProcessorState::new(0, executable.metadata.shots);
    let started = Instant::now();
    let time_limit = Duration::from_millis(limits.max_execution_millis);

    for shot in 0..executable.metadata.shots {
        let mut state = ProcessorState::new(shot, executable.metadata.shots);
        let mut engine = StateVectorEngine::new();
        let mut rng = shot_rng(executable.metadata.seed, shot);

        state.status = ProcessorStatus::Running;
        while state.program_counter < executable.instructions.len() {
            if logical_step >= limits.max_instructions {
                state.status = ProcessorStatus::Trapped;
                return Err(RuntimeError::InstructionBudget {
                    limit: limits.max_instructions,
                    shot,
                    program_counter: state.program_counter,
                });
            }
            if started.elapsed() >= time_limit {
                state.status = ProcessorStatus::Trapped;
                return Err(RuntimeError::TimeBudget {
                    limit_millis: limits.max_execution_millis,
                    shot,
                    program_counter: state.program_counter,
                });
            }
            let pc = state.program_counter;
            let located = &executable.instructions[pc];
            let status_before = state.status;
            let mut allocated_qubit = None;
            let mut freed_qubit = None;
            let mut reset_outcome = None;
            let mut measurement = None;
            let mut classical = None;
            let mut branch_taken = None;
            state.program_counter += 1;
            state.instruction_counter += 1;
            logical_step += 1;

            let backend_result = match &located.instruction {
                Instruction::QAlloc(qubit) => {
                    allocated_qubit = Some(*qubit);
                    let result = engine.allocate(*qubit);
                    if result.is_ok() {
                        state.virtual_qubits.push(*qubit);
                    }
                    result
                }
                Instruction::QReset(qubit) => {
                    let probability_one =
                        engine
                            .probability_one(*qubit)
                            .map_err(|source| RuntimeError::Backend {
                                shot,
                                program_counter: pc,
                                source,
                            })?;
                    let outcome = if probability_one <= f64::EPSILON {
                        false
                    } else if (1.0 - probability_one) <= f64::EPSILON {
                        true
                    } else {
                        rng.random::<f64>() < probability_one
                    };
                    let collapse_result = engine.collapse(*qubit, outcome);
                    if collapse_result.is_ok() {
                        reset_outcome = Some(outcome);
                        if outcome {
                            engine.apply_single(*qubit, SingleQubitGate::X)
                        } else {
                            Ok(())
                        }
                    } else {
                        collapse_result
                    }
                }
                Instruction::QFree(qubit) => {
                    let result = engine.free(*qubit);
                    if result.is_ok() {
                        freed_qubit = Some(*qubit);
                        state.virtual_qubits.retain(|candidate| candidate != qubit);
                    }
                    result
                }
                Instruction::QH(qubit) => engine.apply_single(*qubit, SingleQubitGate::H),
                Instruction::QX(qubit) => engine.apply_single(*qubit, SingleQubitGate::X),
                Instruction::QY(qubit) => engine.apply_single(*qubit, SingleQubitGate::Y),
                Instruction::QZ(qubit) => engine.apply_single(*qubit, SingleQubitGate::Z),
                Instruction::QS(qubit) => engine.apply_single(*qubit, SingleQubitGate::S),
                Instruction::QSdg(qubit) => engine.apply_single(*qubit, SingleQubitGate::Sdg),
```

```
Instruction::QT(qubit) => engine.apply_single(*qubit, SingleQubitGate::T),
Instruction::QTdg(qubit) => engine.apply_single(*qubit, SingleQubitGate::Tdg),
Instruction::QRx { qubit, angle } => {
    engine.apply_single(*qubit, SingleQubitGate::Rx(*angle))
}
Instruction::QRy { qubit, angle } => {
    engine.apply_single(*qubit, SingleQubitGate::Ry(*angle))
}
Instruction::QRz { qubit, angle } => {
    engine.apply_single(*qubit, SingleQubitGate::Rz(*angle))
}
Instruction::QCx { control, target } => engine.controlled_x(*control, *target),
Instruction::QCz { control, target } => engine.controlled_z(*control, *target),
Instruction::QCphase {
    control,
    target,
    angle,
} => engine.controlled_phase(*control, *target, *angle),
Instruction::QSwap { first, second } => engine.swap(*first, *second),
Instruction::QCcx {
    first_control,
    second_control,
    target,
} => engine.controlled_controlled_x(*first_control, *second_control, *target),
Instruction::QMeasure { qubit, destination } => {
    let probability_one =
        engine
            .probability_one(*qubit)
            .map_err(|source| RuntimeError::Backend {
                shot,
                program_counter: pc,
                source,
            })?;
    let outcome = if probability_one <= f64::EPSILON {
        false
    } else if (1.0 - probability_one) <= f64::EPSILON {
        true
    } else {
        rng.random::<f64>() < probability_one
    };
    let result = engine.collapse(*qubit, outcome);
    if result.is_ok() {
        state.measurement_registers[destination.index()] = if outcome {
            MeasurementValue::One
        } else {
            MeasurementValue::Zero
        };
        measurement = Some(MeasurementChange {
            register: *destination,
            value: outcome,
        });
    }
    result
}
Instruction::MovMeasurement {
    destination,
    source,
} => {
    let value = match state.measurement_registers[source.index()] {
        MeasurementValue::Zero => 0,
        MeasurementValue::One => 1,
        MeasurementValue::Unset => {
            state.status = ProcessorStatus::Trapped;
            return Err(RuntimeError::ArchitecturalTrap {
                shot,
                program_counter: pc,
                message: format!("cannot read unset measurement register {source}"),
            });
        }
    };
    state.classical_registers[destination.index()] = value;
    classical = Some(ClassicalChange {
        register: *destination,
        value,
    });
    Ok(())
}
Instruction::MovRegister {
    destination,
    source,
} => {
    let value = state.classical_registers[source.index()];
    state.classical_registers[destination.index()] = value;
    classical = Some(ClassicalChange {
        register: *destination,
        value,
    });
    Ok(())
}
Instruction::LoadImmediate { destination, value } => {
    state.classical_registers[destination.index()] = *value;
    classical = Some(ClassicalChange {
        register: *destination,
        value: *value,
    });
    Ok(())
}
Instruction::ClassicalBinary {
    operation,
    destination,
    left,
    right,
} => {
    let left_value = state.classical_registers[left.index()];
    let right_value = classical_value(&state, *right);
    let value =
        execute_binary(*operation, left_value, right_value).map_err(|message| {
```

```
                RuntimeError::ArchitecturalTrap {
                    shot,
                    program_counter: pc,
                    message,
                }
            })?;
        state.classical_registers[destination.index()] = value;
        classical = Some(ClassicalChange {
            register: *destination,
            value,
        });
        Ok(())
    }
    Instruction::ClassicalNot {
        destination,
        source,
    } => {
        let value = !state.classical_registers[source.index()];
        state.classical_registers[destination.index()] = value;
        classical = Some(ClassicalChange {
            register: *destination,
            value,
        });
        Ok(())
    }
    Instruction::Compare { left, right } => {
        let right_value = classical_value(&state, *right);
        state.comparison = Some(
            match state.classical_registers[left.index()].cmp(&right_value) {
                std::cmp::Ordering::Less => ComparisonValue::Less,
                std::cmp::Ordering::Equal => ComparisonValue::Equal,
                std::cmp::Ordering::Greater => ComparisonValue::Greater,
            },
        );
        Ok(())
    }
    Instruction::Jump { target } => {
        state.program_counter = *target;
        branch_taken = Some(true);
        Ok(())
    }
    Instruction::JumpZero { target } => {
        let Some(comparison) = state.comparison else {
            state.status = ProcessorStatus::Trapped;
            return Err(RuntimeError::ArchitecturalTrap {
                shot,
                program_counter: pc,
                message: "JZ requires an earlier CMP in the executed path".to_owned(),
            });
        };
        let taken = comparison == ComparisonValue::Equal;
        if taken {
            state.program_counter = *target;
        }
        branch_taken = Some(taken);
        Ok(())
    }
    Instruction::JumpNotZero { target }
    | Instruction::JumpLess { target }
    | Instruction::JumpGreater { target } => {
        let Some(comparison) = state.comparison else {
            state.status = ProcessorStatus::Trapped;
            return Err(RuntimeError::ArchitecturalTrap {
                shot,
                program_counter: pc,
                message: format!(
                    "{} requires an earlier CMP",
                    located.instruction.mnemonic()
                ),
            });
        };
        let taken = match located.instruction {
            Instruction::JumpNotZero { .. } => comparison != ComparisonValue::Equal,
            Instruction::JumpLess { .. } => comparison == ComparisonValue::Less,
            Instruction::JumpGreater { .. } => comparison == ComparisonValue::Greater,
            _ => unreachable!("matched conditional branch"),
        };
        if taken {
            state.program_counter = *target;
        }
        branch_taken = Some(taken);
        Ok(())
    }
    Instruction::Halt => {
        state.status = ProcessorStatus::Halted;
        Ok(())
    }
};

if let Err(source) = backend_result {
    state.status = ProcessorStatus::Trapped;
    return Err(RuntimeError::Backend {
        shot,
        program_counter: pc,
        source,
    });
}

trace.push(TraceEvent {
    shot,
    step: logical_step,
    program_counter: pc,
    mnemonic: located.instruction.mnemonic().to_owned(),
    source_span: located.span,
    status_before: trace_status(status_before),
    status_after: trace_status(state.status),
    allocated_qubit,
```

```
                freed_qubit,
                reset_outcome,
                measurement,
                classical,
                branch_taken,
                comparison: state.comparison.map(trace_comparison),
                state_snapshot: state_snapshot(&engine, executable.metadata.trace_level),
            });

            if state.status == ProcessorStatus::Halted {
                break;
            }
        }

        if state.status != ProcessorStatus::Halted {
            state.status = ProcessorStatus::Completed;
            return Err(RuntimeError::MissingHalt { shot });
        }
        let key = measurement_key(&state.measurement_registers, measurement_width);
        *counts.entry(key).or_insert(0) += 1;
        final_state = state;
    }

    Ok(ExecutionResult {
        format: "clio-result-hybrid-path-1".to_owned(),
        source_sha256: executable.source_sha256.clone(),
        engine: ENGINE_IDENTITY.to_owned(),
        rng: RNG_IDENTITY.to_owned(),
        seed: executable.metadata.seed,
        shots: executable.metadata.shots,
        measurement_counts: counts,
        plan: execution_plan,
        final_state,
        trace,
    })
}

const fn trace_comparison(value: ComparisonValue) -> TraceComparison {
    match value {
        ComparisonValue::Less => TraceComparison::Less,
        ComparisonValue::Equal => TraceComparison::Equal,
        ComparisonValue::Greater => TraceComparison::Greater,
    }
}

fn classical_value(state: &ProcessorState, value: ClassicalValue) -> i64 {
    match value {
        ClassicalValue::Register(register) => state.classical_registers[register.index()],
        ClassicalValue::Immediate(immediate) => immediate,
    }
}

fn execute_binary(
    operation: ClassicalBinaryOperation,
    left: i64,
    right: i64,
) -> Result<i64, String> {
    match operation {
        ClassicalBinaryOperation::Add => left
            .checked_add(right)
            .ok_or_else(|| "signed addition overflow".to_owned()),
        ClassicalBinaryOperation::Subtract => left
            .checked_sub(right)
            .ok_or_else(|| "signed subtraction overflow".to_owned()),
        ClassicalBinaryOperation::Multiply => left
            .checked_mul(right)
            .ok_or_else(|| "signed multiplication overflow".to_owned()),
        ClassicalBinaryOperation::Divide => {
            if right == 0 {
                Err("division by zero".to_owned())
            } else {
                left.checked_div(right)
                    .ok_or_else(|| "signed division overflow".to_owned())
            }
        }
        ClassicalBinaryOperation::Modulo => {
            if right == 0 {
                Err("modulo by zero".to_owned())
            } else {
                left.checked_rem(right)
                    .ok_or_else(|| "signed modulo overflow".to_owned())
            }
        }
        ClassicalBinaryOperation::And => Ok(left & right),
        ClassicalBinaryOperation::Or => Ok(left | right),
        ClassicalBinaryOperation::Xor => Ok(left ^ right),
        ClassicalBinaryOperation::ShiftLeft => checked_shift(right)
            .and_then(|shift| left.checked_shl(shift))
            .ok_or_else(|| "invalid or overflowing left shift".to_owned()),
        ClassicalBinaryOperation::ShiftRight => checked_shift(right)
            .and_then(|shift| left.checked_shr(shift))
            .ok_or_else(|| "invalid right shift".to_owned()),
    }
}

fn checked_shift(value: i64) -> Option<u32> {
    u32::try_from(value).ok().filter(|shift| *shift < 64)
}

fn state_snapshot(
    engine: &StateVectorEngine,
    level: clio_core::TraceLevel,
) -> Option<Vec<StateAmplitude>> {
    if engine.qubit_count() > 8
        || !matches!(
            level,
            clio_core::TraceLevel::StateSmall | clio_core::TraceLevel::FullDebug
```

```
        )
    {
        return None;
    }
    Some(
        engine
            .amplitudes()
            .iter()
            .enumerate()
            .map(|(basis_index, amplitude)| StateAmplitude {
                basis_index,
                real: amplitude.re,
                imaginary: amplitude.im,
                probability: amplitude.norm_sqr(),
            })
            .collect(),
    )
}

fn trace_status(status: ProcessorStatus) -> TraceStatus {
    match status {
        ProcessorStatus::Ready => TraceStatus::Ready,
        ProcessorStatus::Running | ProcessorStatus::Paused => TraceStatus::Running,
        ProcessorStatus::Completed => TraceStatus::Completed,
        ProcessorStatus::Halted => TraceStatus::Halted,
        ProcessorStatus::Trapped
        | ProcessorStatus::ResourceRejected
        | ProcessorStatus::ValidationFailed => TraceStatus::Trapped,
    }
}

fn measurement_width(executable: &Executable) -> usize {
    executable
        .instructions
        .iter()
        .filter_map(|located| match located.instruction {
            Instruction::QMeasure { destination, .. } => Some(destination.index() + 1),
            _ => None,
        })
        .max()
        .unwrap_or(0)
}

fn measurement_key(
    registers: &[MeasurementValue; REGISTER_COUNT],
    measurement_width: usize,
) -> String {
    registers[..measurement_width]
        .iter()
        .rev()
        .copied()
        .map(|value| match value {
            MeasurementValue::Zero => '0',
            MeasurementValue::One => '1',
            MeasurementValue::Unset => 'x',
        })
        .collect()
}

fn shot_rng(seed: u64, shot: u64) -> ChaCha8Rng {
    let mut state = splitmix64(seed ^ shot);
    let mut bytes = [0_u8; 32];
    for chunk in bytes.chunks_exact_mut(8) {
        state = splitmix64(state);
        chunk.copy_from_slice(&state.to_le_bytes());
    }
    ChaCha8Rng::from_seed(bytes)
}

fn splitmix64(mut value: u64) -> u64 {
    value = value.wrapping_add(0x9e37_79b9_7f4a_7c15);
    let mut result = value;
    result = (result ^ (result >> 30)).wrapping_mul(0xbf58_476d_1ce4_e5b9);
    result = (result ^ (result >> 27)).wrapping_mul(0x94d0_49bb_1331_11eb);
    result ^ (result >> 31)
}

#[cfg(test)]
mod tests {
    use super::*;
    use clio_assembler::assemble;

    const BELL: &str = ".seed 42\n.shots 1024\n.trace instructions\nQALLOC q0\nQALLOC q1\nQH q0\nQCX q0, q1\nQMEASURE q0, m0\nQMEASURE q1,
       m1\nHALT\n";

    #[test]
    fn bell_run_has_only_correlated_outcomes() {
        let executable = assemble(BELL).expect("assemble Bell");
        let result = run(&executable, ResourceLimits::default()).expect("run Bell");
        assert_eq!(result.measurement_counts.values().sum::<u64>(), 1024);
        assert!(
            result
                .measurement_counts
                .keys()
                .all(|key| key == "00" || key == "11")
        );
        assert!(result.measurement_counts.contains_key("00"));
        assert!(result.measurement_counts.contains_key("11"));
    }

    #[test]
    fn same_seed_reproduces_counts_and_trace() {
        let executable = assemble(BELL).expect("assemble Bell");
        let first = run(&executable, ResourceLimits::default()).expect("first run");
        let second = run(&executable, ResourceLimits::default()).expect("second run");
        assert_eq!(first.measurement_counts, second.measurement_counts);
        assert_eq!(first.trace, second.trace);
```

```
}

#[test]
fn final_measurements_are_not_unset() {
    let executable = assemble(BELL).expect("assemble Bell");
    let result = run(&executable, ResourceLimits::default()).expect("run Bell");
    assert_ne!(
        result.final_state.measurement_registers[0],
        MeasurementValue::Unset
    );
    assert_ne!(
        result.final_state.measurement_registers[1],
        MeasurementValue::Unset
    );
}

#[test]
fn measurement_drives_classical_control_flow() {
    let source = ".seed 9\n.shots 128\n.trace instructions\nQALLOC q0\nQH q0\nQMEASURE q0, m0\nMOV r0, m0\nCMP r0, 0\nJZ
      measured_zero\nLOADI r1, 1\nJMP done\nmeasured_zero: LOADI r1, 0\ndone: HALT\n";
    let executable = assemble(source).expect("assemble hybrid program");
    let result = run(&executable, ResourceLimits::default()).expect("run hybrid program");
    assert!(result.measurement_counts.contains_key("0"));
    assert!(result.measurement_counts.contains_key("1"));
    let final_measurement = result.final_state.measurement_registers[0];
    let expected = i64::from(final_measurement == MeasurementValue::One);
    assert_eq!(result.final_state.classical_registers[1], expected);
    assert!(
        result
            .trace
            .events
            .iter()
            .any(|event| event.branch_taken == Some(true))
    );
    assert!(
        result
            .trace
            .events
            .iter()
            .any(|event| event.branch_taken == Some(false))
    );
}

#[test]
fn moving_an_unset_measurement_traps() {
    let executable = assemble("MOV r0, m0\nHALT\n").expect("assemble");
    assert!(matches!(
        run(&executable, ResourceLimits::default()),
        Err(RuntimeError::ArchitecturalTrap { .. })
    ));
}

#[test]
fn rotation_program_executes_through_runtime() {
    let source = "QALLOC q0\nQRX q0, 0.25\nQRY q0, -0.75\nQRZ q0, 1.5\nQSDG q0\nQTDG q0\nQMEASURE q0, m0\nHALT\n";
    let executable = assemble(source).expect("assemble rotations");
    let result = run(&executable, ResourceLimits::default()).expect("run rotations");
    assert_eq!(result.measurement_counts.values().sum::<u64>(), 1);
    assert_eq!(result.final_state.status, ProcessorStatus::Halted);
}

#[test]
fn reset_then_free_updates_real_engine_and_processor_state() {
    let source =
        ".seed 4\n.shots 8\n.trace instructions\nQALLOC q0\nQX q0\nQRESET q0\nQFREE q0\nHALT\n";
    let executable = assemble(source).expect("assemble lifecycle");
    let result = run(&executable, ResourceLimits::default()).expect("run lifecycle");
    assert!(result.final_state.virtual_qubits.is_empty());
    assert_eq!(
        result
            .trace
            .events
            .iter()
            .filter(|event| event.reset_outcome == Some(true))
            .count(),
        8
    );
    assert_eq!(
        result
            .trace
            .events
            .iter()
            .filter(|event| event.freed_qubit == Some(QubitId::new(0)))
            .count(),
        8
    );
}

#[test]
fn multi_qubit_operations_execute_through_runtime() {
    let source = "QALLOC q0\nQALLOC q1\nQALLOC q2\nQX q0\nQX q1\nQCCX q0, q1, q2\nQCZ q1, q2\nQSWAP q0, q2\nQMEASURE q0, m0\nQMEASURE
      q1, m1\nQMEASURE q2, m2\nHALT\n";
    let executable = assemble(source).expect("assemble multi-qubit program");
    let result = run(&executable, ResourceLimits::default()).expect("run multi-qubit program");
    assert_eq!(result.measurement_counts.get("111"), Some(&1));
}

#[test]
fn classical_arithmetic_logic_and_register_comparison_execute() {
    let source = ".trace off\nLOADI r0, 9\nMOV r1, r0\nADD r2, r0, 3\nSUB r3, r2, r1\nMUL r4, r3, 7\nDIV r5, r4, 3\nMOD r6, r5, 5\nAND
       r7, r0, 3\nOR r8, r7, 8\nXOR r9, r8, r0\nNOT r10, r9\nSHL r11, r7, 3\nSHR r12, r11, 2\nCMP r0, r1\nJNZ fail\nJMP done\nfail:
      LOADI r15, -1\ndone: HALT\n";
    let executable = assemble(source).expect("assemble classical program");
    let result = run(&executable, ResourceLimits::default()).expect("run classical program");
    let registers = result.final_state.classical_registers;
    assert_eq!(registers[2], 12);
    assert_eq!(registers[3], 3);
```

```
        assert_eq!(registers[4], 21);
        assert_eq!(registers[5], 7);
        assert_eq!(registers[6], 2);
        assert_eq!(registers[12], 2);
        assert_eq!(registers[15], 0);
    }

    #[test]
    fn checked_arithmetic_traps_on_invalid_operations() {
        for source in [
            ".trace off\nLOADI r0, 9223372036854775807\nADD r1, r0, 1\nHALT\n",
            ".trace off\nLOADI r0, -9223372036854775808\nDIV r1, r0, -1\nHALT\n",
            ".trace off\nLOADI r0, 7\nDIV r1, r0, 0\nHALT\n",
            ".trace off\nLOADI r0, 7\nMOD r1, r0, 0\nHALT\n",
            ".trace off\nLOADI r0, 1\nSHL r1, r0, 64\nHALT\n",
            ".trace off\nLOADI r0, 1\nSHR r1, r0, -1\nHALT\n",
        ] {
            let executable = assemble(source).expect("assemble trap case");
            assert!(matches!(
                run(&executable, ResourceLimits::default()),
                Err(RuntimeError::ArchitecturalTrap { .. })
            ));
        }
    }

    #[test]
    fn backward_loop_runs_under_instruction_budget() {
        let source =
            ".trace instructions\nLOADI r0, 0\nloop: ADD r0, r0, 1\nCMP r0, 5\nJLT loop\nHALT\n";
        let executable = assemble(source).expect("assemble loop");
        assert!(executable.has_backward_branches);
        let limits = ResourceLimits {
            max_instructions: 100,
            ..ResourceLimits::default()
        };
        let result = run(&executable, limits).expect("bounded loop completes");
        assert_eq!(result.final_state.classical_registers[0], 5);
        assert!(result.trace.events.iter().any(|event| {
            event.mnemonic == "JLT"
                && event.branch_taken == Some(true)
                && event.comparison == Some(TraceComparison::Less)
        }));
        assert!(result.trace.events.iter().any(|event| {
            event.mnemonic == "JLT"
                && event.branch_taken == Some(false)
                && event.comparison == Some(TraceComparison::Equal)
        }));
    }

    #[test]
    fn jgt_jnz_and_jz_follow_explicit_comparisons() {
        let source = ".trace off\nLOADI r0, 2\nCMP r0, 1\nJGT greater\nLOADI r15, -1\ngreater: CMP r0, 0\nJNZ nonzero\nLOADI r15,
          -2\nnonzero: CMP r0, 2\nJZ equal\nLOADI r15, -3\nequal: LOADI r14, 1\nHALT\n";
        let result = run(
            &assemble(source).expect("assemble branches"),
            ResourceLimits::default(),
        )
        .expect("run branches");
        assert_eq!(result.final_state.classical_registers[14], 1);
        assert_eq!(result.final_state.classical_registers[15], 0);
    }

    #[test]
    fn conditional_branch_without_cmp_traps() {
        let executable = assemble(".trace off\nJGT done\ndone: HALT\n").expect("assemble");
        let error = run(&executable, ResourceLimits::default()).expect_err("missing CMP traps");
        assert_eq!(error.code(), "T203");
    }

    #[test]
    fn infinite_loop_hits_instruction_budget() {
        let executable = assemble(".trace off\nloop: JMP loop\nHALT\n").expect("assemble loop");
        let limits = ResourceLimits {
            max_instructions: 25,
            ..ResourceLimits::default()
        };
        assert!(matches!(
            run(&executable, limits),
            Err(RuntimeError::InstructionBudget { limit: 25, .. })
        ));
        let error = run(&executable, limits).expect_err("loop must trap again");
        assert_eq!(error.code(), "T204");
    }

    #[test]
    fn zero_time_budget_traps_before_execution() {
        let executable = assemble(".trace off\nHALT\n").expect("assemble");
        let limits = ResourceLimits {
            max_execution_millis: 0,
            ..ResourceLimits::default()
        };
        assert!(matches!(
            run(&executable, limits),
            Err(RuntimeError::TimeBudget { .. })
        ));
    }

    #[test]
    fn admitted_state_trace_bound_covers_serialized_trace() {
        let source = ".shots 32\n.trace state-small\nQALLOC q0\nQALLOC q1\nQH q0\nQCX q0, q1\nQMEASURE q0, m0\nQMEASURE q1, m1\nHALT\n";
        let executable = assemble(source).expect("assemble");
        let result = run(&executable, ResourceLimits::default()).expect("run");
        let serialized = u64::try_from(serde_json::to_vec(&result.trace).expect("serialize").len())
            .expect("trace size");
        assert!(result.plan.estimated_trace_bytes >= serialized);
    }
}
```

Appendix: crates/clio-assembler/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Semantic validation and assembly for the Bell-path Clio ISA subset.

#![forbid(unsafe_code)]

use clio_core::{
    ClassicalRegister, Diagnostic, MeasurementRegister, ProgramMetadata, QubitId, TraceLevel,
};
use clio_isa::{
    ClassicalBinaryOperation, ClassicalValue, Executable, Instruction, LocatedInstruction,
};
use clio_parser::{ParsedInstruction, parse};
use sha2::{Digest, Sha256};
use std::{
    collections::{HashMap, HashSet},
    fmt::Write as _,
    str::FromStr,
};

/// Parses, validates, and assembles source into a typed executable.
pub fn assemble(source: &str) -> Result<Executable, Vec<Diagnostic>> {
    let parsed = parse(source)?;
    let mut diagnostics = Vec::new();
    let metadata = parse_metadata(&parsed.directives, &mut diagnostics);
    let mut labels = HashMap::new();
    for label in &parsed.labels {
        if labels
            .insert(label.name.as_str(), label.instruction_index)
            .is_some()
        {
            diagnostics.push(Diagnostic::error(
                "E107",
                format!("duplicate label `{}`", label.name),
                label.span,
            ));
        }
    }
    let mut allocated = HashSet::new();
    let mut allocation_closed = false;
    let mut instructions = Vec::with_capacity(parsed.instructions.len());
    let mut required_qubits = 0_u16;

    for parsed_instruction in &parsed.instructions {
        match assemble_instruction(
            parsed_instruction,
            &labels,
            &mut allocated,
            &mut allocation_closed,
            &mut required_qubits,
        ) {
            Ok(instruction) => instructions.push(LocatedInstruction {
                instruction,
                span: Some(parsed_instruction.span),
            }),
            Err(diagnostic) => diagnostics.push(diagnostic),
        }
    }

    if instructions.is_empty() {
        diagnostics.push(Diagnostic {
            code: "E100".to_owned(),
            message: "program contains no instructions".to_owned(),
            span: None,
            help: Some("add at least HALT".to_owned()),
        });
    }

    if diagnostics.is_empty() {
        let has_backward_branches = instructions.iter().enumerate().any(|(index, located)| {
            branch_target(&located.instruction).is_some_and(|target| target <= index)
        });
        Ok(Executable {
            architecture: "clio-vqpu".to_owned(),
            isa_revision: "hybrid-path-1".to_owned(),
            source_sha256: format!("{:x}", Sha256::digest(source.as_bytes())),
            metadata,
            instructions,
            required_qubits,
            has_backward_branches,
        })
    } else {
        Err(diagnostics)
    }
}

/// Produces a stable human-readable numbered instruction listing.
#[must_use]
pub fn disassemble(executable: &Executable) -> String {
    let mut output = String::new();
    for (program_counter, located) in executable.instructions.iter().enumerate() {
        let operands = instruction_operands(&located.instruction);
        writeln!(
            output,
            "{program_counter:04} {:<10} {operands}",
            located.instruction.mnemonic()
        )
        .expect("writing to String cannot fail");
    }
    output
}
```

```
fn instruction_operands(instruction: &Instruction) -> String {
    match instruction {
        Instruction::QAlloc(q)
        | Instruction::QReset(q)
        | Instruction::QFree(q)
        | Instruction::QH(q)
        | Instruction::QX(q)
        | Instruction::QY(q)
        | Instruction::QZ(q)
        | Instruction::QS(q)
        | Instruction::QSdg(q)
        | Instruction::QT(q)
        | Instruction::QTdg(q) => q.to_string(),
        Instruction::QRx { qubit, angle }
        | Instruction::QRy { qubit, angle }
        | Instruction::QRz { qubit, angle } => format!("{qubit}, {angle}"),
        Instruction::QCx { control, target } | Instruction::QCz { control, target } => {
            format!("{control}, {target}")
        }
        Instruction::QCphase {
            control,
            target,
            angle,
        } => format!("{control}, {target}, {angle}"),
        Instruction::QSwap { first, second } => format!("{first}, {second}"),
        Instruction::QCcx {
            first_control,
            second_control,
            target,
        } => format!("{first_control}, {second_control}, {target}"),
        Instruction::QMeasure { qubit, destination } => format!("{qubit}, {destination}"),
        Instruction::MovMeasurement {
            destination,
            source,
        } => format!("{destination}, {source}"),
        Instruction::MovRegister {
            destination,
            source,
        }
        | Instruction::ClassicalNot {
            destination,
            source,
        } => format!("{destination}, {source}"),
        Instruction::LoadImmediate { destination, value } => format!("{destination}, {value}"),
        Instruction::ClassicalBinary {
            destination,
            left,
            right,
            ..
        } => format!("{destination}, {left}, {}", display_value(*right)),
        Instruction::Compare { left, right } => format!("{left}, {}", display_value(*right)),
        Instruction::Jump { target }
        | Instruction::JumpZero { target }
        | Instruction::JumpNotZero { target }
        | Instruction::JumpLess { target }
        | Instruction::JumpGreater { target } => format!("@{target}"),
        Instruction::Halt => String::new(),
    }
}

fn display_value(value: ClassicalValue) -> String {
    match value {
        ClassicalValue::Register(register) => register.to_string(),
        ClassicalValue::Immediate(immediate) => immediate.to_string(),
    }
}

fn branch_target(instruction: &Instruction) -> Option<usize> {
    match instruction {
        Instruction::Jump { target }
        | Instruction::JumpZero { target }
        | Instruction::JumpNotZero { target }
        | Instruction::JumpLess { target }
        | Instruction::JumpGreater { target } => Some(*target),
        _ => None,
    }
}

fn parse_metadata(
    directives: &[clio_parser::Directive],
    diagnostics: &mut Vec<Diagnostic>,
) -> ProgramMetadata {
    let mut metadata = ProgramMetadata::default();
    let mut seen_directives = HashSet::new();
    for directive in directives {
        if !seen_directives.insert(directive.name.as_str()) {
            diagnostics.push(Diagnostic::error(
                "E101",
                format!("duplicate .{} directive", directive.name),
                directive.span,
            ));
            continue;
        }
        match directive.name.as_str() {
            "program" => metadata.name.clone_from(&directive.value),
            "seed" => match directive.value.parse() {
                Ok(seed) => metadata.seed = seed,
                Err(_) => diagnostics.push(Diagnostic::error(
                    "E102",
                    "seed must be an unsigned 64-bit integer",
                    directive.span,
                )),
            },
            "shots" => match directive.value.parse::<u64>() {
                Ok(0) | Err(_) => diagnostics.push(Diagnostic::error(
                    "E103",
                    "shots must be a positive unsigned integer",
```

```
                    directive.span,
                )),
                Ok(shots) => metadata.shots = shots,
            },
            "trace" => match directive.value.to_ascii_lowercase().as_str() {
                "off" => metadata.trace_level = TraceLevel::Off,
                "summary" => metadata.trace_level = TraceLevel::Summary,
                "instructions" => metadata.trace_level = TraceLevel::Instructions,
                "state-small" => metadata.trace_level = TraceLevel::StateSmall,
                "full-debug" => metadata.trace_level = TraceLevel::FullDebug,
                _ => diagnostics.push(Diagnostic::error(
                    "E104",
                    "unknown trace level",
                    directive.span,
                )),
            },
            "budget" => match parse_memory_budget(&directive.value) {
                Some(bytes) => metadata.memory_budget_bytes = Some(bytes),
                None => diagnostics.push(
                    Diagnostic::error(
                        "E105",
                        "budget must have the form memory=<positive size>",
                        directive.span,
                    )
                    .with_help("use bytes, KB, MB, GB, KiB, MiB, or GiB"),
                ),
            },
            _ => diagnostics.push(Diagnostic::error(
                "E106",
                format!("unknown directive .{}", directive.name),
                directive.span,
            )),
        }
    }
    metadata
}

fn parse_memory_budget(value: &str) -> Option<u64> {
    let raw = value.strip_prefix("memory=")?;
    let digit_count = raw.bytes().take_while(u8::is_ascii_digit).count();
    let (number, suffix) = raw.split_at(digit_count);
    let number = number.parse::<u64>().ok()?;
    let multiplier = match suffix.to_ascii_lowercase().as_str() {
        "" | "b" | "bytes" => 1,
        "kb" => 1_000,
        "mb" => 1_000_000,
        "gb" => 1_000_000_000,
        "kib" => 1 << 10,
        "mib" => 1 << 20,
        "gib" => 1 << 30,
        _ => return None,
    };
    number.checked_mul(multiplier).filter(|bytes| *bytes > 0)
}

fn assemble_instruction(
    parsed: &ParsedInstruction,
    labels: &HashMap<&str, usize>,
    allocated: &mut HashSet<QubitId>,
    allocation_closed: &mut bool,
    required_qubits: &mut u16,
) -> Result<Instruction, Diagnostic> {
    let wrong_arity = |expected: usize| {
        Diagnostic::error(
            "E110",
            format!(
                "{} expects {expected} operand(s), found {}",
                parsed.mnemonic,
                parsed.operands.len()
            ),
            parsed.span,
        )
    };
    match parsed.mnemonic.as_str() {
        "QALLOC" => {
            if parsed.operands.len() != 1 {
                return Err(wrong_arity(1));
            }
            let qubit = parse_qubit(&parsed.operands[0], parsed)?;
            if *allocation_closed {
                return Err(Diagnostic::error(
                    "E125",
                    "allocation after QFREE is not supported by the initial lifecycle model",
                    parsed.span,
                ));
            }
            if qubit.index() != usize::from(*required_qubits) {
                return Err(Diagnostic::error(
                    "E111",
                    format!("expected contiguous allocation of q{required_qubits}"),
                    parsed.span,
                ));
            }
            if !allocated.insert(qubit) {
                return Err(Diagnostic::error(
                    "E112",
                    format!("{qubit} is already allocated"),
                    parsed.span,
                ));
            }
            *required_qubits = required_qubits.saturating_add(1);
            Ok(Instruction::QAlloc(qubit))
        }
        "QRESET" => {
            if parsed.operands.len() != 1 {
                return Err(wrong_arity(1));
            }
```

```
    Ok(Instruction::QReset(parse_allocated(
        &parsed.operands[0],
        parsed,
        allocated,
    )?))
}
"QFREE" => {
    if parsed.operands.len() != 1 {
        return Err(wrong_arity(1));
    }
    let qubit = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    let highest_allocated = allocated.iter().map(|candidate| candidate.index()).max();
    if highest_allocated != Some(qubit.index()) {
        return Err(Diagnostic::error(
            "E126",
            "QFREE currently requires the highest mapped qubit",
            parsed.span,
        ));
    }
    allocated.remove(&qubit);
    *allocation_closed = true;
    Ok(Instruction::QFree(qubit))
}
"QH" => {
    if parsed.operands.len() != 1 {
        return Err(wrong_arity(1));
    }
    let qubit = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    Ok(Instruction::QH(qubit))
}
"QX" | "QY" | "QZ" | "QS" | "QSDG" | "QT" | "QTDG" => {
    if parsed.operands.len() != 1 {
        return Err(wrong_arity(1));
    }
    let qubit = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    Ok(match parsed.mnemonic.as_str() {
        "QX" => Instruction::QX(qubit),
        "QY" => Instruction::QY(qubit),
        "QZ" => Instruction::QZ(qubit),
        "QS" => Instruction::QS(qubit),
        "QSDG" => Instruction::QSdg(qubit),
        "QT" => Instruction::QT(qubit),
        "QTDG" => Instruction::QTdg(qubit),
        _ => unreachable!("matched fixed gate mnemonic"),
    })
}
"QRX" | "QRY" | "QRZ" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    let qubit = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    let angle = parse_angle(&parsed.operands[1], parsed)?;
    Ok(match parsed.mnemonic.as_str() {
        "QRX" => Instruction::QRx { qubit, angle },
        "QRY" => Instruction::QRy { qubit, angle },
        "QRZ" => Instruction::QRz { qubit, angle },
        _ => unreachable!("matched rotation mnemonic"),
    })
}
"QCX" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    let control = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    let target = parse_allocated(&parsed.operands[1], parsed, allocated)?;
    if control == target {
        return Err(Diagnostic::error(
            "E113",
            "QCX control and target must be distinct",
            parsed.span,
        ));
    }
    Ok(Instruction::QCx { control, target })
}
"QCPHASE" => {
    if parsed.operands.len() != 3 { return Err(wrong_arity(3)); }
    let control = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    let target = parse_allocated(&parsed.operands[1], parsed, allocated)?;
    if control == target { return Err(Diagnostic::error("E113", "QCPHASE control and target must be distinct", parsed.span)); }
    let angle = parse_angle(&parsed.operands[2], parsed)?;
    Ok(Instruction::QCphase { control, target, angle })
}
"QCZ" | "QSWAP" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    let first = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    let second = parse_allocated(&parsed.operands[1], parsed, allocated)?;
    if first == second {
        return Err(Diagnostic::error(
            "E113",
            "multi-qubit operands must be distinct",
            parsed.span,
        ));
    }
    if parsed.mnemonic == "QCZ" {
        Ok(Instruction::QCz {
            control: first,
            target: second,
        })
    } else {
        Ok(Instruction::QSwap { first, second })
    }
}
"QCCX" => {
    if parsed.operands.len() != 3 {
        return Err(wrong_arity(3));
```

```
    }
    let first_control = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    let second_control = parse_allocated(&parsed.operands[1], parsed, allocated)?;
    let target = parse_allocated(&parsed.operands[2], parsed, allocated)?;
    if first_control == second_control
        || first_control == target
        || second_control == target
    {
        return Err(Diagnostic::error(
            "E113",
            "QCCX operands must be distinct",
            parsed.span,
        ));
    }
    Ok(Instruction::QCcx {
        first_control,
        second_control,
        target,
    })
}
"QMEASURE" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    let qubit = parse_allocated(&parsed.operands[0], parsed, allocated)?;
    let destination = MeasurementRegister::from_str(&parsed.operands[1]).map_err(|_| {
        Diagnostic::error(
            "E114",
            "expected a measurement register m0..m15",
            parsed.span,
        )
    })?;
    Ok(Instruction::QMeasure { qubit, destination })
}
"MOV" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    let destination = parse_classical(&parsed.operands[0], parsed)?;
    if let Ok(source) = MeasurementRegister::from_str(&parsed.operands[1]) {
        Ok(Instruction::MovMeasurement {
            destination,
            source,
        })
    } else {
        Ok(Instruction::MovRegister {
            destination,
            source: parse_classical(&parsed.operands[1], parsed)?,
        })
    }
}
"LOADI" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    Ok(Instruction::LoadImmediate {
        destination: parse_classical(&parsed.operands[0], parsed)?,
        value: parse_immediate(&parsed.operands[1], parsed)?,
    })
}
"ADD" | "SUB" | "MUL" | "DIV" | "MOD" | "AND" | "OR" | "XOR" | "SHL"
| "SHR" => {
    if parsed.operands.len() != 3 {
        return Err(wrong_arity(3));
    }
    let operation = match parsed.mnemonic.as_str() {
        "ADD" => ClassicalBinaryOperation::Add,
        "SUB" => ClassicalBinaryOperation::Subtract,
        "MUL" => ClassicalBinaryOperation::Multiply,
        "DIV" => ClassicalBinaryOperation::Divide,
        "MOD" => ClassicalBinaryOperation::Modulo,
        "AND" => ClassicalBinaryOperation::And,
        "OR" => ClassicalBinaryOperation::Or,
        "XOR" => ClassicalBinaryOperation::Xor,
        "SHL" => ClassicalBinaryOperation::ShiftLeft,
        "SHR" => ClassicalBinaryOperation::ShiftRight,
        _ => unreachable!("matched classical binary mnemonic"),
    };
    Ok(Instruction::ClassicalBinary {
        operation,
        destination: parse_classical(&parsed.operands[0], parsed)?,
        left: parse_classical(&parsed.operands[1], parsed)?,
        right: parse_classical_value(&parsed.operands[2], parsed)?,
    })
}
"NOT" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    Ok(Instruction::ClassicalNot {
        destination: parse_classical(&parsed.operands[0], parsed)?,
        source: parse_classical(&parsed.operands[1], parsed)?,
    })
}
"CMP" => {
    if parsed.operands.len() != 2 {
        return Err(wrong_arity(2));
    }
    Ok(Instruction::Compare {
        left: parse_classical(&parsed.operands[0], parsed)?,
        right: parse_classical_value(&parsed.operands[1], parsed)?,
    })
}
"JMP" | "JZ" | "JNZ" | "JLT" | "JGT" => {
    if parsed.operands.len() != 1 {
        return Err(wrong_arity(1));
    }
```

```
            let label = parsed.operands[0].as_str();
            let target = labels.get(label).copied().ok_or_else(|| {
                Diagnostic::error(
                    "E119",
                    format!("unresolved branch label `{label}`"),
                    parsed.span,
                )
            })?;
            match parsed.mnemonic.as_str() {
                "JMP" => Ok(Instruction::Jump { target }),
                "JZ" => Ok(Instruction::JumpZero { target }),
                "JNZ" => Ok(Instruction::JumpNotZero { target }),
                "JLT" => Ok(Instruction::JumpLess { target }),
                "JGT" => Ok(Instruction::JumpGreater { target }),
                _ => unreachable!("matched branch mnemonic"),
            }
        }
        "HALT" => {
            if !parsed.operands.is_empty() {
                return Err(wrong_arity(0));
            }
            Ok(Instruction::Halt)
        }
        _ => Err(Diagnostic::error(
            "E115",
            format!("unknown or unsupported mnemonic {}", parsed.mnemonic),
            parsed.span,
        )
        .with_help(
            "supported mnemonics include QALLOC, single-qubit gates, QCX, QMEASURE, MOV, LOADI, CMP, JMP, JZ, and HALT",
        )),
    }
}

fn parse_classical(
    value: &str,
    parsed: &ParsedInstruction,
) -> Result<ClassicalRegister, Diagnostic> {
    ClassicalRegister::from_str(value).map_err(|_| {
        Diagnostic::error(
            "E120",
            format!("expected a classical register r0..r15, found `{value}`"),
            parsed.span,
        )
    })
}

fn parse_immediate(value: &str, parsed: &ParsedInstruction) -> Result<i64, Diagnostic> {
    value.parse::<i64>().map_err(|_| {
        Diagnostic::error(
            "E121",
            format!("expected a signed 64-bit integer, found `{value}`"),
            parsed.span,
        )
    })
}

fn parse_classical_value(
    value: &str,
    parsed: &ParsedInstruction,
) -> Result<ClassicalValue, Diagnostic> {
    if let Ok(immediate) = value.parse::<i64>() {
        Ok(ClassicalValue::Immediate(immediate))
    } else {
        parse_classical(value, parsed).map(ClassicalValue::Register)
    }
}

fn parse_angle(value: &str, parsed: &ParsedInstruction) -> Result<f64, Diagnostic> {
    let angle = value.parse::<f64>().map_err(|_| {
        Diagnostic::error(
            "E123",
            format!("expected an angle in radians, found `{value}`"),
            parsed.span,
        )
    })?;
    if angle.is_finite() {
        Ok(angle)
    } else {
        Err(Diagnostic::error(
            "E124",
            "rotation angle must be finite",
            parsed.span,
        ))
    }
}

fn parse_qubit(value: &str, parsed: &ParsedInstruction) -> Result<QubitId, Diagnostic> {
    QubitId::from_str(value).map_err(|_| {
        Diagnostic::error(
            "E116",
            format!("expected a virtual-qubit reference, found `{value}`"),
            parsed.span,
        )
    })
}

fn parse_allocated(
    value: &str,
    parsed: &ParsedInstruction,
    allocated: &HashSet<QubitId>,
) -> Result<QubitId, Diagnostic> {
    let qubit = parse_qubit(value, parsed)?;
    if allocated.contains(&qubit) {
        Ok(qubit)
    } else {
        Err(Diagnostic::error(
```

```
            "E117",
            format!("{qubit} is used before allocation"),
            parsed.span,
        ))
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    const BELL: &str = ".program bell\n.seed 42\n.shots 16\nQALLOC q0\nQALLOC q1\nQH q0\nQCX q0, q1\nQMEASURE q0, m0\nQMEASURE q1,
      m1\nHALT\n";

    #[test]
    fn assembles_bell_subset() {
        let executable = assemble(BELL).expect("Bell source assembles");
        assert_eq!(executable.instructions.len(), 7);
        assert_eq!(executable.required_qubits, 2);
        assert_eq!(executable.metadata.seed, 42);
    }

    #[test]
    fn rejects_wrong_measurement_destination() {
        let source = "QALLOC q0\nQMEASURE q0, r0\nHALT\n";
        let diagnostics = assemble(source).expect_err("r0 is not a measurement register");
        assert!(diagnostics.iter().any(|item| item.code == "E114"));
    }

    #[test]
    fn rejects_use_before_allocation() {
        let diagnostics = assemble("QH q0\nHALT\n").expect_err("q0 is unallocated");
        assert!(diagnostics.iter().any(|item| item.code == "E117"));
    }

    #[test]
    fn resolves_hybrid_branch_labels() {
        let source = "QALLOC q0\nQMEASURE q0, m0\nMOV r0, m0\nCMP r0, 0\nJZ zero\nLOADI r1, 1\nJMP done\nzero: LOADI r1, 0\ndone: HALT\n";
        let executable = assemble(source).expect("hybrid source assembles");
        assert_eq!(
            executable.instructions[4].instruction,
            Instruction::JumpZero { target: 7 }
        );
        assert_eq!(
            executable.instructions[6].instruction,
            Instruction::Jump { target: 8 }
        );
    }

    #[test]
    fn rejects_unknown_branch_labels() {
        let diagnostics = assemble("JMP missing\nHALT\n").expect_err("missing label");
        assert!(diagnostics.iter().any(|item| item.code == "E119"));
    }

    #[test]
    fn resolves_and_marks_backward_branches() {
        let executable = assemble("again: LOADI r0, 1\nJMP again\nHALT\n")
            .expect("backward branch assembles under runtime budgets");
        assert!(executable.has_backward_branches);
    }

    #[test]
    fn assembles_fixed_and_rotation_gates() {
        let source = "QALLOC q0\nQX q0\nQY q0\nQZ q0\nQS q0\nQSDG q0\nQT q0\nQTDG q0\nQRX q0, 0.5\nQRY q0, -1.25\nQRZ q0, 3.0\nHALT\n";
        let executable = assemble(source).expect("gate program assembles");
        assert_eq!(executable.instructions.len(), 12);
        assert!(matches!(
            executable.instructions[8].instruction,
            Instruction::QRx { angle, .. } if (angle - 0.5).abs() < f64::EPSILON
        ));
    }

    #[test]
    fn rejects_non_finite_rotation_angles() {
        let diagnostics =
            assemble("QALLOC q0\nQRX q0, NaN\nHALT\n").expect_err("NaN angle must be rejected");
        assert!(diagnostics.iter().any(|item| item.code == "E124"));
    }

    #[test]
    fn assembles_lifecycle_and_multi_qubit_operations() {
        let source = "QALLOC q0\nQALLOC q1\nQALLOC q2\nQCZ q0, q1\nQSWAP q1, q2\nQCCX q0, q1, q2\nQRESET q2\nQFREE q2\nHALT\n";
        let executable = assemble(source).expect("lifecycle program assembles");
        assert!(matches!(
            executable.instructions[5].instruction,
            Instruction::QCcx { .. }
        ));
        assert!(matches!(
            executable.instructions[7].instruction,
            Instruction::QFree(_)
        ));
    }

    #[test]
    fn free_closes_allocation_and_prevents_use_after_free() {
        let use_after_free =
            assemble("QALLOC q0\nQFREE q0\nQH q0\nHALT\n").expect_err("freed qubit use must fail");
        assert!(use_after_free.iter().any(|item| item.code == "E117"));
        let reallocate = assemble("QALLOC q0\nQFREE q0\nQALLOC q1\nHALT\n")
            .expect_err("allocation after free must fail");
        assert!(reallocate.iter().any(|item| item.code == "E125"));
        assemble("QALLOC q0\nQALLOC q1\nQFREE q1\nQFREE q0\nHALT\n")
            .expect("sequential highest-first release is valid");
    }

    #[test]
```

```
    fn assembles_complete_classical_subset() {
        let source = "LOADI r0, 9\nMOV r1, r0\nADD r2, r0, 3\nSUB r2, r2, r1\nMUL r3, r0, r1\nDIV r4, r3, r0\nMOD r5, r3, 8\nAND r6, r0,
          r1\nOR r7, r0, 2\nXOR r8, r0, r1\nNOT r9, r0\nSHL r10, r0, 2\nSHR r11, r10, r1\nCMP r0, r1\nJNZ ne\nJLT lt\nJGT gt\nne:
          HALT\nlt: HALT\ngt: HALT\n";
        let executable = assemble(source).expect("classical subset assembles");
        assert_eq!(executable.instructions[2].instruction.mnemonic(), "ADD");
        assert_eq!(executable.instructions[14].instruction.mnemonic(), "JNZ");
    }

    #[test]
    fn disassembly_contains_numbered_typed_operands() {
        let executable = assemble("LOADI r0, 2\nADD r1, r0, 3\nHALT\n").expect("assemble");
        let listing = disassemble(&executable);
        assert!(listing.contains("0000 LOADI      r0, 2"));
        assert!(listing.contains("0001 ADD        r1, r0, 3"));
        assert!(listing.contains("0002 HALT"));
    }
}
```

Appendix: crates/clio-resource/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Conservative resource estimation and execution admission.

#![forbid(unsafe_code)]

use clio_isa::Executable;
use serde::{Deserialize, Serialize};
use thiserror::Error;

const COMPLEX_BYTES: u64 = 16;
const FIXED_RUNTIME_OVERHEAD: u64 = 64 * 1024;
// Covers the largest currently serialized bounded state-small event observed by the
// frozen evidence protocol, with headroom for JSON field and numeric variation.
const TRACE_EVENT_ESTIMATE: u64 = 640;

/// Runtime safety limits supplied by the host application.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct ResourceLimits {
    /// Maximum active virtual qubits.
    pub max_qubits: u16,
    /// Maximum estimated total memory.
    pub max_total_memory_bytes: u64,
    /// Maximum shots.
    pub max_shots: u64,
    /// Maximum total executed instructions across shots.
    pub max_instructions: u64,
    /// Maximum estimated trace bytes.
    pub max_trace_bytes: u64,
    /// Maximum wall-clock execution time in milliseconds.
    pub max_execution_millis: u64,
}

impl Default for ResourceLimits {
    fn default() -> Self {
        Self {
            max_qubits: 25,
            max_total_memory_bytes: 512 * 1024 * 1024,
            max_shots: 1_000_000,
            max_instructions: 10_000_000,
            max_trace_bytes: 256 * 1024 * 1024,
            max_execution_millis: 30_000,
        }
    }
}

/// A checked pre-allocation execution plan.
#[derive(Clone, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct ExecutionPlan {
    /// Required active qubits.
    pub qubits: u16,
    /// Number of complex amplitudes.
    pub amplitude_count: u64,
    /// Exact raw double-complex state bytes.
    pub raw_state_bytes: u64,
    /// Conservative state plus runtime overhead.
    pub estimated_runtime_bytes: u64,
    /// Conservative trace bytes.
    pub estimated_trace_bytes: u64,
    /// Conservative total bytes.
    pub estimated_total_bytes: u64,
    /// Maximum instruction executions for the straight-line program.
    pub estimated_instructions: u64,
    /// Shots.
    pub shots: u64,
}

/// Deterministic resource admission failure.
#[derive(Clone, Debug, Eq, Error, PartialEq, Serialize, Deserialize)]
pub enum ResourceError {
    /// Checked arithmetic could not represent a requested scale.
    #[error("resource calculation overflowed")]
    ArithmeticOverflow,
    /// The qubit count exceeds the configured limit.
    #[error("required qubits {required} exceed limit {limit}")]
    QubitLimit {
        /// Required qubits.
        required: u16,
        /// Configured maximum.
        limit: u16,
    },
    /// The shot count exceeds the configured limit.
    #[error("requested shots {requested} exceed limit {limit}")]
    ShotLimit {
        /// Requested shots.
        requested: u64,
        /// Configured maximum.
        limit: u64,
    },
    /// The instruction execution bound exceeds the configured limit.
    #[error("estimated instructions {estimated} exceed limit {limit}")]
    InstructionLimit {
        /// Estimated executions.
        estimated: u64,
        /// Configured maximum.
        limit: u64,
    },
    /// The trace estimate exceeds the configured limit.
    #[error("estimated trace bytes {estimated} exceed limit {limit}")]
    TraceLimit {
        /// Estimated trace bytes.
```

```
        estimated: u64,
        /// Configured maximum.
        limit: u64,
    },
    /// The total memory estimate exceeds the effective budget.
    #[error("estimated memory {estimated} bytes exceeds budget {budget} bytes")]
    MemoryLimit {
        /// Estimated total bytes.
        estimated: u64,
        /// Effective budget.
        budget: u64,
    },
}

/// Builds and admits a plan before any state-vector allocation.
pub fn plan(
    executable: &Executable,
    limits: ResourceLimits,
) -> Result<ExecutionPlan, ResourceError> {
    if executable.required_qubits > limits.max_qubits {
        return Err(ResourceError::QubitLimit {
            required: executable.required_qubits,
            limit: limits.max_qubits,
        });
    }
    if executable.metadata.shots > limits.max_shots {
        return Err(ResourceError::ShotLimit {
            requested: executable.metadata.shots,
            limit: limits.max_shots,
        });
    }

    let amplitude_count = 1_u64
        .checked_shl(u32::from(executable.required_qubits))
        .ok_or(ResourceError::ArithmeticOverflow)?;
    let raw_state_bytes = amplitude_count
        .checked_mul(COMPLEX_BYTES)
        .ok_or(ResourceError::ArithmeticOverflow)?;
    let engine_overhead = raw_state_bytes / 8;
    let estimated_runtime_bytes = raw_state_bytes
        .checked_add(engine_overhead)
        .and_then(|bytes| bytes.checked_add(FIXED_RUNTIME_OVERHEAD))
        .ok_or(ResourceError::ArithmeticOverflow)?;
    let instruction_count = u64::try_from(executable.instructions.len())
        .map_err(|_| ResourceError::ArithmeticOverflow)?;
    let estimated_instructions = if executable.has_backward_branches {
        limits.max_instructions
    } else {
        instruction_count
            .checked_mul(executable.metadata.shots)
            .ok_or(ResourceError::ArithmeticOverflow)?
    };
    if estimated_instructions > limits.max_instructions {
        return Err(ResourceError::InstructionLimit {
            estimated: estimated_instructions,
            limit: limits.max_instructions,
        });
    }
    let estimated_trace_bytes =
        if matches!(executable.metadata.trace_level, clio_core::TraceLevel::Off) {
            0
        } else {
            estimated_instructions
                .checked_mul(TRACE_EVENT_ESTIMATE)
                .ok_or(ResourceError::ArithmeticOverflow)?
        };
    if estimated_trace_bytes > limits.max_trace_bytes {
        return Err(ResourceError::TraceLimit {
            estimated: estimated_trace_bytes,
            limit: limits.max_trace_bytes,
        });
    }
    let estimated_total_bytes = estimated_runtime_bytes
        .checked_add(estimated_trace_bytes)
        .ok_or(ResourceError::ArithmeticOverflow)?;
    let effective_budget = executable
        .metadata
        .memory_budget_bytes
        .map_or(limits.max_total_memory_bytes, |declared| {
            declared.min(limits.max_total_memory_bytes)
        });
    if estimated_total_bytes > effective_budget {
        return Err(ResourceError::MemoryLimit {
            estimated: estimated_total_bytes,
            budget: effective_budget,
        });
    }
    Ok(ExecutionPlan {
        qubits: executable.required_qubits,
        amplitude_count,
        raw_state_bytes,
        estimated_runtime_bytes,
        estimated_trace_bytes,
        estimated_total_bytes,
        estimated_instructions,
        shots: executable.metadata.shots,
    })
}

#[cfg(test)]
mod tests {
    use super::*;
    use clio_assembler::assemble;

    #[test]
    fn exact_raw_state_formula_is_used() {
        let executable =
```

```
            assemble(".shots 1\n.trace off\nQALLOC q0\nQALLOC q1\nHALT\n").expect("valid program");
        let plan = plan(&executable, ResourceLimits::default()).expect("admitted");
        assert_eq!(plan.amplitude_count, 4);
        assert_eq!(plan.raw_state_bytes, 64);
    }

    #[test]
    fn rejects_before_exceeding_declared_memory() {
        let executable = assemble(".shots 1\n.trace off\n.budget memory=65536B\nQALLOC q0\nHALT\n")
            .expect("valid program");
        assert!(matches!(
            plan(&executable, ResourceLimits::default()),
            Err(ResourceError::MemoryLimit { .. })
        ));
    }
}
```

Appendix: crates/clio-trace/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Bounded structured execution trace model for the first executable path.

#![forbid(unsafe_code)]

use clio_core::{ClassicalRegister, MeasurementRegister, QubitId, SourceSpan, TraceLevel};
use serde::{Deserialize, Serialize};

/// Trace-visible processor lifecycle status.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub enum TraceStatus {
    /// Ready to run.
    Ready,
    /// Executing an instruction.
    Running,
    /// Completed by fall-through.
    Completed,
    /// Stopped by `HALT`.
    Halted,
    /// Failed with a typed runtime trap.
    Trapped,
}

/// A changed measurement register.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct MeasurementChange {
    /// Destination register.
    pub register: MeasurementRegister,
    /// Measured bit.
    pub value: bool,
}

/// A changed classical register.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct ClassicalChange {
    /// Destination register.
    pub register: ClassicalRegister,
    /// New signed value.
    pub value: i64,
}

/// One amplitude in a bounded state snapshot.
#[derive(Clone, Copy, Debug, PartialEq, Serialize, Deserialize)]
pub struct StateAmplitude {
    /// Basis-array index under ADR-0001.
    pub basis_index: usize,
    /// Real component.
    pub real: f64,
    /// Imaginary component.
    pub imaginary: f64,
    /// Squared magnitude.
    pub probability: f64,
}

/// Trace-visible signed comparison state.
#[derive(Clone, Copy, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub enum TraceComparison {
    /// Left was less than right.
    Less,
    /// Operands were equal.
    Equal,
    /// Left was greater than right.
    Greater,
}

/// One real processor instruction transition.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct TraceEvent {
    /// Shot index, starting at zero.
    pub shot: u64,
    /// Monotonic logical step across this run.
    pub step: u64,
    /// Program counter before execution.
    pub program_counter: usize,
    /// Executed mnemonic.
    pub mnemonic: String,
    /// Optional source location.
    pub source_span: Option<SourceSpan>,
    /// Status before execution.
    pub status_before: TraceStatus,
    /// Status after execution.
    pub status_after: TraceStatus,
    /// Allocated logical qubit, when applicable.
    pub allocated_qubit: Option<QubitId>,
    /// Released logical qubit, when applicable.
    pub freed_qubit: Option<QubitId>,
    /// Sampled value encountered while resetting, when applicable.
    pub reset_outcome: Option<bool>,
    /// Measurement mutation, when applicable.
    pub measurement: Option<MeasurementChange>,
    /// Classical register mutation, when applicable.
    pub classical: Option<ClassicalChange>,
    /// Branch decision, when applicable.
    pub branch_taken: Option<bool>,
    /// Comparison/flag state after the instruction.
    pub comparison: Option<TraceComparison>,
    /// Complete state for at most eight qubits at state-enabled trace levels.
    pub state_snapshot: Option<Vec<StateAmplitude>>,
}
```

```
/// Version-identified trace produced by the runtime.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct ExecutionTrace {
    /// Internal trace schema identifier.
    pub format: String,
    /// Requested detail level.
    pub level: TraceLevel,
    /// Events retained within the admitted bound.
    pub events: Vec<TraceEvent>,
}

impl ExecutionTrace {
    /// Creates an empty trace.
    #[must_use]
    pub fn new(level: TraceLevel) -> Self {
        Self {
            format: "clio-trace-bell-path-1".to_owned(),
            level,
            events: Vec::new(),
        }
    }

    /// Records an event unless tracing is off.
    pub fn push(&mut self, event: TraceEvent) {
        if self.level != TraceLevel::Off {
            self.events.push(event);
        }
    }
}
```

Appendix: crates/clio-validation/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Correctness validation for supported Clio execution results.

#![forbid(unsafe_code)]

use clio_runtime::ExecutionResult;
use serde::{Deserialize, Serialize};
use std::collections::HashMap;

/// Bell-state known-answer validation report.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct BellValidationReport {
    /// Whether all required checks passed.
    pub passed: bool,
    /// Total observed shots.
    pub observed_shots: u64,
    /// Count of forbidden outcomes other than `00` and `11`.
    pub forbidden_outcomes: u64,
    /// Empirical probability of `00`.
    pub probability_00: f64,
    /// Total variation distance from the ideal Bell distribution.
    pub total_variation_distance: f64,
}

/// Validates counts against the ideal Bell distribution.
#[must_use]
#[allow(clippy::cast_precision_loss)] // Counts are bounded to 1,000,000 by runtime admission.
pub fn validate_bell_result(result: &ExecutionResult) -> BellValidationReport {
    let observed_shots: u64 = result.measurement_counts.values().sum();
    let count_00 = result.measurement_counts.get("00").copied().unwrap_or(0);
    let count_11 = result.measurement_counts.get("11").copied().unwrap_or(0);
    let forbidden_outcomes = observed_shots.saturating_sub(count_00.saturating_add(count_11));
    let denominator = observed_shots.max(1) as f64;
    let probability_00 = count_00 as f64 / denominator;
    let probability_11 = count_11 as f64 / denominator;
    let forbidden_probability = forbidden_outcomes as f64 / denominator;
    let total_variation_distance =
        0.5 * ((probability_00 - 0.5).abs() + (probability_11 - 0.5).abs() + forbidden_probability);
    BellValidationReport {
        passed: observed_shots == result.shots && observed_shots > 0 && forbidden_outcomes == 0,
        observed_shots,
        forbidden_outcomes,
        probability_00,
        total_variation_distance,
    }
}

/// GHZ known-answer distribution report for arbitrary measured width.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct GhzValidationReport {
    /// Whether only all-zero and all-one outcomes were observed for every shot.
    pub passed: bool,
    /// Width of the observed bit strings.
    pub measured_qubits: usize,
    /// Total observed shots.
    pub observed_shots: u64,
    /// Count outside the two ideal GHZ outcomes.
    pub forbidden_outcomes: u64,
    /// Total variation distance from the ideal equal two-outcome distribution.
    pub total_variation_distance: f64,
}

/// Validates a fully measured GHZ result.
#[must_use]
#[allow(clippy::cast_precision_loss)] // Runtime admission bounds shot counts to exact f64 integers.
pub fn validate_ghz_result(result: &ExecutionResult) -> GhzValidationReport {
    let measured_qubits = result
        .measurement_counts
        .keys()
        .map(String::len)
        .max()
        .unwrap_or(0);
    let zeros = "0".repeat(measured_qubits);
    let ones = "1".repeat(measured_qubits);
    let observed_shots: u64 = result.measurement_counts.values().sum();
    let zero_count = result.measurement_counts.get(&zeros).copied().unwrap_or(0);
    let one_count = result.measurement_counts.get(&ones).copied().unwrap_or(0);
    let forbidden_outcomes = observed_shots.saturating_sub(zero_count.saturating_add(one_count));
    let denominator = observed_shots.max(1) as f64;
    let total_variation_distance = 0.5
        * ((zero_count as f64 / denominator - 0.5).abs()
            + (one_count as f64 / denominator - 0.5).abs()
            + forbidden_outcomes as f64 / denominator);
    GhzValidationReport {
        passed: measured_qubits >= 3 && observed_shots == result.shots && forbidden_outcomes == 0,
        measured_qubits,
        observed_shots,
        forbidden_outcomes,
        total_variation_distance,
    }
}

/// Measurement-driven branching conformance report.
#[derive(Clone, Debug, PartialEq, Eq, Serialize, Deserialize)]
pub struct HybridBranchValidationReport {
    /// Whether every shot preserved measurement-to-classical agreement.
    pub passed: bool,
    /// Shots with a recorded `m0` measurement.
    pub measured_shots: u64,
    /// Shots whose `r1` write disagreed with `m0` or was missing.
```

```
    pub mismatched_shots: u64,
    /// Conditional branches observed as taken.
    pub taken_branches: u64,
    /// Conditional branches observed as not taken.
    pub not_taken_branches: u64,
}

/// Validates the trace contract of the canonical measurement-branching workload.
#[must_use]
pub fn validate_hybrid_branch_result(result: &ExecutionResult) -> HybridBranchValidationReport {
    let mut measurements = HashMap::new();
    let mut final_r1 = HashMap::new();
    let mut taken_branches = 0;
    let mut not_taken_branches = 0;

    for event in &result.trace.events {
        if let Some(measurement) = event.measurement
            && measurement.register.index() == 0
        {
            measurements.insert(event.shot, measurement.value);
        }
        if let Some(classical) = event.classical
            && classical.register.index() == 1
        {
            final_r1.insert(event.shot, classical.value);
        }
        match event.branch_taken {
            Some(true) if event.mnemonic == "JZ" => taken_branches += 1,
            Some(false) if event.mnemonic == "JZ" => not_taken_branches += 1,
            _ => {}
        }
    }

    let mismatched_shots = (0..result.shots)
        .filter(|shot| {
            let expected = measurements.get(shot).map(|value| i64::from(*value));
            expected.is_none() || final_r1.get(shot).copied() != expected
        })
        .count() as u64;
    HybridBranchValidationReport {
        passed: measurements.len() as u64 == result.shots
            && mismatched_shots == 0
            && taken_branches > 0
            && not_taken_branches > 0,
        measured_shots: measurements.len() as u64,
        mismatched_shots,
        taken_branches,
        not_taken_branches,
    }
}

/// Known-answer report for a teleportation workload with a verification measurement.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct TeleportationValidationReport {
    /// Whether every receiver verification bit matched the expected value.
    pub passed: bool,
    /// Expected receiver verification bit.
    pub expected_receiver_bit: bool,
    /// Total observed shots.
    pub observed_shots: u64,
    /// Shots with an incorrect receiver verification bit.
    pub failed_receiver_shots: u64,
    /// Fraction of correct receiver verification measurements.
    pub verification_success_rate: f64,
    /// Number of distinct sender measurement/correction combinations observed.
    pub correction_combinations_observed: usize,
}

/// Validates deterministic receiver verification for a canonical teleportation workload.
#[must_use]
#[allow(clippy::cast_precision_loss)] // Runtime admission bounds shot counts to exact f64 integers.
pub fn validate_teleportation_result(
    result: &ExecutionResult,
    expected_receiver_bit: bool,
) -> TeleportationValidationReport {
    let expected = if expected_receiver_bit { '1' } else { '0' };
    let observed_shots: u64 = result.measurement_counts.values().sum();
    let failed_receiver_shots = result
        .measurement_counts
        .iter()
        .filter(|(outcome, _)| !outcome.starts_with(expected))
        .map(|(_, count)| count)
        .sum();
    let correction_combinations_observed = result
        .measurement_counts
        .keys()
        .filter_map(|outcome| outcome.get(1..))
        .collect::<std::collections::HashSet<_>>()
        .len();
    let successful = observed_shots.saturating_sub(failed_receiver_shots);
    TeleportationValidationReport {
        passed: observed_shots == result.shots
            && failed_receiver_shots == 0
            && correction_combinations_observed == 4,
        expected_receiver_bit,
        observed_shots,
        failed_receiver_shots,
        verification_success_rate: successful as f64 / observed_shots.max(1) as f64,
        correction_combinations_observed,
    }
}

/// Exact Bernstein-Vazirani hidden-string recovery report.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct BernsteinVaziraniValidationReport {
    /// Whether every shot recovered the hidden string.
    pub passed: bool,
```

```
    /// Declared hidden string.
    pub expected_secret: String,
    /// Total shots matching the secret.
    pub matching_shots: u64,
    /// Total observed shots.
    pub observed_shots: u64,
}

/// Validates deterministic hidden-string recovery.
#[must_use]
pub fn validate_bernstein_vazirani_result(
    result: &ExecutionResult,
    expected_secret: &str,
) -> BernsteinVaziraniValidationReport {
    let observed_shots: u64 = result.measurement_counts.values().sum();
    let matching_shots = result
        .measurement_counts
        .get(expected_secret)
        .copied()
        .unwrap_or(0);
    BernsteinVaziraniValidationReport {
        passed: observed_shots == result.shots && matching_shots == result.shots,
        expected_secret: expected_secret.to_owned(),
        matching_shots,
        observed_shots,
    }
}

/// Deutsch-Jozsa constant/balanced classification report.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct DeutschJozsaValidationReport {
    /// Whether every shot produced the expected classification.
    pub passed: bool,
    /// Expected oracle class.
    pub expected_constant: bool,
    /// Shots classified as constant by the all-zero result.
    pub constant_shots: u64,
    /// Shots classified as balanced by a nonzero result.
    pub balanced_shots: u64,
    /// Total observed shots.
    pub observed_shots: u64,
}

/// Validates Deutsch-Jozsa classification.
#[must_use]
pub fn validate_deutsch_jozsa_result(
    result: &ExecutionResult,
    expected_constant: bool,
) -> DeutschJozsaValidationReport {
    let observed_shots: u64 = result.measurement_counts.values().sum();
    let constant_shots = result
        .measurement_counts
        .iter()
        .filter(|(outcome, _)| outcome.chars().all(|bit| bit == '0'))
        .map(|(_, count)| count)
        .sum();
    let balanced_shots = observed_shots.saturating_sub(constant_shots);
    let passed = observed_shots == result.shots
        && if expected_constant {
            constant_shots == result.shots
        } else {
            balanced_shots == result.shots
        };
    DeutschJozsaValidationReport {
        passed,
        expected_constant,
        constant_shots,
        balanced_shots,
        observed_shots,
    }
}

/// Known-answer report for small Grover search.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct GroverValidationReport {
    /// Whether the marked state met the required success threshold.
    pub passed: bool,
    /// Marked output string.
    pub marked_state: String,
    /// Marked observations.
    pub marked_shots: u64,
    /// Total observations.
    pub observed_shots: u64,
    /// Empirical marked-state probability.
    pub success_probability: f64,
}

/// Validates a small Grover workload against its marked state and threshold.
#[must_use]
#[allow(clippy::cast_precision_loss)]
pub fn validate_grover_result(
    result: &ExecutionResult,
    marked_state: &str,
    minimum_success: f64,
) -> GroverValidationReport {
    let observed_shots: u64 = result.measurement_counts.values().sum();
    let marked_shots = result
        .measurement_counts
        .get(marked_state)
        .copied()
        .unwrap_or(0);
    let success_probability = marked_shots as f64 / observed_shots.max(1) as f64;
    GroverValidationReport {
        passed: observed_shots == result.shots && success_probability >= minimum_success,
        marked_state: marked_state.to_owned(),
        marked_shots,
        observed_shots,
```

```
        success_probability,
    }
}

/// Exact QFT/inverse-QFT round-trip report.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct QftRoundtripValidationReport {
    /// Whether every shot recovered the input basis state.
    pub passed: bool,
    /// Expected recovered state.
    pub expected_state: String,
    /// Exactly matching shots.
    pub matching_shots: u64,
    /// Total observations.
    pub observed_shots: u64,
}

/// Validates exact recovery after a QFT/inverse-QFT pair.
#[must_use]
pub fn validate_qft_roundtrip_result(
    result: &ExecutionResult,
    expected_state: &str,
) -> QftRoundtripValidationReport {
    let observed_shots = result.measurement_counts.values().sum();
    let matching_shots = result
        .measurement_counts
        .get(expected_state)
        .copied()
        .unwrap_or(0);
    QftRoundtripValidationReport {
        passed: observed_shots == result.shots && matching_shots == result.shots,
        expected_state: expected_state.to_owned(),
        matching_shots,
        observed_shots,
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use clio_assembler::assemble;
    use clio_resource::ResourceLimits;
    use clio_runtime::run;
    use std::fmt::Write as _;

    #[test]
    fn real_bell_execution_passes_known_answer_validation() {
        let source = ".seed 42\n.shots 1024\n.trace off\nQALLOC q0\nQALLOC q1\nQH q0\nQCX q0, q1\nQMEASURE q0, m0\nQMEASURE q1,
          m1\nHALT\n";
        let executable = assemble(source).expect("assemble");
        let result = run(&executable, ResourceLimits::default()).expect("run");
        let report = validate_bell_result(&result);
        assert!(report.passed);
        assert_eq!(report.forbidden_outcomes, 0);
        assert!(report.total_variation_distance < 0.1);
    }

    #[test]
    fn real_hybrid_execution_preserves_measurement_control() {
        let source = ".seed 9\n.shots 128\n.trace instructions\nQALLOC q0\nQH q0\nQMEASURE q0, m0\nMOV r0, m0\nCMP r0, 0\nJZ
          measured_zero\nLOADI r1, 1\nJMP done\nmeasured_zero: LOADI r1, 0\ndone: HALT\n";
        let executable = assemble(source).expect("assemble");
        let result = run(&executable, ResourceLimits::default()).expect("run");
        let report = validate_hybrid_branch_result(&result);
        assert!(report.passed);
        assert_eq!(report.measured_shots, 128);
        assert_eq!(report.mismatched_shots, 0);
    }

    #[test]
    fn ghz_widths_have_only_all_zero_or_all_one_outcomes() {
        for width in 3_u16..=5 {
            let mut source = ".seed 42\n.shots 512\n.trace off\n".to_owned();
            for qubit in 0..width {
                writeln!(source, "QALLOC q{qubit}").expect("write source");
            }
            source.push_str("QH q0\n");
            for target in 1..width {
                writeln!(source, "QCX q0, q{target}").expect("write source");
            }
            for qubit in 0..width {
                writeln!(source, "QMEASURE q{qubit}, m{qubit}").expect("write source");
            }
            source.push_str("HALT\n");
            let executable = assemble(&source).expect("assemble GHZ");
            let result = run(&executable, ResourceLimits::default()).expect("run GHZ");
            let report = validate_ghz_result(&result);
            assert!(report.passed, "GHZ-{width} must pass");
            assert_eq!(report.measured_qubits, usize::from(width));
            assert_eq!(report.forbidden_outcomes, 0);
        }
    }

    #[test]
    fn teleportation_known_states_pass_receiver_verification() {
        let cases = [
            (
                include_str!("../../../examples/teleportation/zero/main.clio"),
                false,
            ),
            (
                include_str!("../../../examples/teleportation/one/main.clio"),
                true,
            ),
            (
                include_str!("../../../examples/teleportation/plus/main.clio"),
                false,
```

```
            ),
        ];
        for (source, expected) in cases {
            let executable = assemble(source).expect("assemble teleportation");
            let result = run(&executable, ResourceLimits::default()).expect("run teleportation");
            let report = validate_teleportation_result(&result, expected);
            assert!(report.passed);
            assert_eq!(report.failed_receiver_shots, 0);
            assert_eq!(report.correction_combinations_observed, 4);
        }
    }

    #[test]
    fn bernstein_vazirani_recovers_checked_in_secrets() {
        let cases = [
            (
                include_str!("../../../examples/bernstein-vazirani/secret-1011/main.clio"),
                "1011",
            ),
            (
                include_str!("../../../examples/bernstein-vazirani/secret-00101/main.clio"),
                "00101",
            ),
        ];
        for (source, secret) in cases {
            let result = run(
                &assemble(source).expect("assemble BV"),
                ResourceLimits::default(),
            )
            .expect("run BV");
            assert!(validate_bernstein_vazirani_result(&result, secret).passed);
        }
    }

    #[test]
    fn deutsch_jozsa_classifies_constant_and_balanced_oracles() {
        let cases = [
            (
                include_str!("../../../examples/deutsch-jozsa/constant-zero/main.clio"),
                true,
            ),
            (
                include_str!("../../../examples/deutsch-jozsa/constant-one/main.clio"),
                true,
            ),
            (
                include_str!("../../../examples/deutsch-jozsa/balanced-parity/main.clio"),
                false,
            ),
        ];
        for (source, expected_constant) in cases {
            let result = run(
                &assemble(source).expect("assemble DJ"),
                ResourceLimits::default(),
            )
            .expect("run DJ");
            assert!(validate_deutsch_jozsa_result(&result, expected_constant).passed);
        }
    }

    #[test]
    fn grover_examples_amplify_their_marked_states() {
        let cases = [
            (
                include_str!("../../../examples/grover/2-qubit/main.clio"),
                "11",
                1.0,
            ),
            (
                include_str!("../../../examples/grover/3-qubit/main.clio"),
                "111",
                0.70,
            ),
        ];
        for (source, marked, threshold) in cases {
            let result = run(
                &assemble(source).expect("assemble Grover"),
                ResourceLimits::default(),
            )
            .expect("run Grover");
            assert!(validate_grover_result(&result, marked, threshold).passed);
        }
    }

    #[test]
    fn qft_example_recovers_input_exactly() {
        let source = include_str!("../../../examples/qft/3-qubit-roundtrip/main.clio");
        let result = run(
            &assemble(source).expect("assemble QFT"),
            ResourceLimits::default(),
        )
        .expect("run QFT");
        assert!(validate_qft_roundtrip_result(&result, "101").passed);
    }
}
```

Appendix: crates/clio-sdk/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Typed Rust developer API for the executable Bell-state workflow.

#![forbid(unsafe_code)]

use clio_assembler::{assemble, disassemble as disassemble_executable};
use clio_core::{Diagnostic, TraceLevel};
use clio_isa::{Executable, Instruction};
use clio_resource::{ExecutionPlan, ResourceError, ResourceLimits, plan};
use clio_runtime::{ExecutionResult, RuntimeError, run};
use clio_trace::StateAmplitude;
use clio_validation::{
    BellValidationReport, BernsteinVaziraniValidationReport, DeutschJozsaValidationReport,
    GhzValidationReport, GroverValidationReport, HybridBranchValidationReport,
    QftRoundtripValidationReport, TeleportationValidationReport, validate_bell_result,
    validate_bernstein_vazirani_result, validate_deutsch_jozsa_result, validate_ghz_result,
    validate_grover_result, validate_hybrid_branch_result, validate_qft_roundtrip_result,
    validate_teleportation_result,
};
use serde::{Deserialize, Serialize};
use thiserror::Error;

/// Unified SDK error preserving validation and runtime categories.
#[derive(Debug, Error)]
pub enum SdkError {
    /// Source parsing or semantic validation failed.
    #[error("program validation failed with {} diagnostic(s)", .0.len())]
    Validation(Vec<Diagnostic>),
    /// Resource planning failed.
    #[error(transparent)]
    Resource(#[from] ResourceError),
    /// Runtime execution failed.
    #[error(transparent)]
    Runtime(#[from] RuntimeError),
}

impl SdkError {
    /// Returns a stable machine-readable category or trap code.
    #[must_use]
    pub fn code(&self) -> &str {
        match self {
            Self::Validation(_) => "V100",
            Self::Resource(_) => "R100",
            Self::Runtime(error) => error.code(),
        }
    }
}

/// Parses, validates, and assembles source.
pub fn build(source: &str) -> Result<Executable, SdkError> {
    assemble(source).map_err(SdkError::Validation)
}

/// Produces a checked execution plan without allocating a state vector.
pub fn estimate(source: &str, limits: ResourceLimits) -> Result<ExecutionPlan, SdkError> {
    let executable = build(source)?;
    Ok(plan(&executable, limits)?)
}

/// Builds source and returns a numbered typed instruction listing.
pub fn disassemble(source: &str) -> Result<String, SdkError> {
    Ok(disassemble_executable(&build(source)?))
}

/// Executes source through parser, assembler, admission, runtime, and Clio Engine.
pub fn execute(source: &str, limits: ResourceLimits) -> Result<ExecutionResult, SdkError> {
    let executable = build(source)?;
    Ok(run(&executable, limits)?)
}

/// Runs one shot with bounded small-state snapshots enabled.
pub fn inspect(source: &str, limits: ResourceLimits) -> Result<ExecutionResult, SdkError> {
    let mut executable = build(source)?;
    executable.metadata.shots = 1;
    executable.metadata.trace_level = TraceLevel::StateSmall;
    Ok(run(&executable, limits)?)
}

/// Non-mutating final-state observations for a bounded (at most eight-qubit) program.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct ObservationReport {
    /// Squared state norm.
    pub norm_squared: f64,
    /// Exact basis probabilities in little-endian basis-index order.
    pub basis_probabilities: Vec<f64>,
    /// `[P(0), P(1)]` for each logical qubit.
    pub marginals: Vec<[f64; 2]>,
    /// Single-qubit `[<X>, <Y>, <Z>]` expectations.
    pub pauli_expectations: Vec<[f64; 3]>,
    /// Reduced single-qubit density matrices, encoded as `[real, imaginary]` pairs.
    pub reduced_qubit_states: Vec<[[f64; 2]; 2]>,
}

/// Executes one unmeasured shot and computes final-state debugger observations.
pub fn observe(source: &str, limits: ResourceLimits) -> Result<ObservationReport, SdkError> {
    let result = inspect(source, limits)?;
    let snapshot = result
        .trace
        .events
        .iter()
```

```
        .rev()
        .find_map(|event| event.state_snapshot.as_ref())
        .cloned()
        .unwrap_or_default();
    Ok(observe_snapshot(&snapshot))
}

fn observe_snapshot(snapshot: &[StateAmplitude]) -> ObservationReport {
    let amplitudes = snapshot
        .iter()
        .map(|value| (value.real, value.imaginary))
        .collect::<Vec<_>>();
    let qubits = amplitudes.len().ilog2() as usize;
    let basis_probabilities = snapshot
        .iter()
        .map(|value| value.probability)
        .collect::<Vec<_>>();
    let norm_squared = basis_probabilities.iter().sum();
    let mut marginals = Vec::with_capacity(qubits);
    let mut pauli_expectations = Vec::with_capacity(qubits);
    let mut reduced_qubit_states = Vec::with_capacity(qubits);
    for qubit in 0..qubits {
        let bit = 1_usize << qubit;
        let p1 = basis_probabilities
            .iter()
            .enumerate()
            .filter(|(index, _)| index & bit != 0)
            .map(|(_, value)| value)
            .sum::<f64>();
        let mut coherence = (0.0, 0.0);
        for index in 0..amplitudes.len() {
            if index & bit == 0 {
                let a = amplitudes[index];
                let b = amplitudes[index | bit];
                coherence.0 += a.0 * b.0 + a.1 * b.1;
                coherence.1 += a.1 * b.0 - a.0 * b.1;
            }
        }
        marginals.push([norm_squared - p1, p1]);
        pauli_expectations.push([
            2.0 * coherence.0,
            -2.0 * coherence.1,
            norm_squared - 2.0 * p1,
        ]);
        reduced_qubit_states.push([
            [[norm_squared - p1, 0.0], [coherence.0, coherence.1]],
            [[coherence.0, -coherence.1], [p1, 0.0]],
        ]);
    }
    ObservationReport {
        norm_squared,
        basis_probabilities,
        marginals,
        pauli_expectations,
        reduced_qubit_states,
    }
}

/// Validation report for a recognized canonical workload family.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
#[serde(tag = "kind", content = "report", rename_all = "kebab-case")]
pub enum SupportedValidationReport {
    /// Bell/GHZ-style correlation report.
    Bell(BellValidationReport),
    /// Multi-qubit GHZ correlation report.
    Ghz(GhzValidationReport),
    /// Measurement-driven classical-control report.
    HybridBranch(HybridBranchValidationReport),
    /// Quantum teleportation known-answer report.
    Teleportation(TeleportationValidationReport),
    /// Bernstein-Vazirani hidden-string report.
    BernsteinVazirani(BernsteinVaziraniValidationReport),
    /// Deutsch-Jozsa classification report.
    DeutschJozsa(DeutschJozsaValidationReport),
    /// Small Grover marked-state amplification report.
    Grover(GroverValidationReport),
    /// QFT followed by inverse-QFT recovery report.
    QftRoundtrip(QftRoundtripValidationReport),
}

/// Executes and validates a currently supported canonical program.
pub fn validate(
    source: &str,
    limits: ResourceLimits,
) -> Result<SupportedValidationReport, SdkError> {
    let executable = build(source)?;
    let hybrid = executable
        .instructions
        .iter()
        .any(|located| matches!(located.instruction, Instruction::MovMeasurement { .. }));
    let result = run(&executable, limits)?;
    let teleportation = executable.metadata.name.starts_with("teleport_");
    let bernstein_vazirani = executable.metadata.name.starts_with("bv_");
    let deutsch_jozsa = executable.metadata.name.starts_with("deutsch_jozsa_");
    let grover = executable.metadata.name.starts_with("grover_");
    let qft = executable.metadata.name.starts_with("qft_roundtrip_");
    Ok(if grover {
        let marked = executable.metadata.name.trim_start_matches("grover_");
        let threshold = if marked.len() == 2 { 1.0 } else { 0.70 };
        SupportedValidationReport::Grover(validate_grover_result(&result, marked, threshold))
    } else if qft {
        let expected = executable
            .metadata
            .name
            .trim_start_matches("qft_roundtrip_");
        SupportedValidationReport::QftRoundtrip(validate_qft_roundtrip_result(&result, expected))
    } else if bernstein_vazirani {
```

```
        let secret = executable.metadata.name.trim_start_matches("bv_");
        SupportedValidationReport::BernsteinVazirani(validate_bernstein_vazirani_result(
            &result, secret,
        ))
    } else if deutsch_jozsa {
        let expected_constant = executable.metadata.name.contains("constant");
        SupportedValidationReport::DeutschJozsa(validate_deutsch_jozsa_result(
            &result,
            expected_constant,
        ))
    } else if teleportation {
        let expected_receiver_bit = executable.metadata.name == "teleport_one";
        SupportedValidationReport::Teleportation(validate_teleportation_result(
            &result,
            expected_receiver_bit,
        ))
    } else if hybrid {
        SupportedValidationReport::HybridBranch(validate_hybrid_branch_result(&result))
    } else if executable.required_qubits >= 3 {
        SupportedValidationReport::Ghz(validate_ghz_result(&result))
    } else {
        SupportedValidationReport::Bell(validate_bell_result(&result))
    })
}

/// Re-exported default resource-limit type for SDK consumers.
pub use clio_resource::ResourceLimits as ExecutionLimits;

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn full_source_to_result_workflow_executes() {
        let source = ".seed 7\n.shots 32\n.trace off\nQALLOC q0\nQALLOC q1\nQH q0\nQCX q0, q1\nQMEASURE q0, m0\nQMEASURE q1, m1\nHALT\n";
        let result = execute(source, ResourceLimits::default()).expect("execute Bell");
        assert_eq!(result.measurement_counts.values().sum::<u64>(), 32);
    }

    #[test]
    fn inspection_enables_small_state_snapshots() {
        let result =
            inspect("QALLOC q0\nQH q0\nHALT\n", ResourceLimits::default()).expect("inspect state");
        assert!(
            result
                .trace
                .events
                .iter()
                .any(|event| event.state_snapshot.is_some())
        );
    }

    #[test]
    fn observation_reports_plus_state_without_measurement() {
        let report =
            observe("QALLOC q0\nQH q0\nHALT\n", ResourceLimits::default()).expect("observe plus");
        assert!((report.norm_squared - 1.0).abs() < 1.0e-12);
        assert!((report.marginals[0][1] - 0.5).abs() < 1.0e-12);
        assert!((report.pauli_expectations[0][0] - 1.0).abs() < 1.0e-12);
    }
}
```

Appendix: crates/clio-replay/src/lib.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Portable, checksummed Clio replay packages.

#![forbid(unsafe_code)]

use clio_isa::Executable;
use clio_resource::ResourceLimits;
use clio_runtime::{ENGINE_IDENTITY, ExecutionResult, RNG_IDENTITY};
use clio_sdk::{SdkError, build, execute};
use serde::{Deserialize, Serialize};
use sha2::{Digest, Sha256};
use thiserror::Error;

/// Stable replay envelope schema.
pub const REPLAY_FORMAT: &str = "clio-replay-package-1";

/// Self-contained replay envelope.
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct ReplayPackage {
    /// Package schema.
    pub format: String,
    /// Original Clio source.
    pub source: String,
    /// Typed assembled executable.
    pub executable: Executable,
    /// Original deterministic result.
    pub result: ExecutionResult,
    /// Required engine semantic identity.
    pub engine: String,
    /// Required RNG semantic identity.
    pub rng: String,
    /// Producer OS.
    pub producer_os: String,
    /// Producer architecture.
    pub producer_architecture: String,
    /// SHA-256 over all preceding semantic payload fields.
    pub payload_sha256: String,
}

/// Successful replay verification details.
#[derive(Clone, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct ReplayVerification {
    /// Package integrity and compatibility passed.
    pub passed: bool,
    /// Source hash recovered from reassembly.
    pub source_sha256: String,
    /// Whether reproduced result equals the retained result exactly.
    pub exact_result_match: bool,
}

/// Replay creation or verification failure.
#[derive(Debug, Error)]
pub enum ReplayError {
    /// Build or execution failed.
    #[error(transparent)]
    Sdk(#[from] SdkError),
    /// Package bytes or compatibility identity do not match.
    #[error("replay package is incompatible or corrupted: {0}")]
    Incompatible(String),
    /// JSON encoding failed.
    #[error(transparent)]
    Json(#[from] serde_json::Error),
}

/// Builds, executes, and packages a source program.
pub fn create_package(source: &str, limits: ResourceLimits) -> Result<ReplayPackage, ReplayError> {
    let executable = build(source)?;
    let result = execute(source, limits)?;
    let mut package = ReplayPackage {
        format: REPLAY_FORMAT.to_owned(),
        source: source.to_owned(),
        executable,
        result,
        engine: ENGINE_IDENTITY.to_owned(),
        rng: RNG_IDENTITY.to_owned(),
        producer_os: std::env::consts::OS.to_owned(),
        producer_architecture: std::env::consts::ARCH.to_owned(),
        payload_sha256: String::new(),
    };
    package.payload_sha256 = payload_hash(&package)?;
    Ok(package)
}

/// Checks integrity and semantic compatibility, then re-executes exactly.
pub fn verify_package(
    package: &ReplayPackage,
    limits: ResourceLimits,
) -> Result<ReplayVerification, ReplayError> {
    if package.format != REPLAY_FORMAT {
        return Err(ReplayError::Incompatible(
            "unknown package schema".to_owned(),
        ));
    }
    if package.engine != ENGINE_IDENTITY {
        return Err(ReplayError::Incompatible(
            "engine identity mismatch".to_owned(),
        ));
    }
    if package.rng != RNG_IDENTITY {
        return Err(ReplayError::Incompatible(
```

```
            "RNG identity mismatch".to_owned(),
        ));
    }
    if payload_hash(package)? != package.payload_sha256 {
        return Err(ReplayError::Incompatible(
            "payload checksum mismatch".to_owned(),
        ));
    }
    let rebuilt = build(&package.source)?;
    if rebuilt != package.executable {
        return Err(ReplayError::Incompatible(
            "assembled executable mismatch".to_owned(),
        ));
    }
    let reproduced = execute(&package.source, limits)?;
    let exact_result_match = reproduced == package.result;
    if !exact_result_match {
        return Err(ReplayError::Incompatible(
            "deterministic result mismatch".to_owned(),
        ));
    }
    Ok(ReplayVerification {
        passed: true,
        source_sha256: rebuilt.source_sha256,
        exact_result_match,
    })
}

fn payload_hash(package: &ReplayPackage) -> Result<String, serde_json::Error> {
    let mut copy = package.clone();
    copy.payload_sha256.clear();
    Ok(format!("{:x}", Sha256::digest(serde_json::to_vec(&copy)?)))
}

#[cfg(test)]
mod tests {
    use super::*;

    const BELL: &str = ".seed 42\n.shots 32\n.trace instructions\nQALLOC q0\nQALLOC q1\nQH q0\nQCX q0, q1\nQMEASURE q0, m0\nQMEASURE q1,
      m1\nHALT\n";

    #[test]
    fn package_roundtrip_replays_exactly() {
        let package = create_package(BELL, ResourceLimits::default()).expect("package");
        let encoded = serde_json::to_vec(&package).expect("encode");
        let decoded: ReplayPackage = serde_json::from_slice(&encoded).expect("decode");
        assert!(
            verify_package(&decoded, ResourceLimits::default())
                .expect("verify")
                .passed
        );
    }

    #[test]
    fn mutation_and_identity_drift_are_rejected() {
        let mut package = create_package(BELL, ResourceLimits::default()).expect("package");
        package.source.push_str("# mutation\n");
        assert!(matches!(
            verify_package(&package, ResourceLimits::default()),
            Err(ReplayError::Incompatible(_))
        ));
        let mut package = create_package(BELL, ResourceLimits::default()).expect("package");
        package.engine = "future-engine".to_owned();
        assert!(matches!(
            verify_package(&package, ResourceLimits::default()),
            Err(ReplayError::Incompatible(_))
        ));
    }
}
```

Appendix: crates/clio-studio/src/main.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Local-only Clio Studio server backed by the real SDK and runtime.

#![forbid(unsafe_code)]

use clio_replay::{ReplayPackage, create_package, verify_package};
use clio_resource::ResourceLimits;
use clio_sdk::{build, estimate, execute, inspect, observe, validate};
use serde::Deserialize;
use serde_json::{Value, json};
use std::{
    env,
    io::{Read, Write},
    net::{TcpListener, TcpStream},
    process::ExitCode,
};

const INDEX: &str = include_str!("../web/index.html");
const STYLE: &str = include_str!("../web/style.css");
const APP: &str = include_str!("../web/app.js");

#[derive(Deserialize)]
struct SourceRequest {
    source: String,
}

#[derive(Deserialize)]
struct ReplayRequest {
    package: ReplayPackage,
}

fn main() -> ExitCode {
    let address = env::args()
        .nth(1)
        .unwrap_or_else(|| "127.0.0.1:4317".to_owned());
    let listener = match TcpListener::bind(&address) {
        Ok(value) => value,
        Err(error) => {
            eprintln!("failed to bind {address}: {error}");
            return ExitCode::FAILURE;
        }
    };
    println!("Clio Studio: http://{address}");
    for stream in listener.incoming() {
        match stream {
            Ok(mut stream) => {
                if let Err(error) = serve(&mut stream) {
                    eprintln!("request failed: {error}");
                }
            }
            Err(error) => eprintln!("connection failed: {error}"),
        }
    }
    ExitCode::SUCCESS
}

fn serve(stream: &mut TcpStream) -> Result<(), Box<dyn std::error::Error>> {
    let mut buffer = vec![0_u8; 2 * 1024 * 1024];
    let read = stream.read(&mut buffer)?;
    let request = String::from_utf8_lossy(&buffer[..read]);
    let (head, body) = request.split_once("\r\n\r\n").unwrap_or((&request, ""));
    let first = head.lines().next().unwrap_or_default();
    let mut parts = first.split_whitespace();
    let method = parts.next().unwrap_or_default();
    let path = parts.next().unwrap_or("/");
    let response = route(method, path, body);
    stream.write_all(&response)?;
    Ok(())
}

fn route(method: &str, path: &str, body: &str) -> Vec<u8> {
    match (method, path) {
        ("GET", "/") => response(200, "text/html; charset=utf-8", INDEX.as_bytes()),
        ("GET", "/style.css") => response(200, "text/css; charset=utf-8", STYLE.as_bytes()),
        ("GET", "/app.js") => response(200, "text/javascript; charset=utf-8", APP.as_bytes()),
        ("GET", "/api/examples") => json_response(
            200,
            json!({"bell": include_str!("../../../examples/bell-state/main.clio"), "grover":
              include_str!("../../../examples/grover/2-qubit/main.clio"), "qft":
              include_str!("../../../examples/qft/3-qubit-roundtrip/main.clio"), "teleportation":
              include_str!("../../../examples/teleportation/zero/main.clio")}),
        ),
        ("POST", endpoint) => match serde_json::from_str::<SourceRequest>(body) {
            Ok(request) => source_route(endpoint, &request.source),
            Err(_) if endpoint == "/api/replay/verify" => {
                match serde_json::from_str::<ReplayRequest>(body) {
                    Ok(request) => {
                        result_json(verify_package(&request.package, ResourceLimits::default()))
                    }
                    Err(error) => json_response(400, json!({"ok":false,"error":error.to_string()})),
                }
            }
            Err(error) => json_response(400, json!({"ok":false,"error":error.to_string()})),
        },
        _ => json_response(404, json!({"ok":false,"error":"not found"})),
    }
}

fn source_route(endpoint: &str, source: &str) -> Vec<u8> {
    let limits = ResourceLimits::default();
```

```
    match endpoint {
        "/api/build" => result_json(build(source)),
        "/api/run" => result_json(execute(source, limits)),
        "/api/inspect" => result_json(inspect(source, limits)),
        "/api/observe" => result_json(observe(source, limits)),
        "/api/estimate" => result_json(estimate(source, limits)),
        "/api/validate" => result_json(validate(source, limits)),
        "/api/replay/create" => result_json(create_package(source, limits)),
        _ => json_response(404, json!({"ok":false,"error":"unknown API endpoint"})),
    }
}

fn result_json<T: serde::Serialize, E: std::fmt::Display>(result: Result<T, E>) -> Vec<u8> {
    match result {
        Ok(value) => json_response(200, json!({"ok":true,"data":value})),
        Err(error) => json_response(422, json!({"ok":false,"error":error.to_string()})),
    }
}

#[allow(clippy::needless_pass_by_value)] // Call sites construct one-shot JSON response values.
fn json_response(status: u16, value: Value) -> Vec<u8> {
    response(
        status,
        "application/json; charset=utf-8",
        &serde_json::to_vec(&value).unwrap_or_default(),
    )
}

fn response(status: u16, content_type: &str, body: &[u8]) -> Vec<u8> {
    let reason = if status < 300 {
        "OK"
    } else if status == 404 {
        "Not Found"
    } else {
        "Unprocessable Content"
    };
    let mut output = format!("HTTP/1.1 {status} {reason}\r\nContent-Type: {content_type}\r\nContent-Length: {}\r\nConnection:
      close\r\nX-Content-Type-Options: nosniff\r\nContent-Security-Policy: default-src 'self'; style-src 'self'; script-src
      'self'\r\n\r\n", body.len()).into_bytes();
    output.extend_from_slice(body);
    output
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn real_run_endpoint_executes_bell() {
        let body = serde_json::to_string(&json!({"source":".shots 8\nQALLOC q0\nQALLOC q1\nQH q0\nQCX q0, q1\nQMEASURE q0, m0\nQMEASURE
          q1, m1\nHALT\n"})).expect("json");
        let response = route("POST", "/api/run", &body);
        assert!(String::from_utf8_lossy(&response).contains("measurement_counts"));
    }
}
```

Appendix: crates/clio-cli/src/main.rs

The following checked-in artifact is reproduced for semantic and implementation audit.

```
//! Clio CLI process entry point for the executable Bell-state workflow.

#![forbid(unsafe_code)]

use clio_sdk::{
    ExecutionLimits, SdkError, build, disassemble, estimate, execute, inspect, observe, validate,
};
use std::{env, fs, process::ExitCode};

fn main() -> ExitCode {
    let arguments = env::args().skip(1).collect::<Vec<_>>();
    match dispatch(&arguments) {
        Ok(()) => ExitCode::SUCCESS,
        Err((code, message)) => {
            eprintln!("{message}");
            ExitCode::from(code)
        }
    }
}

fn dispatch(arguments: &[String]) -> Result<(), (u8, String)> {
    let Some(command) = arguments.first().map(String::as_str) else {
        print_help();
        return Ok(());
    };
    if matches!(command, "help" | "--help" | "-h") {
        print_help();
        return Ok(());
    }
    if matches!(command, "version" | "--version" | "-V") {
        println!("clio foundation-build {}", env!("CARGO_PKG_VERSION"));
        return Ok(());
    }

    let path = arguments
        .get(1)
        .ok_or_else(|| (2, format!("clio {command}: missing source path")))?;
    let json = arguments.iter().any(|argument| argument == "--json");
    let source =
        fs::read_to_string(path).map_err(|error| (3, format!("failed to read {path}: {error}")))?;
    let limits = parse_limits(arguments)?;

    match command {
        "check" => match build(&source) {
            Ok(executable) => {
                if json {
                    println!(
                        "{}",
                        serde_json::json!({
                            "valid": true,
                            "program": executable.metadata.name,
                            "instructions": executable.instructions.len(),
                            "source_sha256": executable.source_sha256,
                        })
                    );
                } else {
                    println!(
                        "Valid Clio program: {} ({} instructions)",
                        executable.metadata.name,
                        executable.instructions.len()
                    );
                }
                Ok(())
            }
            Err(error) => Err(sdk_error(4, error, json)),
        },
        "build" => match build(&source) {
            Ok(executable) => {
                let output = serde_json::to_string_pretty(&executable)
                    .map_err(|error| (5, format!("serialization failed: {error}")))?;
                println!("{output}");
                Ok(())
            }
            Err(error) => Err(sdk_error(4, error, json)),
        },
        "disasm" => match disassemble(&source) {
            Ok(listing) => {
                print!("{listing}");
                Ok(())
            }
            Err(error) => Err(sdk_error(4, error, json)),
        },
        "estimate" => match estimate(&source, limits) {
            Ok(plan) => {
                if json {
                    println!(
                        "{}",
                        serde_json::to_string_pretty(&plan)
                            .map_err(|error| (5, error.to_string()))?
                    );
                } else {
                    println!("Clio execution plan");
                    println!("Virtual qubits: {}", plan.qubits);
                    println!("Shots: {}", plan.shots);
                    println!("Raw state memory: {} bytes", plan.raw_state_bytes);
                    println!(
                        "Estimated total memory: {} bytes",
                        plan.estimated_total_bytes
                    );
```

```
                    println!("Status: ADMITTED");
                }
                Ok(())
            }
            Err(error) => Err(sdk_error(6, error, json)),
        },
        "run" | "trace" => match execute(&source, limits) {
            Ok(result) => {
                if json || command == "trace" {
                    println!(
                        "{}",
                        serde_json::to_string_pretty(&result)
                            .map_err(|error| (5, error.to_string()))?
                    );
                } else {
                    println!("Program: {}", result.source_sha256);
                    println!("Shots: {}", result.shots);
                    println!("Measurement counts:");
                    for (outcome, count) in result.measurement_counts {
                        println!("  {outcome}: {count}");
                    }
                    println!("Status: {:?}", result.final_state.status);
                    println!("Trace events: {}", result.trace.events.len());
                }
                Ok(())
            }
            Err(error) => Err(sdk_error(7, error, json)),
        },
        "inspect" => match inspect(&source, limits) {
            Ok(result) => {
                let snapshots = result
                    .trace
                    .events
                    .iter()
                    .filter_map(|event| {
                        event.state_snapshot.as_ref().map(|snapshot| {
                            (event.program_counter, event.mnemonic.as_str(), snapshot)
                        })
                    })
                    .collect::<Vec<_>>();
                println!(
                    "{}",
                    serde_json::to_string_pretty(&snapshots)
                        .map_err(|error| (5, error.to_string()))?
                );
                Ok(())
            }
            Err(error) => Err(sdk_error(7, error, json)),
        },
        "observe" => match observe(&source, limits) {
            Ok(report) => {
                println!(
                    "{}",
                    serde_json::to_string_pretty(&report)
                        .map_err(|error| (5, error.to_string()))?
                );
                Ok(())
            }
            Err(error) => Err(sdk_error(7, error, json)),
        },
        "validate" => match validate(&source, limits) {
            Ok(report) => {
                println!(
                    "{}",
                    serde_json::to_string_pretty(&report)
                        .map_err(|error| (5, error.to_string()))?
                );
                Ok(())
            }
            Err(error) => Err(sdk_error(8, error, json)),
        },
        _ => Err((
            2,
            format!(
                "unknown command `{command}`; executable commands are check, build, disasm, estimate, run, trace, inspect, observe, and
                  validate"
            ),
        )),
    }
}

fn parse_limits(arguments: &[String]) -> Result<ExecutionLimits, (u8, String)> {
    let mut limits = ExecutionLimits::default();
    let mut index = 2;
    while index < arguments.len() {
        match arguments[index].as_str() {
            "--json" => index += 1,
            "--instruction-limit" => {
                let value = arguments.get(index + 1).ok_or_else(|| {
                    (
                        2,
                        "--instruction-limit requires a positive integer".to_owned(),
                    )
                })?;
                limits.max_instructions = value
                    .parse::<u64>()
                    .ok()
                    .filter(|value| *value > 0)
                    .ok_or_else(|| (2, "invalid --instruction-limit".to_owned()))?;
                index += 2;
            }
            "--time-limit-ms" => {
                let value = arguments.get(index + 1).ok_or_else(|| {
                    (
                        2,
                        "--time-limit-ms requires a nonnegative integer".to_owned(),
                    )
```

```
                })?;
                limits.max_execution_millis = value
                    .parse::<u64>()
                    .map_err(|_| (2, "invalid --time-limit-ms".to_owned()))?;
                index += 2;
            }
            option if option.starts_with('-') => {
                return Err((2, format!("unknown option `{option}`")));
            }
            _ => index += 1,
        }
    }
    Ok(limits)
}

fn sdk_error(code: u8, error: SdkError, json: bool) -> (u8, String) {
    if json {
        let details = match &error {
            SdkError::Validation(diagnostics) => {
                serde_json::to_value(diagnostics).unwrap_or_else(|_| serde_json::json!([]))
            }
            _ => serde_json::json!([]),
        };
        (
            code,
            serde_json::json!({
                "ok": false,
                "code": error.code(),
                "error": error.to_string(),
                "diagnostics": details,
            })
            .to_string(),
        )
    } else if let SdkError::Validation(diagnostics) = error {
        let rendered = diagnostics
            .into_iter()
            .map(|diagnostic| {
                let location = diagnostic.span.map_or_else(String::new, |span| {
                    format!(" at {}:{}", span.line, span.column)
                });
                let help = diagnostic
                    .help
                    .map_or_else(String::new, |help| format!("\n  help: {help}"));
                format!(
                    "error[{}]{location}: {}{help}",
                    diagnostic.code, diagnostic.message
                )
            })
            .collect::<Vec<_>>()
            .join("\n");
        (code, rendered)
    } else {
        (code, error.to_string())
    }
}

fn print_help() {
    println!("Clio VQPU foundation CLI");
    println!(
        "Usage: clio <check|build|disasm|estimate|run|trace|inspect|observe|validate> <program.clio> [options]"
    );
    println!("Options: --json --instruction-limit N --time-limit-ms N");
    println!("       clio <help|version>");
    println!("This internal build executes the verified ISA subset documented in Clio Spec.");
}
```